

分类号 TP302

学号 23023014

UDC 004.8

密级 公开

工学硕士学位论文

面向异构 GPU 集群的多模态大语言模型
并行训练技术研究

硕士生姓名 范锦山

学科专业 电子信息

研究方向 并行智能计算

指导教师 李东升 研究员

协助指导教师 赖志权 副研究员

国防科技大学研究生院

二〇二六年三月

Parallel Training Strategy for Training Multimodal Large Language Models on Heterogeneous GPU Clusters

Candidate: Jinshan Fan

Supervisor: Prof. Dongsheng Li

Associate Supervisor: Prof. Zhiquan Lai

A dissertation

Submitted in partial fulfillment of the requirements

for the degree of Master of Engineering

in Computer Technology

Graduate School of National University of Defense Technology

Changsha, Hunan, P. R. China

March, 2026

独 创 性 声 明

本人声明所呈交的学位论文是我本人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表和撰写过的研究成果，也不包含为获得国防科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示谢意。

学位论文题目： 面向异构 GPU 集群的多模态大语言模型并行训练技术研究

学位论文作者签名： _____ 日期： _____ 年 _____ 月 _____ 日

学位论文版权使用授权书

本人完全了解国防科技大学有关保留、使用学位论文的规定。本人授权国防科技大学可以保留并向国家有关部门或机构送交论文的复印件和电子文档，允许论文被查阅和借阅；可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。

（保密学位论文在解密后适用本授权书。）

学位论文题目： 面向异构 GPU 集群的多模态大语言模型并行训练技术研究

学位论文作者签名： _____ 日期： _____ 年 _____ 月 _____ 日

作者指导教师签名： _____ 日期： _____ 年 _____ 月 _____ 日

目 录

摘 要	i
Abstract	ii
第一章 绪论	1
1.1 研究背景	1
1.1.1 训练多模态大语言模型面临的挑战	1
1.1.2 异构 GPU 集群下的并行训练挑战	4
1.2 国内外研究现状	7
1.2.1 多模态大语言模型并行训练策略研究现状	7
1.2.2 异构 GPU 集群并行训练策略研究现状	8
1.3 研究内容与贡献	10
1.3.1 MMoH: 多模态大语言模型在异构 GPU 集群上的混合并行 ..	10
1.3.2 HR-MMoH: 多模态大语言模型在异构 GPU 集群上的重计算	11
1.4 论文组织结构	11
第二章 相关工作	13
2.1 多模态大语言模型	13
2.1.1 多模态大语言模型的模型架构	13
2.1.2 多模态大语言模型的训练数据	14
2.1.3 多模态大语言模型的训练过程	16
2.2 并行与分布式训练技术	17
2.2.1 数据并行与模型并行	17
2.2.2 零冗余优化器 (ZeRO)	19
2.3 异构集群上的大模型并行训练	20
2.3.1 异构 GPU 集群下的并行训练策略	21
2.3.2 异构 GPU 集群并行训练的工业实践	22
2.4 重计算显存优化技术	23
2.4.1 重计算基本原理与分类	23
2.4.2 重计算的主流实现及其局限性	25
2.5 本章小结	25
第三章 MMoH: 多模态大语言模型在异构 GPU 集群上的高效混合并行	27
3.1 引言	27
3.1.1 问题与动机	27

3.1.2	MMoH 框架概览	30
3.2	异构设备映射与复合粒度模型划分	32
3.2.1	需求驱动的设备拓扑识别	33
3.2.2	复合粒度抽象	34
3.2.3	冻结感知的模型划分	36
3.3	提前终止流水线调度策略	37
3.3.1	冗余识别与剪枝规则	38
3.3.2	调度流程与实现细节	39
3.3.3	对成本模型与划分策略的影响	40
3.3.4	实现要点	41
3.4	实验验证	41
3.4.1	实验设置	42
3.4.2	实验结果	44
3.4.3	实验分析	45
3.5	本章小结	47
第四章	HR-MMoH: 多模态大语言模型在异构 GPU 集群上的重计算策略	49
4.1	引言	49
4.1.1	问题与动机	49
4.1.2	HR-MMoH 框架概览	50
4.2	异构重计算	50
4.2.1	现有的重计算策略与定量分析	50
4.2.2	异构重计算策略设计	56
4.3	实验验证	60
4.3.1	实验设置	60
4.3.2	实验结果	61
4.3.3	实验分析	65
4.4	本章小结	65
第五章	总结与展望	67
5.1	全文工作总结	67
5.1.1	主要创新点	67
5.1.2	研究的局限性	68
5.2	未来工作展望	68
5.2.1	理论深化	69
5.2.2	应用拓展	69

致谢	71
参考文献	73
作者在学期间取得的学术成果	83
附录 A 本文中英文对照表	85

表 目 录

表 2.1	多模态大语言模型训练中常用的部分数据集规模示意	15
表 3.1	实验用异构 GPU 集群配置	43
表 3.2	实验用 MLLM 模块配置（模型参数）	43
表 3.3	不同冻结场景下的模型划分对比。	45
表 4.1	Megatron-LM 重计算相关配置参数	52
表 4.2	Megatron selective 粒度下可配置的子模块	53
表 4.3	定量分析小节主要符号说明	54
表 4.4	单层 Transformer 中间激活量	56
表 A.1	本文主要术语中英文对照表	85

图 目 录

图 1.1 多模态大语言模型的核心组成部分	3
图 1.2 NVIDIA GPU 架构与代表性产品演进	5
图 1.3 使用 Megatron 训练多模态大语言模型	9
图 2.1 MLLM 多阶段训练中典型模块冻结与更新关系示意	17
图 2.2 数据并行示意	18
图 2.3 张量并行示意	18
图 2.4 流水线并行示意	19
图 2.5 专家并行 (EP) 中门控路由与专家分布示意	20
图 2.6 零冗余优化器 ZeRO 示意	20
图 2.7 重计算 (激活检查点) 示意	23
图 3.1 主流 MLLMs 的编码器—投影层—LLM 结构及冻结方式。	28
图 3.2 LLM 与 MLLM-3B 各层计算开销与显存占用的变化趋势 (NF 为不冻结, FB 为 freeze-both)。	29
图 3.3 不同并行策略在两种冻结场景下训练 MLLM-3B 的迭代时间, MMoH1/MMoH2 为 MMoH 规划器针对各场景得到的策略。	30
图 3.4 MMoH 框架总览。	31
图 3.5 复合粒度抽象示意。(a) 编码器: 将连续多层合并为编码器层块; (b) LLM Backbone: 将单层拆分为注意力块与 FFN 块, 不增加流水级间通信量。	35
图 3.6 提前终止调度器效果示意。	40
图 3.7 提前终止在其它流水线调度机制上的适用性。上: Chimera (双向流水线); 下: Hanayo (波式流水线)。	42
图 3.8 四种冻结场景下 MMoH 与基线在两种异构集群上的训练吞吐对比。柱越长表示吞吐越高, OOM 表示显存溢出。	44
图 3.9 MMoH 与 Whale、Espresso 在 MLLM-12B 上、两种冻结设置下的负载均衡对比。分布越集中表示划分越均衡, 柱高越低表示迭代时间越短。	46
图 3.10 提前终止调度器消融。Espresso+ 为接入该调度器的 Espresso, MMoH-为去掉该调度器的 MMoH。	47
图 4.1 HR-MMoH 框架总览及其与 MMoH 的衔接关系。MMoH 负责划分与冻结感知调度; HR-MMoH 在各流水级上配置差异化的重计算策略, 以进一步适配异构显存与算力。	51
图 4.2 注意力子层中间激活示意	54
图 4.3 MLP 子层中间激活示意	55

图 4.4 不同冻结策略下各框架 iteration time (seconds/iteration) 对比（自左至右、自上至下依次为 no-freeze、freeze-encoder、freeze-llm、freeze-both）。	62
图 4.5 不同冻结策略下各框架在 none、full、selective 及 HR-MMoH 配置族上的最短 iteration time (seconds/iteration) 对比。	63
图 4.6 不同冻结策略下 MMoH 与 HR-MMoH 的 iteration time 及配置对比（消融）。	64

摘 要

多模态大语言模型（MLLM）融合视觉、语言等模态的感知与生成能力，是当前人工智能研究的重要方向，其训练依赖分布式与并行计算。实际部署中的 GPU 集群往往由多代、多型号设备混合组成，存在显存、算力与通信的异构；而 MLLM 在结构上由编码器、投影层与 LLM 基座等异构模块组成，训练时又广泛采用分阶段冻结策略，导致各阶段计算与显存需求差异显著。现有并行策略（如 Megatron-LM、ZeRO）多假设同构集群与单一模型形态，在异构环境下易产生负载不均衡、资源利用率低及冻结场景下的冗余计算与通信等问题。

本文从流水线划分与调度、重计算优化两个层面开展研究。在划分与调度方面，提出 MMoH 框架：通过需求驱动的设备拓扑识别、复合粒度模型抽象与冻结感知的整数规划划分，在给定异构集群与 MLLM 配置下自动生成与当前训练阶段相匹配的并行策略；通过提前终止调度器在连续冻结阶段省略不必要的反向计算与阶段间通信，在保持收敛性的前提下提升吞吐。在重计算优化方面，针对现有框架中重计算采用全局统一配置、在异构集群上难以同时兼顾显存安全与计算效率的问题，提出 HR-MMoH 的异构重计算方法，并通过基于剖析的配置策略与 MMoH 形成“划分—调度—重计算”的联合优化。

在两种异构集群与两种规模 MLLM（3B、12B）上的实验表明，MMoH 在多种冻结场景下相对 Megatron-LM、Whale、Espresso 均可取得最高吞吐，最高可达约 1.77 倍加速，提前终止调度器可带来约 8%–19% 的额外增益；在第四章放大 batch 与序列长度的端到端实验中，HR-MMoH 在 no-freeze、freeze-encoder、freeze-llm、freeze-both 四种冻结场景下均取得最高最大吞吐，迭代时间相对 Espresso 缩短约 6.1%–23.1%，相对可训练子集上的 Whale 缩短约 11.0%–40.6%，从而验证了其 MMoH 协同优化的有效性。本文为异构 GPU 集群上 MLLM 的高效并行训练提供了可参考的技术路径。

关键词: 多模态大语言模型；异构 GPU 集群；流水线并行；重计算

Abstract

Multimodal large language models (MLLMs) integrate perception and generation across modalities such as vision and language, and their training relies on distributed and parallel computing. In real deployments, GPU clusters are typically composed of mixed generations and device types, resulting in heterogeneity in memory capacity, compute capability, and communication bandwidth. At the same time, MLLMs contain heterogeneous modules (e.g., encoders, projectors, and LLM backbones), and training widely adopts stage-wise freezing, which leads to substantial stage-dependent variation in compute and memory demands. Existing parallel strategies (e.g., Megatron-LM and ZeRO) are largely designed under homogeneous-cluster and uniform-model assumptions, and therefore often suffer from load imbalance, low resource utilization, and redundant computation and communication under freezing in heterogeneous settings.

This thesis studies the problem from two aspects: pipeline partitioning/scheduling and recomputation optimization. For partitioning and scheduling, we propose MMoH, which combines demand-driven device topology identification, compound-granularity model abstraction, and freeze-aware integer-programming partitioning to automatically generate parallel strategies that match the current training stage for a given heterogeneous cluster and MLLM configuration. It further introduces an early-termination scheduler to skip unnecessary backward computation and inter-stage communication in consecutive freezing stages, improving throughput while preserving convergence. For recomputation optimization, we observe that existing frameworks usually apply a globally uniform recomputation configuration, which struggles to jointly balance memory safety and computational efficiency on heterogeneous clusters. We therefore propose heterogeneous recomputation in HR-MMoH, and integrate a profiling-based configuration strategy with MMoH to form joint optimization over partitioning, scheduling, and recomputation.

Experiments on two heterogeneous clusters and two MLLM scales (3B and 12B) show that, under multiple freezing scenarios, MMoH achieves the highest throughput among Megatron-LM, Whale, and Espresso, with up to about $1.77\times$ speedup; the early-termination scheduler provides an additional gain of about 8%–19%. In Chapter 4 end-to-end experiments with enlarged batchsize and sequence length, HR-MMoH achieves the highest maximum throughput in all four scenarios (no-freeze, freeze-encoder, freeze-llm,

and freeze-both), reducing iteration time by about 6.1%–23.1% compared with Espresso and by about 11.0%–40.6% compared with Whale on the trainable subset. These results provide a practical technical path for efficient parallel training of MLLMs on heterogeneous GPU clusters.

Key Words: Multimodal large language model; Heterogeneous GPU cluster; Pipeline parallelism; Recomputation

第一章 绪论

1.1 研究背景

1.1.1 训练多模态大语言模型面临的挑战

深度神经网络（Deep Neural Network, DNN）在计算机视觉（Computer Vision, CV）、自然语言处理（Natural Language Processing, NLP）、语音识别（Speech Recognition, SR）等众多前沿领域取得了优异的效果，近年来逐步演变为诸多关键前沿技术领域的核心基础 [1]。2017 年，Vaswani 等人提出基于自注意力机制（Self-Attention）的 Transformer 架构 [2]，摒弃了传统的循环神经网络（Recurrent Neural Network, RNN）与卷积神经网络（Convolutional Neural Network, CNN）结构，由于可以并行计算而且对于长距离信息捕捉能力强，Transformer 架构迅速成为自然语言处理与跨模态建模的主流模型结构。

在自然语言处理方面，基于 Transformer 的预训练大语言模型（Large Language Models, LLMs）不断刷新在何种基准测试成绩。BERT[3] 通过双向编码与掩码预训练在语言理解类任务上表现突出；GPT-2[4]、T5[5] 等自回归（Autoregressive）或编码器 - 解码器（Encoder-Decoder）模型则在文本生成上有优势；GPT-3[6] 与后续的 GPT-4[7] 等大规模语言模型更在少样本（Few-Shot）与零样本（Zero-Shot）设定下展现出接近人类的表达能力。在计算机视觉领域，Vision Transformer（ViT）[8] 及后续的规模化工作 [9] 表明：将图像切分为块（patch）并经 Transformer 编码，即可在 ImageNet 等基准数据集上超越传统卷积神经网络。这表明 Transformer 架构的模型不仅可以处理文本，还具有处理视觉等其余模态信息的潜力。

随着单模态模型已经接近甚至超越人类水平，将语言、视觉、音频、表格等多模态信息纳入统一的模型框架进行联合表示与推理，成为当前学术界和工业界研究的新方向。多模态（Multimodality）是指可以同时处理两种或两种以上模态信息的能力。在机器学习与人工智能中，常见模态包括文本、图像、视频、音频、表格以及各类传感器数据，更贴近于人类接触到的复杂信息。多模态大语言模型（Multimodal Large Language Models, MLLMs）特指在大语言模型的基础上，融合对视觉、听觉等其它模态的感知与生成能力的一类模型 [10–12]。MLLMs 的先驱工作 CLIP[13]（Contrastive Language-Image Pre-Training）通过使用图像文本对进行训练，使得模型拥有理解图片的能力，DALL-E[14] 则与 CLIP 相反，展示了从文本生成图像的可行性。近年来，MLLMs 在架构与训练范式上发展迅速：BLIP-2[10] 提出 Q-Former 连接冻结的视觉编码器与 LLM 进行训练；InstructBLIP[15] 在此基础

上加强了视觉信息的提取能力，在多项零样本任务上取得 SOTA 结果；LLaVA[11] 与 LLaVA-1.5[16] 采用简单线性投影层实现视觉与语言模态对齐，验证了“小连接器 + 大模型”的有效性；Qwen-VL[17]、InternVL[18] 等系列多模态模型则进一步在长上下文、高分辨率与多图理解上扩展模型能力。这类模型既保留 LLMs 在语言理解、推理与生成上的优势，又能够直接处理图像、视频等多模态输入，实现跨模态交互与推理任务，因而被认为是迈向通用人工智能（Artificial General Intelligence, AGI）的潜在路径 [19, 20]。多模态大语言模型能力的拓展既依赖参数与数据规模的提升，也依赖编码器、投影层、基座模型与可选生成器等模块的协同设计：前者关系到模型可达的效果上限，后者则直接决定训练阶段中计算与显存在各模块间的分布形态。

从功能表现来看，MLLMs 可完成图像描述、视觉问答、图文对话、文档理解，以及基于文本的图像或视频生成等复杂任务，在诸多前沿领域比如自动驾驶、智能助手等场景具有广阔应用前景；在垂直领域，MLLMs 已延伸至具身智能 [21]、医疗影像理解 [22] 等。多模态大模型的相关评估常使用 VQAv2[23]、GQA[24]、MMMUI[25] 等多模态基准，对模型规模与训练数据提出了更高要求。从结构上看，MLLMs 通常包含以下核心组件，如图1.1：（1）*Modality Encoder*，负责将图像、视频等非文本输入编码为特征表示，常采用预训练的视觉模型（如 CLIP、ViT）作为 backbone；（2）*Input Projector*，将编码后的多模态特征映射到与 LLM 词嵌入对齐的语义空间，常见形式包括线性层、Q-Former[10] 等轻量模块；（3）*LLM Backbone*，即大规模语言模型主体，承担主要的理解与生成计算，参数量通常占整体模型的 70%–80%；（4）*Output Projector*（在需要多模态生成时），将 LLM 输出映射到与目标模态生成器对齐的表示空间；（5）*Modality Generator*（在需要多模态生成时），负责生成图像、音频等目标模态，常采用扩散模型（如 Stable Diffusion）等。其中 LLMs 处于 MLLMs 架构的核心，多模态能力是在其基础上进行的扩展与增强。

在大语言模型占据主体参数量的架构下，针对 LLMs 总结出的扩展定律（Scaling Laws）与数据规模规律，同样深刻影响着 MLLMs 训练的资源需求。已有实证研究表明，模型性能通常会随参数规模与训练数据规模的增加而稳定提升，这一现象被总结为“扩展定律”（Scaling Laws）[26]。Kaplan 等 [26] 的早期工作表明，在给定算力预算下，模型规模与数据规模应按幂律关系共同扩展。Hoffmann 等 [27] 进一步指出，当前大语言模型普遍“训练不足”，在计算最优意义下，模型参数量与训练 token 数应近似同比例扩展（约 20 倍 token/ 参数），该结论对预训练数据规模与模型架构选择产生了深远影响。近期工作 [28] 则从推理成本角度拓展了 Chinchilla 最优，指出在考虑部署与推理时，最优扩展策略会随推理请求规模发生变化。为追求更强性能，工业界与学术界不断推出参数量更大、数据规模

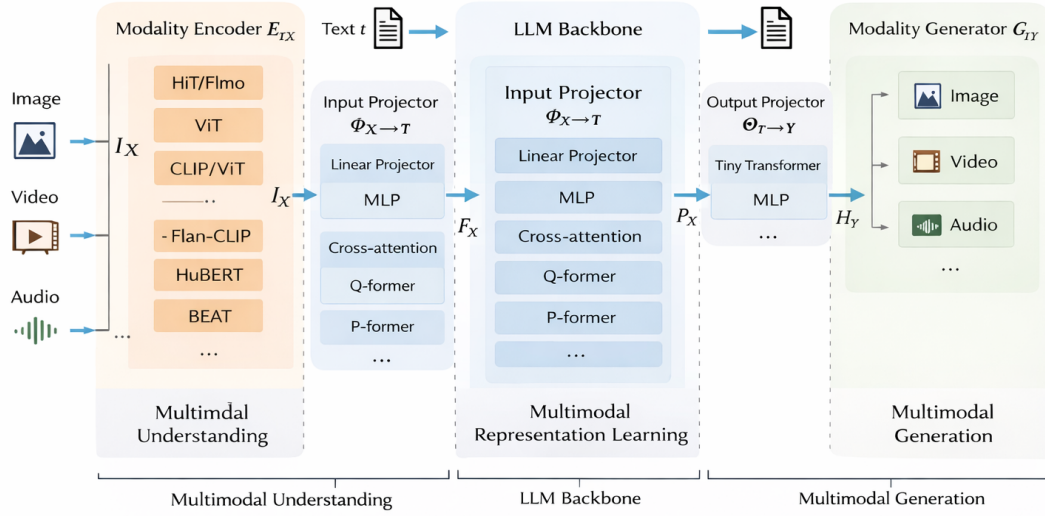


图 1.1 多模态大语言模型的核心组成部分

更大的模型，从数亿参数（如 BERT）到千亿、万亿参数（如 GPT-3、Gopher[29]、DeepSeek-V3[30]、Qwen2.5[31]、LLaMA 2/3[32, 33]、Gemma[34]、Mistral[35] 等开源与闭源 LLMs），单卡或单机 GPU 的显存与算力已无法容纳整模型的完整训练流程 [36, 37]。相应地，多模态大模型的预训练与指令微调所需的数据量也急剧增长，从数百万图文对到数十亿级样本，单机算力、存储与 I/O 同样成为瓶颈；数据质量与混合比例对下游性能同样有显著影响 [38]，长上下文与多模态联合基准 [39] 的普及进一步推高了对训练规模与效率的需求。在上述模型与数据规模的约束下，单台计算设备的显存与算力均难以独立支撑完整的训练流程。因此必须借助分布式与并行训练技术，将模型参数、优化器状态、激活值与数据分散至多台设备，通过节点间的通信协同完成大规模模型的训练 [40, 41]。

目前，支撑大规模 MLLMs 训练的主流并行策略可归纳为两大类。第一类是以计算与模型切分为核心的多维混合并行：数据并行 [42, 43] 在各设备上复制完整模型、按批次划分样本，通信集中在梯度 AllReduce，实现简单且在小模型上扩展性好；张量并行 [36, 37] 将单层内的矩阵按行或列切分到多卡，降低单卡显存并保留层内并行，是千亿级稠密模型的主流选择；流水线并行 [44, 45] 将模型按层划分为若干流水级并分配到不同设备，通过微批次与调度重叠计算与通信，缓解单设备层数过多的问题；序列并行 [36] 与专家并行 [46] 则分别在长序列注意力与混合专家（MoE）结构中沿序列维或专家维切分，与张量、流水线并行组合后可进一

步扩展模型规模。第二类是以显存优化为核心的零冗余优化器系列：ZeRO[40, 41] 将优化器状态、梯度与参数分片存于各卡，前向与反向时按需聚合，在保持数据并行通信量的同时显著降低单卡显存；ZeRO-Infinity[41] 进一步将部分状态卸载至 CPU 甚至是 NVMe 上，突破 GPU 显存墙；ZeRO++[47] 通过量化与通信重算技术减少跨节点通信量，提升多节点训练效率。上述并行策略常与混合精度训练[48]、重计算[49]、Flash Attention[50, 51] 等显存与计算优化技术结合，共同构成当前大模型与 MLLMs 训练的基础设施；PyTorch 的 FSDP[52] 与 DeepSpeed ZeRO 类似，通过全分片参数与优化器状态实现单卡显存的大幅降低。

然而，上述策略在设计时多假设同构集群与单一形态的 Transformer 模型，其划分粒度、负载均衡与通信优化均围绕“各层计算与显存相对均匀”这一前提展开；在面对异构 GPU 集群与模块异构的 MLLMs 时，负载与显存分布的不均匀性被放大，仍存在效率与可扩展性方面的挑战。在实际应用中，企业与研究机构常面临定制化多模态模型的高频微调需求，这要求训练系统具备较高的可扩展性与资源利用率。然而，受预算与硬件采购周期的限制，多数集群难以维持绝对的同构性，往往需要在现有的异构设备组合上开展预训练与微调实验。因此，提升异构 GPU 集群下的训练效率、使其逼近同构环境的表现，具有迫切的现实需求。同时，因负载不均或通信瓶颈导致的计算资源浪费不仅会延长训练周期，亦会显著增加算力成本。由此可见，针对异构集群设计高效的 MLLMs 并行训练方案，既是系统层面的技术挑战，也是落地部署中的关键环节。

1.1.2 异构 GPU 集群下的并行训练挑战

多模态大语言模型的训练效率与成本高度依赖底层硬件平台。目前，工业界与学术界普遍采用 NVIDIA GPU 作为 MLLMs 训练的主要加速器，并通过多卡、多机集群进行分布式训练。然而，NVIDIA 约每 1-2 年迭代一代计算架构并推出多款不同定位的 GPU，导致显存容量、算力与互联方式持续分化，实际部署中的集群往往由多代、多型号 GPU 混合组成，形成显存异构、算力异构与通信异构并存的局面[53, 54]。对单层计算与显存相对均匀的传统 LLMs，尚可“按层数或算力比例”划分在一定程度上缓解异构；而对编码器、投影层与 LLMs 在结构与参数量上差异显著的 MLLMs，模块间负载差异与设备能力差异叠加，使得简单比例划分难以同时兼顾负载均衡与显存安全。

图1.2 概括了近年来 NVIDIA GPU 的架构演进。2016-2017 年的 Pascal（如 P100）与 Volta（V100）奠定了数据中心 GPU 与 Tensor Core 的基础；2018 年 Turing（RTX 20 系列）引入消费级光追与 INT8 推理；2020 年 Ampere 架构推出 A100（40GB/80GB HBM2e）与消费级 RTX 30 系列（如 RTX 3090 24GB），支持

PCIe 4.0 与 NVLink 3.0, 成为当前许多实验室与云平台的主力 [55]。2022 年 Hopper 架构的 H100 (80GB/96GB HBM3) 针对大语言模型训练做了专门优化 [56], 并广泛配备 NVLink 4.0 与 NVSwitch, 单机多卡带宽可达数百 GB/s; 同年 Ada Lovelace 架构的 RTX 40 系列 (如 RTX 4090 24GB) 在消费级市场提供高性价比算力。2024 年起, Blackwell 架构的 B100/B200 等数据中心 GPU 逐步量产, 采用 HBM3e、更高带宽的 NVLink5 与 PCIe 6.0, 进一步拉大与旧代设备的性能与互联差距 [57]。按 NVIDIA 已披露的路线图, 后续还将推出 Rubin (约 2026 年)、Rubin Ultra (约 2027 年), 以及面向推理与能效深度优化的 Feynman 架构 (约 2028 年), 后者将采用下一代 HBM 与 3D 堆叠内存等技术, 进一步凸显代际间显存与互联的差异。与此同时, 不同代际与型号在 PCIe 版本 (3.0/4.0/5.0/6.0)、是否配备 NVLink/NVSwitch、以及 InfiniBand 与 RoCE 等节点间网络上的差异, 使得通信带宽与延迟在集群内也存在异构性。

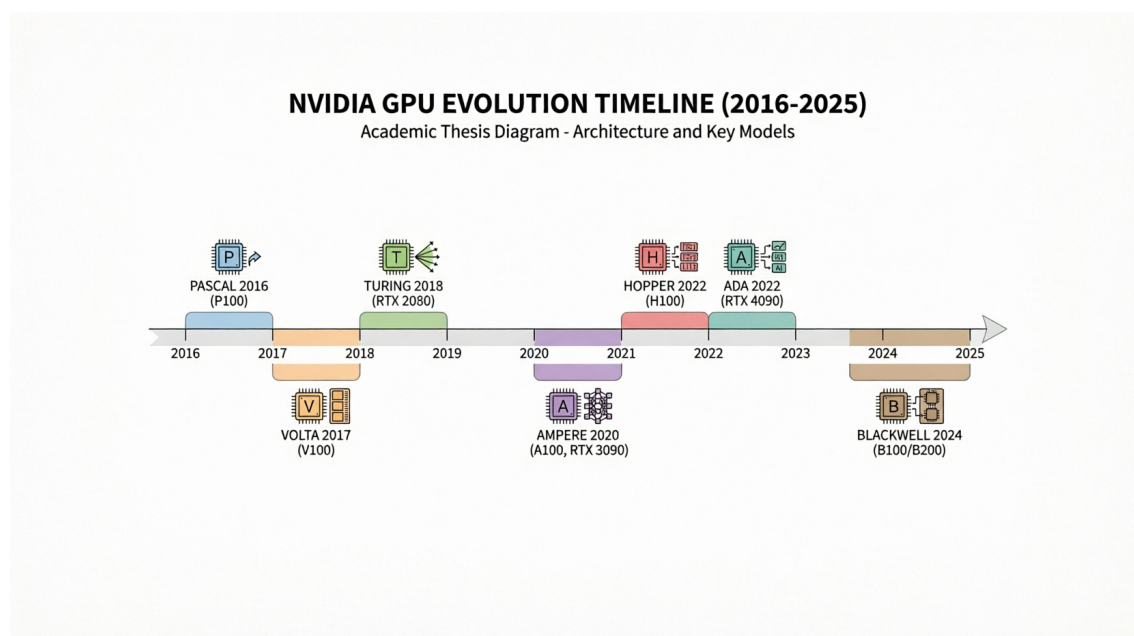


图 1.2 NVIDIA GPU 架构与代表性产品演进

在大多数高校、研究所与中小企业的实际环境中, 受预算与采购周期限制, 集群常同时包含多代 GPU (例如部分 A100 或 H100 与大量 RTX 3090/4090、甚至更早的 V100/RTX 2080), 单机内也可能混用不同显存与互联配置, 难以做到“全集群同构”。而主流并行训练框架 (如 Megatron-LM、DeepSpeed) 在设计时多假设同构设备与统一拓扑 [36, 40], 在异构集群上直接使用往往导致: 负载无法按算力与显存合理分配、通信成为瓶颈或部分高端卡空闲、以及因“木桶效应”整体效率下降 [53, 54]。因此, 在显存、算力与通信三重异构的 GPU 集群上高效训练

MLLMs, 是学术界和工业界共同面临且亟待解决的问题。

从系统层面看, 异构带来的问题不仅在于单卡能力差异, 还在于拓扑与调度: 节点内 NVLink 与节点间 InfiniBand 的带宽可相差一个数量级以上, 若流水线或张量并行的划分未考虑链路差异, 跨节点通信极易成为瓶颈; 同时, MLLMs 各阶段对编码器、投影层与 LLMs 的冻结策略不同, 导致同一模型在不同训练阶段下各层的计算与显存占比发生明显变化, 若沿用固定划分与调度, 难以在整轮训练中保持负载均衡。因此, 面向异构集群的 MLLMs 训练需要同时考虑硬件拓扑、模型结构与多训练阶段, 通过自适应的划分与高效的调度使各设备利用率最大化。

除设备侧算力、显存与通信差异外, 模型结构异构也会显著改变并行划分与负载形态。典型 MLLM 中, *Modality Encoder* 多采用视觉主干 (如 CLIP、ViT 等), 将图像或视频编码为特征; *LLMs Backbone* 常为 GPT 类或 Llama 类等大语言模型 [33], 参数量往往占整模型的约 70%–80%; 可选的 *Modality Generator* 则可能是扩散模型 (如 Stable Diffusion[58]) 等, 用于生成图像、视频或音频。LLM 部分通常由大量同构 Transformer 层堆叠; Encoder 与 Generator 往往融合卷积、U-Net[59] 或扩散去噪网络等不同算子; Input/Output Projector 则可能是线性层、小型 Transformer 或 Q-Former[10] 等轻量连接器。与“单层形态相对一致、逐层堆叠”的传统稠密 LLM 相比, MLLM 各模块在算子类型、参数量与激活体积上差异显著, 同一划分策略在不同 GPU 上呈现的时间与显存开销不再近似齐次; 在异构集群上, 设备差异与模块差异叠加, 负载不均与显存风险更易被放大。

多模态大语言模型的训练流程也与纯文本 LLM 常见做法不同。得益于视觉、语言与生成等领域成熟的预训练成果, 当前主流范式大量复用已有强模型, 并在训练中通过冻结与解冻控制可训参数规模。BLIP-2[10] 较早系统性地采用“冻结视觉编码器与 LLM、主要训练 Q-Former 等连接器”的策略完成跨模态对齐。此后, 这种基于冻结与局部更新的策略被广泛应用于各类多模态大语言模型中, 如近期的 NextGPT[60]、DreamLLM[61] 等工作。此外, 多模态大语言模型的训练通常被划分为多个阶段 (如模态对齐、指令微调等), 不同阶段针对特定能力解冻不同模块。当某一模块被冻结时, 其反向传播的计算量会大幅下降; 且优化器亦无需为冻结参数维护一阶与二阶矩等状态变量 (如使用 Adam[62] 优化器时), 这显著降低了训练过程中的显存压力。但在异构 GPU 集群中, 这种由模块冻结带来的算力、显存与通信需求的时变性, 反而放大了各设备间的负载不均。此外, 云上与混合部署中的弹性训练与按需资源供给, 使得集群内设备型号与拓扑更加多样, 对异构感知的划分与调度、以及训练成本与性能的联合优化提出了进一步要求 [63–65]。

硬件层面的设备异构与模型层面的模块异构、分阶段冻结交织, 构成了当前

有限资源下高效训练的核心难点。为此，本课题拟结合现有并行策略，深入分析实际 GPU 集群中的异构场景，基于 Megatron 等主流并行训练框架，针对 MLLMs 的结构特点以及集群内显存、算力与带宽的异构性，研究细粒度的混合并行技术和重计算显存优化技术，旨在提出一套面向异构复杂环境的自适应并行训练方案来缓解异构集群下训练 MLLMs 效率的问题。

1.2 国内外研究现状

本节从文献角度归纳与本文密切相关的研究脉络：先概述 MLLM 的规模演进、数据需求与主流并行技术栈及其同构假设局限，再综述面向异构 GPU 集群的并行与调度工作，并与绪论中的问题动机相呼应。

1.2.1 多模态大语言模型并行训练策略研究现状

近年来，基座语言模型、视觉模型与多模态大语言模型持续向更大参数规模与更大数据规模演进。Kaplan 等 [26] 通过大量实验归纳出扩展定律（Scaling Law）：模型性能随模型参数量与训练数据量的幂律关系持续提升，这为“做大模型、用大数据”提供了理论依据；Chinchilla[27] 进一步表明，在计算最优意义下，模型规模与训练 token 数应同比例扩展，后续研究 [28] 从推理成本角度拓展了最优缩放策略。提升大模型性能的两条主要途径是增大模型参数与扩大数据规模 [1]：通过提升参数量可增强对复杂场景的拟合与泛化能力；通过扩大数据规模可使模型学习到更充分的特征表示，更好地适应复杂应用场景；参数高效微调（如 LoRA[66]、QLoRA[67]）则在资源受限下广泛用于 MLLM 的适配与部署 [68]。

自 Transformer[2] 提出以来，语言模型规模迅猛增长：从 BERT[3] 的亿级参数，到 GPT-2[4]、T5[5] 的十亿级，再到 GPT-3[6] 的千亿级与 GPT-4[7] 等闭源模型，以及 DeepSeek-V3[30]、Qwen2.5[31] 等开源千亿级模型。多模态大语言模型在此基础上融合视觉编码器与语言模型，代表性工作包括 BLIP-2[10]（Q-Former 连接视觉与语言）、InstructBLIP[15]（指令感知视觉特征）、LLaVA/LLaVA-1.5[11, 16]（线性投影对齐）、Flamingo[12]（交错图文与 gated 交叉注意力）、Qwen-VL[17]、InternVL[18]、MiniGPT-4[19] 等，其参数量从数十亿到数百亿不等，且训练数据从数百万图文对到数十亿级样本（如 LAION-5B[69]）；相关综述从架构、对齐策略、数据与效率等角度对 MLLMs 进行了系统梳理 [20, 70, 71]。单卡或单机在显存与算力上已无法容纳完整模型与训练流程 [36, 37]，并行与分布式训练成为训练大模型与 MLLM 的必由之路 [40, 41]。

训练数据规模同样持续扩大：图像方面，Open Images[72]、COCO[73] 等需 TB 级存储；LAION-5B 等开放图文对达数十亿样本；文本方面，Common Crawl、

The Pile[74] 等为 LLM 预训练提供数百 GB 至数万亿 token 量级语料；视频方面，YouTube-8M[75]、WebVid[76] 等基准需 PB 级存储与高吞吐数据管线；多模态理解与推理则依赖 VQAv2[23]、TextVQA[77] 等数据集与长上下文基准 [39]。在单机无法存储完整数据集与模型的前提下，并行训练（数据并行、模型并行及其组合）成为必然选择。

在并行策略方面，工业界与学术界已形成以数据并行、张量并行、流水线并行及 ZeRO 等为核心的技术栈，并被 Megatron-LM、DeepSpeed 等框架广泛应用于千亿级参数模型训练 [36, 40, 41]。Galvatron[1] 针对 Transformer 在多 GPU 上的自动并行进行了系统探索，将策略搜索与编译优化结合；Colossal-Auto[49] 等将自动并行策略搜索与重计算统一自动化，为大规模与异构环境下的联合优化提供了参考。然而，上述框架在设计时多假设同构 GPU 集群与单一形态的 Transformer 模型，对 MLLMs 的模块异构（编码器、投影层、LLM、生成器结构各异）与分阶段冻结考虑不足；在实际部署中，集群往往由多代、多型号 GPU 混合组成，存在显存、算力与通信的异构 [53, 54]，直接使用传统并行策略易出现负载不均、通信瓶颈与“木桶效应”。因此，面向异构 GPU 环境设计多模态大模型并行训练方法与重计算策略，实现自感知、自适应、细粒度的划分与调度，对提高算力利用率、充分利用显存空间十分必要。而要支撑上述目标，需依托现有并行训练的形态，并在此基础上针对异构集群与 MLLM 做专门适配。

1.2.2 异构 GPU 集群并行训练策略研究现状

并行训练是当前工业界与学术界训练多模态大语言模型的主要手段，主流做法可概括为两类：以计算与张量切分为核心的多维混合并行，以及以显存分片为核心的零冗余优化器（ZeRO）系列。

多维混合并行通过在不同维度上划分模型或数据以降低单机负载。数据并行 [42, 43] 在各设备上复制完整模型、按批次划分样本，通信集中在梯度 AllReduce，实现简单且扩展性好。张量并行 [36, 37]（如 Megatron-LM）将单层内矩阵按行或列切分到多卡，降低单卡显存，是千亿级稠密模型的主流选择。流水线并行将模型按层划分为若干阶段并分配到不同设备：GPipe[44] 通过微批次与多阶段调度减少流水线气泡；PipeDream[45] 提出 1F1B 等调度进一步压缩空闲时间。此外，序列并行 [36]、专家并行 [46] 等与张量、流水线并行组合后可进一步扩展模型规模。NVIDIA 的 Megatron-LM[36, 37] 联合使用张量、流水线与数据并行，已支撑在数千张 GPU 上训练千亿至万亿级参数的大语言模型；Korthikanti 等 [78] 针对超大 Transformer 提出序列并行与选择性重计算，与张量并行协同以降低激活显存。零冗余优化器 ZeRO[40, 41] 在数据并行基础上将优化器状态、梯度与参数分

片存于各卡、按需聚合，显著降低单卡显存；ZeRO-Infinity[41] 将部分状态卸载至 CPU/NVMe；ZeRO++[47] 通过量化与通信重算减少跨节点通信量。

然而，上述策略多假设同构设备与统一拓扑。Megatron-LM 在设计时未考虑设备异构与模型异构；在实际异构环境下训练 MLLMs 时，往往将不同型号、显存与带宽的 GPU 当作同构设备统一配置，其典型并行策略如图1.3所示。对于 MLLM 中结构各异的 Modality Encoder、LLM Backbone、Projector 与 Modality Generator，Megatron 缺乏针对性的细粒度划分与映射，编码器与生成器的并行方式通常被动跟随 LLM 主干划分，难以根据各子模块的算力与显存需求及不同 GPU 能力做自适应分配，导致显存小、算力弱或通信慢的 GPU 成为瓶颈，整体利用率与训练效率下降 [53, 54]。

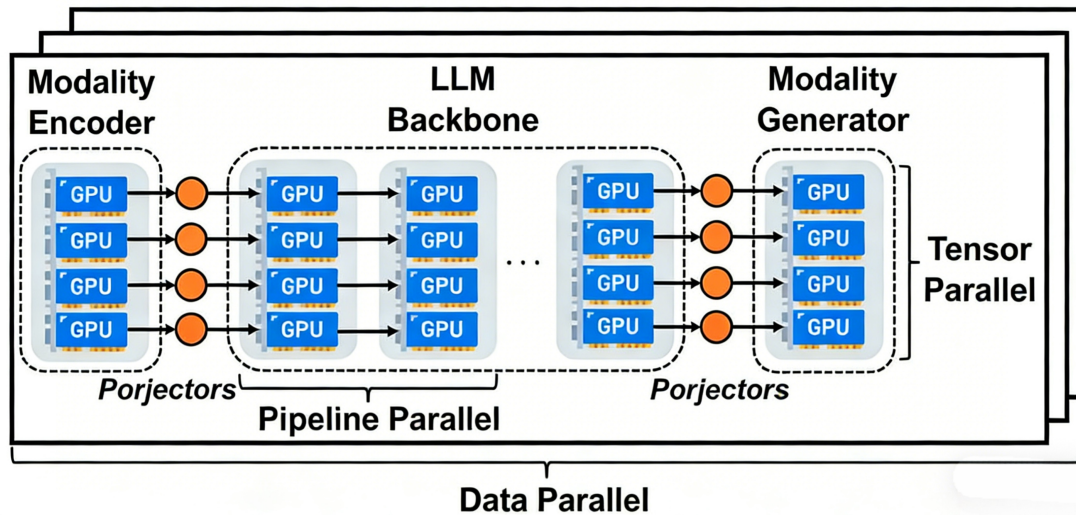


图 1.3 使用 Megatron 训练多模态大语言模型

针对异构 GPU 集群的并行训练，已有不少研究。HetPipe[54] 较早将流水线并行与数据并行结合，通过虚拟工作节点（VW）在异构设备间分配负载，但未显式建模设备排列与通信拓扑，也未考虑 MLLM 的模块异构与冻结策略。Whale[53] 针对显存与算力异构提出基于硬件感知的负载均衡方法，按比例对各设备分配不同大小的模型分片与批次，但未考虑网络异构，划分粒度相对粗糙。AMP[79] 在异构环境下为大模型自动搜索并行策略，同时考虑模型异构与设备异构，通过开销模型与动态规划求解近似最优配置，但划分粒度为全局粗粒度，对通信与显存估计的准确性有限。近年来，面向异构集群的分布式训练系统与调度方法受到持续关注。HetHub[80] 支持 AMD、NVIDIA 等多厂商异构 GPU 集群，将通信器、性能预测器与自动并行统一规划；HexiScale[81] 将数据、流水线与张量并行的非对称划分形式化，采用层次图划分算法在异构环境下分配计算；Espresso[82] 面向成本

效益的异构 GPU 资源供给与阶段放置, 集成 Profiler、GPU Allocator、Stage Placer 与 Performance Estimator; 上述工作多为通用 LLM 或同构扩展设计, 对 MLLM 的模块异构、分阶段冻结与显存一算力一通信三重异构的联合优化仍不足。

ZeRO[40, 41] 的梯度 / 参数 All-Gather 等通信在跨节点异构网络中易成为瓶颈; MiCS[85] 对异构网络下的高带宽链路利用仍有不足; ZeRO++[47] 的量化与通信重算可能引入精度与通用性代价; AMSP[86] 等侧重跨节点通信削减, 对集群内存存储、算力与带宽的细粒度异构利用仍有限。因此, 在异构 GPU 集群上高效训练 MLLMs, 仍需在 DeepSpeed 与 Megatron 等现有框架基础上, 引入对设备能力与模型结构的感知与自适应优化。

1.3 研究内容与贡献

1.3.1 MMoH: 多模态大语言模型在异构 GPU 集群上的混合并行

针对现有并行策略在异构集群适配上的局限性, 本课题以实际 GPU 集群中普遍存在的显存、算力和网络异构为切入点, 研究细粒度的混合自动并行优化技术。提出适用于异构 GPU 环境的 MLLMs 自适应并行训练方案, 以期在复杂集群下缓解负载不均与通信瓶颈, 提升整体训练效率。

针对异构环境下显存利用不均、通信链路差异导致的带宽瓶颈、算力差异导致的设备利用率下降、模块结构差异导致的算力分配困难, 以及分阶段冻结引起的计算与显存需求随时间变化等问题, 本课题提出一种面向异构设备与异构模型结构的多模态大语言模型计算图层级划分方法, 通过为不同属性的计算设备分配不同大小的部分模型, 实现计算资源、存储空间、通信带宽的充分利用, 争取最大限度提高训练速度。

MLLM 训练通常划分为多阶段 (如对齐、指令微调等), 各阶段对 Encoder、Projector、LLM 等模块采取不同冻结策略: 冻结模块的反传与优化器状态维护显著减少, 而未冻结模块仍承担主要计算与通信。阶段切换后, 若仍沿用固定流水线划分与 1F1B 等常规调度, 易出现阶段间耗时失衡、流水线气泡居高不下的现象, 并可能伴随与冻结状态不相称的冗余反传与 P2P 通信。

针对上述问题, 本课题提出面向多模态大语言模型的提前终止流水线调度策略。该策略感知各训练阶段的冻结格局, 与划分方案协同调整; 并在连续冻结的流水级上省略不必要的反向计算与阶段间通信, 在保持梯度语义一致的前提下提高多卡流水线的有效利用率与训练吞吐。

1.3.2 HR-MMoH: 多模态大语言模型在异构 GPU 集群上的重计算

重计算通过在前向时丢弃部分中间激活、在反向时按需重算，以计算换显存，是支撑大模型与 MLLMs 在有限显存下训练的重要手段。然而，现有重计算策略多采用全局统一的配置（如对全部层或按固定间隔插入检查点），未考虑异构集群中设备间显存与算力的差异：显存紧张、算力相对充裕的设备适合更激进的检查点以释放显存；显存宽裕、算力紧张或通信繁忙的设备则不宜过度重算，否则会加重计算或通信瓶颈。此外，MLLMs 各子模块（Encoder、LLM、Projector、Generator）的层数、每层激活体积与计算量各不相同，在同一设备上对不同模块采用相同检查点策略也难以达到最优。

本课题针对上述不足，提出一种面向异构设备与异构模型的重计算优化方法。该方法在给定层级划分与设备映射的基础上，以各设备的显存上限以及算力为约束，为不同设备上的不同模型阶段分别搜索或配置差异化的检查点策略，在满足显存不溢出的前提下，最小化因重计算带来的额外时间开销，并兼顾流水线不同流水级的负载均衡。通过将重计算与前述的复合粒度划分、流水线调度相结合，实现计算、存储与通信在异构环境下的联合优化，进一步提升 MLLMs 在异构 GPU 集群上的训练效率。

1.4 论文组织结构

本论文共分为五章，具体结构如下：

第一章为绪论。在规模扩展与并行训练需求、设备与拓扑异构、模型结构与分阶段冻结三个层面阐述研究背景与意义，归纳国内外相关研究现状，并概述本文的研究内容与核心贡献。

第二章为相关工作。从 MLLMs 架构特点、分布式训练基础、异构 GPU 并行策略以及重计算显存优化等方面梳理现有研究的发展脉络与局限性。

第三章介绍本文提出的 MMoH 框架，重点阐述需求驱动的设备拓扑识别、复合粒度模型抽象、冻结感知划分及提前终止流水线调度方法，并给出实验评估。

第四章介绍 HR-MMoH 异构重计算优化，详细说明如何在异构计算资源与不同模型模块间实现重计算策略的细粒度自适应配置并进行实验验证。

第五章为总结与展望。对全文工作进行回顾，并探讨后续可能的研究方向。

第二章 相关工作

本章对本文涉及的相关技术进行展开叙述，全章按以下顺序组织：(1) MLLMs 的架构模块、训练数据规模与多阶段冻结训练范式相关研究；(2) 数据、张量、流水线、序列与专家并行及 ZeRO 等主流并行训练方法；(3) 异构集群上的大模型并行训练与调度相关研究；(4) 重计算策略分类、框架实现与同构全局配置的局限。在此基础上收束到本文关注点：在异构设备 + 模块异构 + 分阶段冻结叠加时，现有同构导向划分、调度与统一重算策略仍难以同时保证负载均衡、显存安全与吞吐，从而为第三、四章 MMoH 与 HR-MMoH 提供研究空间。

2.1 多模态大语言模型

2.1.1 多模态大语言模型的模型架构

多模态大语言模型 (MLLMs) 将视觉、语言与 (可选) 生成能力统一到以大规模语言模型为核心的异构架构中。其典型组成如图1.1所示，下文结合对各模块的代表性实现做详细展开。

Modality Encoder 即模态编码器，负责将图像、视频、音频等非文本输入编码为高维特征表示，以便与语言模型的词嵌入空间统一。实践中多采用在大规模图文或视频等多模态数据上预训练好的视觉模型作为编码器：CLIP[13] 通过对比学习视觉信息，被 LLaVA[11]、MiniGPT-4[19] 等广泛采用；ViT[8] 及其更大规模的变体 [9] 则将图像切分为多个小的 patch 后经 Transformer 编码，让其它模态信息的处理也使用了与语言模型相同的架构，在 Flamingo[12]、InternVL[18] 等工作中用作视觉 backbone。InternVL[18] 将视觉基础模型规模扩展至 60 亿参数，并在多图、长文本与文档理解上取得领先；Qwen-VL[17] 是阿里巴巴最先进的多模态模型，支持百万像素级高分辨率与多轮对话，在 DocVQA、ChartQA 等基准上表现突出。模态编码器输出通常为序列形式的特征，再经后续模块映射到 LLM 空间，从而完成视觉等其它模态数据与语言模型的对齐的第一步 [87]。

Input Projector 将编码器输出的多模态特征映射到与 LLMs 词嵌入维度与语义空间对齐的表示，使 LLMs 能够将视觉等信息与文本统一处理。常见实现包括线性层、轻量 Transformer 层、以及 BLIP-2[10] 提出的 Q-Former (通过可学习的 query 从编码器特征中抽取与任务相关的表示)；InstructBLIP[15] 在此基础上引入指令感知的 Q-Former，根据用户指令从图像中抽取与任务更相关的特征，在 ScienceQA 等零样本任务上显著优于 BLIP-2。该模块参数量远小于 LLM，常与编码器一起在预训练或对齐阶段更新。

LLM Backbone 即大规模语言模型主体，承担主要的语言理解与生成计算，参数量通常占整个 MLLM 的 80%–90%[10, 11]。基座模型多选用已在海量文本上预训练好的 Transformer 模型，如 GPT 系列 [4, 6]、LLaMA[33] 等；在资源受限或强调高效微调时，常对 LLMs 进行冻结训练或仅微调部分层。

Output Projector 与 Modality Generator 用于更加复杂的多模态生成任务（如图生图、文生图、文生视频、文生音频等）：前者将 LLMs 的输出映射到模态生成器可接受的表示空间，后者根据该表示生成图像、视频或音频。生成器多采用扩散模型，如 Stable Diffusion[58] 中基于 U-Net[59] 的潜在扩散结构。Next-GPT[60]、DreamLLM[61] 等工作在此基础上实现了任意模态到任意模态的生成与转换，极大程度上扩展了多模态技术的应用范围。

在训练策略方面，BLIP-2[10] 率先系统性地采用“冻结 Encoder + 冻结 LLM + 仅训练轻量 Projector”的范式，在显著降低训练成本的同时达到与全参数微调可比的效果，后续多数 MLLMs 均沿用或发展此类设计。这种模块异构（编码器、投影层、LLM、生成器在结构、参数量与计算特性上差异显著）与分阶段冻结策略，使得在异构 GPU 集群上为各模块分配合适的算力与显存、并设计均衡的并行与调度策略变得尤为困难。

从实现角度看，编码器多为卷积或 ViT 类结构，单层计算与显存占用与 Transformer 解码层不同；投影层参数量小、计算密集度高，在流水线中若与 LLMs 层混合划分，易造成流水级间负载不均；LLMs 主体层数多、激活体积大，常需配合重计算与张量并行以控制单卡显存。生成器（若存在）多为 U-Net 或扩散结构，其前向与反向计算模式与语言模型差异较大。因此，在设计面向 MLLMs 的并行与调度方案时，需要针对各子模块的特性分别建模计算、显存与通信，并考虑不同训练阶段下冻结比例变化带来的动态负载差异，而不能简单套用单一 Transformer 的划分假设；这也为本文第三章中针对各模块与冻结状态的细粒度划分与调度提供了直接动机。

2.1.2 多模态大语言模型的训练数据

在数据与模型规模的文献共识方面，Kaplan 与 Chinchilla 等关于扩展定律与计算最优配比的讨论 [26–28] 已经在第一章提及，此处不再复述。大语言模型通常在海量文本语料（如 Common Crawl、The Pile[74]、书籍与代码）上预训练，规模可达数万亿 token。对 MLLMs 训练而言，关键之处在于：在文本预训练之外，还需大规模图片 - 文本或视频 - 文本等多模态配对数据支撑对齐、指令化与生成等训练阶段，使存储与计算压力在单机维度上进一步凸显。多模态理解与推理的评估常采用 MMMU[25] 等多模态评估基准 [71]，其题目与数据形态的多样性也反过

来推动训练数据在模态与任务类型上的扩展。

除数据体量本身外，当前 MLLMs 的主流训练配方还往往同时推高有效批量规模与序列长度，从而对单设备显存形成额外压力。一方面，为稳定梯度估计、提高分布式下优化器收敛性，预训练与指令微调在实践中常采用较大的全局批量（global batchsize），并借助梯度累积、数据并行等手段在单步内等效放大 micro-batch，使部分中间激活需常驻或频繁换入换出显存，这部分中间激活随着批量大小增大线性增加；另一方面，文档与图表理解、多图对话、高分辨率输入与视频帧序列等任务，使经 patch 化或采样后的视觉 token 与文本 token 拼接后的序列显著变长，语言侧亦普遍向更长上下文扩展 [39]，代表性工作如 LLaVA-NeXT 等亦持续抬高训练与推理中的上下文规模 [88]。在 Transformer 模型结构下，注意力与中间激活的显存占用随序列长度与批量近似线性或超线性增长，多模态序列还会在长度维上叠加视觉 token，进一步增加中间激活的显存占用。因此，即便模型参数量固定，更大 batchsize 与更长序列仍会与模块异构带来的不均匀显存分布相叠加，使显存成为制约单卡可训练配置与端到端吞吐的关键因素之一，导致重计算技术成为现在训练 MLLMs 的必要技术。这也为本文第四章中针对更大 batchsize 与更长序列的重计算优化提供了直接动机。

表2.1列举了若干代表性数据集的规模与用途。图像方面，Open Images[72] 包含约 900 万张图像及框注，存储需求约 18TB；JFT、LAION 等内部或开放图文对数据集规模可达数亿至数十亿样本，用于视觉—语言预训练。视频方面，YouTube-8M[75] 等基准包含数百万视频与标签，用于视频理解与生成的数据集往往需要 PB 级存储与高吞吐的数据管线。文本方面，Common Crawl、The Pile 等为 LLM 预训练提供数万亿 token 的语料。综合上述规模可知，单机显存既难以容纳完整模型与优化器状态，单机存储与 I/O 带宽也难以在训练周期内高效供给全量数据；在这一双重约束下，并行训练（数据并行、模型并行及其组合）成为必然选择。与此同时，高质量的输入以及对于更长上下文的需求，对计算设备的显存也提出了更高要求，重计算成为不可或缺的技术手段。

表 2.1 多模态大语言模型训练中常用的部分数据集规模示意

类型	规模量级	存储量级	典型用途
图像文本对	数百万 – 数十亿	TB–PB	图文预训练、对齐（如 Open Images[72]）
视频	数百万 – 数亿	百 TB–PB	视频理解、生成（如 YouTube-8M[75]）
文本 / 语料	数百亿 – 数万亿 token	PB 级	LLM 预训练、指令数据

2.1.3 多模态大语言模型的训练过程

多模态大语言模型的训练通常采用多阶段、分模块冻结的策略，在不同阶段设定不同的训练目标与可训练参数集合，以兼顾训练速度与模型质量 [10, 11]。各阶段的先后顺序遵循从粗粒度对齐到细粒度能力塑造的递进关系：典型阶段包括（1）模态对齐（或预训练），在大规模图文对或视频—文本对上训练 Input Projector（及可选的部分 Encoder/LLM），使视觉等模态特征与 LLM 语义空间对齐；（2）指令微调（Instruction Tuning），在高质量指令—回答数据上微调，以提升遵循指令与对话能力，此时常冻结 Encoder、仅微调 Projector 与 LLM 部分层；（3）多模态生成微调（若需文生图等），在生成数据上训练 Output Projector 与 Modality Generator，LLM 可冻结或低秩微调。各训练阶段对冻结与更新的模块设定不同，因而在同一模型上，不同阶段对应不同的计算与显存需求分布。换言之，模块异构在时间维度上表现为阶段依赖的、动态变化的负载形态。实践中，各阶段采用的典型超参与数据规模因模型与任务而异：LLaVA[11] 在约 60 万图文对上做视觉—语言对齐与指令微调；InstructBLIP[15] 在 13 个指令化数据集上微调；Qwen-VL[17]、LLaVA-NeXT[88] 等则进一步扩大指令数据与上下文长度；闭源多模态模型如 Gemini[89]、PaLM[90] 系列则在更大规模数据与算力下完成预训练与对齐，其具体数据配比与阶段划分多未完全公开。

冻结会显著改变各模块的反向计算量与显存占用。其原因是：冻结参数不参与梯度计算与更新，因而不需为其保存梯度或中间激活用于反向；优化器状态（如 Adam 的动量与方差）仅需为可训练参数维护，故冻结比例越高，单步显存与计算量通常越低 [10]。在流水线并行下，若某流水级被冻结，其对应设备上的反向计算远少于未冻结的流水级，容易造成流水级间负载不均与流水线气泡，导致流水线效率下降。在异构 GPU 集群下，设备间本就存在显存、算力与带宽差异；不同训练阶段的冻结策略带来的动态模型异构（即同一模型在不同训练阶段呈现不同的有效计算与显存分布）与设备异构叠加，使得静态的、一次性确定的划分与调度难以在各阶段均达到较高的训练效率。因此，一种能够随训练阶段与冻结状态自适应调整划分与调度的方案具有明确价值，也是本文第三章 MMoH 设计的目标之一。

图2.1概括了上述多阶段训练中常见的模块冻结与更新关系，便于理解不同阶段对算力与显存的需求差异。

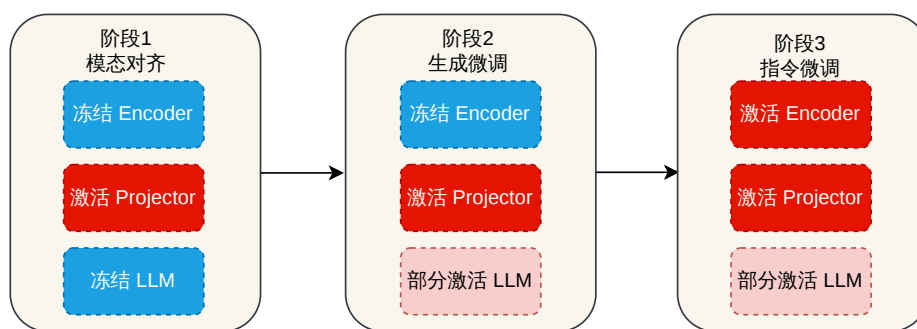


图 2.1 MLLM 多阶段训练中典型模块冻结与更新关系示意

2.2 并行与分布式训练技术

2.2.1 数据并行与模型并行

数据并行 (DP)。 数据并行将训练数据在各计算设备间划分, 每个设备持有一份完整模型副本, 本地前向传播与反向传播后通过梯度 All-Reduce 集合通信同步梯度数据 [42, 43, 91]。如图2.2所示, 各设备独立计算所分配 mini-batch 的梯度, 再在前向传播和反向传播都结束后聚合梯度并更新一致参数。DP 实现简单、扩展性好, 是 PyTorch DDP[42]、Horovod[43] 等框架的默认方式。PyTorch 的 FSDP[52] 在数据并行基础上对参数、梯度与优化器状态进行分片, 与 ZeRO-3 思路相近, 可在单卡显存受限时支撑千亿级稠密模型训练; Parallax[92] 针对稀疏与异构场景对参数服务器与 All-Reduce 做了灵活选择。数据并行的局限在于单卡需容纳完整模型与优化器状态 (DP) 或需额外通信聚合分片 (FSDP), 无法单独支撑超大参数规模, 常与张量 / 流水线并行或 ZeRO 组合使用。

张量并行 (TP) 与序列并行 (SP)。 张量并行在层内划分模型权重与计算, 使多设备协同完成单层的前向与反向, 组合后等价于完整层 [36, 37]。如图2.3所示, 每设备仅持有部分参数与激活, 层间通过 All-Gather 或者 All-Reduce 等集合通信传递梯度数据。Megatron-LM[37] 针对 Transformer 提出按行 / 列交替划分 FFN 与注意力权重, 在减少同步次数的同时控制通信量; GShard[46] 在 MoE 中结合张量并行与专家划分。TP 通信密集且对带宽敏感, 实践中多限于单节点或高带宽拓扑内, 并与流水线、数据并行组合 [36]。序列并行 (SP) 针对长序列场景, 将输入沿序列维划分到多卡, 每卡仅处理一段子序列, 通过通信在注意力等层交换信息以保持语义一致 [78, 93]。DeepSpeed-Ulysses[93] 将序列均匀分片, 在注意力层采用双阶段 All-to-All 重分布, 使通信量随序列长度与设备数成比例缩放, 支持百万

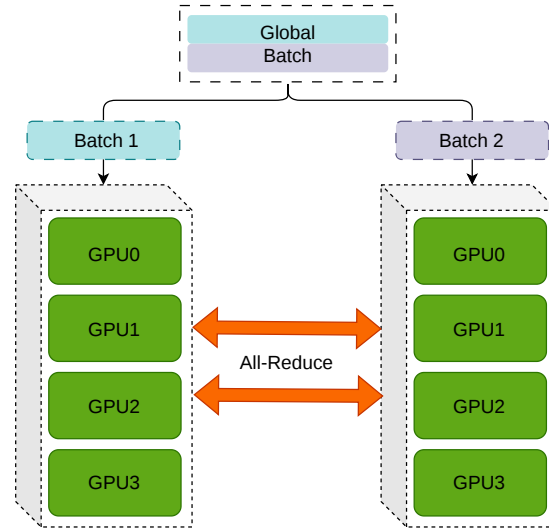


图 2.2 数据并行示意

级 token 训练；Megatron 中的序列并行实现常与张量并行共用进程组，与 TP、PP、DP 组合构成多维混合并行 [78]。

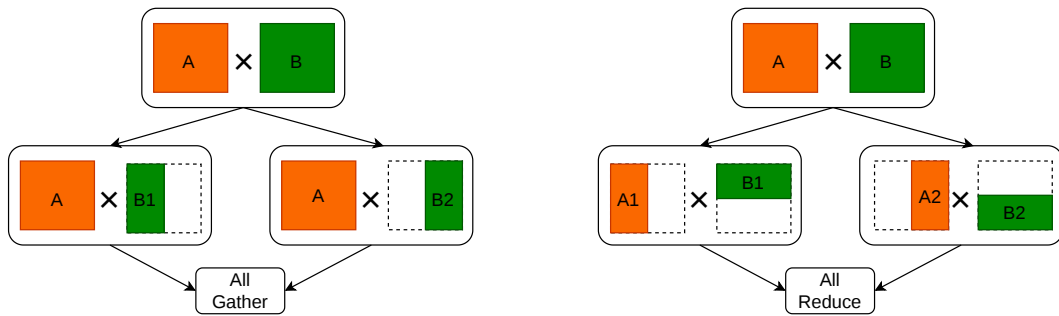


图 2.3 张量并行示意

流水线并行 (PP)。流水线并行在层间划分模型，每设备持有若干连续模型层构成一个流水级，如图2.4所示。GPipe 使用多个 micro-batch 依次流过各流水级以重叠计算、降低流水线气泡率 [44]。PipeDream[45] 提出 1F1B (1 Forward 1 Backward) 等异步调度流水线技术；TeraPipe[94, 95] 在序列内做 token 级流水线，实现更细粒度的层间并行。Chimera[96] 采用双向流水线减少气泡；Hanayo[97] 等波式调度进一步优化多流水级下的计算重叠。近年来，ZeroBubble[98] 在同步语义下将反向拆分为“对输入的梯度”与“对参数的梯度”两阶段，结合 1F1B1W 等调度与参数更新重叠，实现了近似零气泡，在相同显存下相较 1F1B 提升吞吐约 23%–31%；DualPipe[99] 采用双向 micro-batch 流动与前后向重叠，在 DeepSeek 等超大规模

训练中用于降低 bubble 与通信压力。PP 通信量相对较小，适合跨节点扩展，常与节点内 TP、全局 DP 组合 [37]。

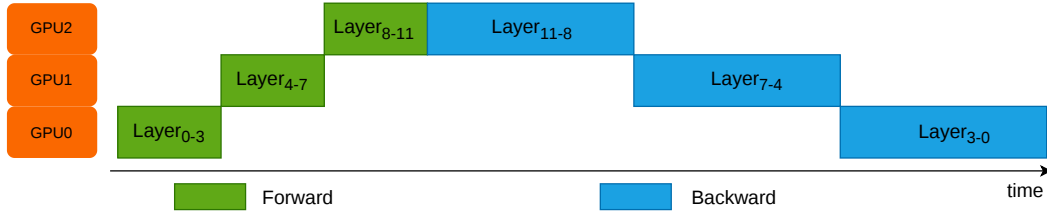


图 2.4 流水线并行示意

上下文并行（CP）与专家并行（EP）。上下文并行（CP）在如今序列长度越来越长的场景下得到了广泛应用：单卡显存难以容纳全长序列的激活时，将输入沿序列维度划分到多卡，在注意力层通过 All-to-All 集合通信进行分块计算保持注意力分数的完整性。与张量并行共用进程组的序列并行（Sequence Parallelism）不同，上下文并行把模型输入在序列维度做切分，不同的序列块使用不同的 GPU 进行计算，称为 Context Parallelism（CP）[78]。CP 在 8K+ token 场景下显存收益显著，但会引入额外通信与拓扑约束，需与 Flash Attention、重计算等技术配合使用。专家并行（EP）针对混合专家模型（MoE）：不同 token 由不同子网络（专家）处理，EP 将多个专家分布到不同设备，每设备仅持有部分专家，前向时根据路由将激活发送至对应专家所在设备，再汇总结果 [46]。与张量并行（每设备持所有专家的部分权重）不同，EP 下每设备只存部分专家的全量权重，通信模式为 All-to-All 的 token 重分布，适合专家数多、单专家规模适中的 MoE。GShard[46] 将 MoE 与张量并行结合并做自动分片；DeepSeek-MoE[101] 等在有限资源下通过 MoE 扩展语言模型规模；DualPipe[99] 等也在流水线中结合专家并行以支撑超大规模 MoE 训练。EP 的主要挑战是专家负载不均与路由带来的通信开销，需高带宽互联与负载均衡设计。图2.5展示了专家并行中的典型路由过程：门控网络先为 token 选择 Top- k 专家，再通过 All-to-All 集合通信将 token 分发到对应 GPU 上的专家执行计算，最后汇聚计算结果。

2.2.2 零冗余优化器（ZeRO）

ZeRO[40, 41] 在数据并行基础上，将优化器状态、梯度与参数分片存于各计算设备，前向与反向时按需聚合，在保持 DP 通信量的前提下显著降低单卡显存，可以支持上千亿参数模型的训练。如图2.6所示，ZeRO 三阶段分别将优化器状态、梯度与参数进行分片来减少单卡的显存占用；ZeRO-Infinity[41] 进一步将部分状态卸载至 CPU/NVMe 以突破 GPU 显存墙。ZeRO++[47] 通过量化与通信重

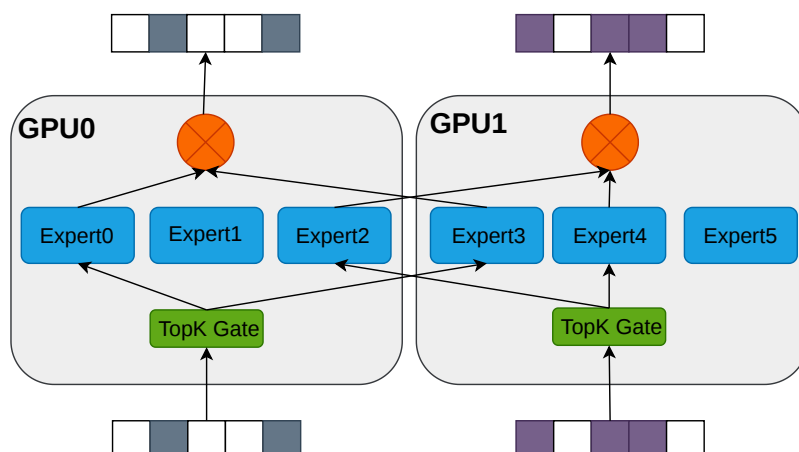


图 2.5 专家并行 (EP) 中门控路由与专家分布示意

算减少跨节点通信量；MiCS[85]、AMSP[86] 等在云环境与统一划分空间上做了扩展。ZeRO-3 与张量并行、流水线并行的组合在工程上存在约束（如 DeepSpeed 与 Megatron 的 3D 并行通常采用 ZeRO-1/2 与 TP+PP），需根据框架与集群拓扑谨慎配置 [102, 103]，在实际部署中需在显存节省、通信开销与实现复杂度之间权衡。ZeRO 与 TP、PP、SP 等组合构成当前大模型与 MLLM 训练的主流技术手段。

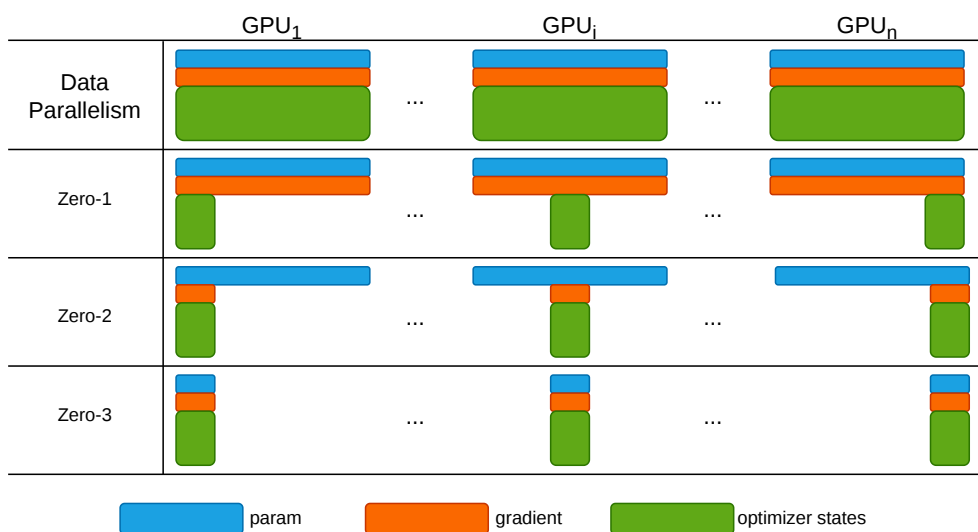


图 2.6 零冗余优化器 ZeRO 示意

2.3 异构集群上的大模型并行训练

第一章从实际部署出发，已说明常见情形：同一集群中往往混有多代 GPU，显存容量、算力与互联带宽并不一致；若仍按同构假设选用默认并行配置，容易

出现负载倾斜、通信受限以及单点设备制约整体效率等问题。本节不再展开现象层面论述，而沿文献脉络梳理近年面向算力与互联不均的异构集群并行训练工作，并说明其在多模态大模型——编码器、投影层与语言骨干结构差异显著，且训练各阶段冻结范围常需调整——的情形下仍存在的不足。

2.3.1 异构 GPU 集群下的并行训练策略

围绕异构环境下的并行训练，已有若干代表性工作从不同角度展开了探索。

HetPipe [54] 较早提出了针对异构 GPU 设备的解决思路。它将若干 GPU 组织为虚拟工作节点（VW），节点内部以流水线并行处理小批量，节点之间叠加数据并行来提升训练吞吐。通过强弱设备搭配成组，该方法在一定程度上避免了算力较弱的设备成为单一流水瓶颈。不过，它并未对 VW 内部的拓扑结构、设备排列与通信开销进行精细建模。更关键的是，该方法默认模型为相对规整的层堆叠结构，而对于编码器、投影层与大语言模型各自“形态迥异”的多模态大语言模型，以及训练过程中分阶段切换冻结策略等复杂场景，并没有现成的解法。

Whale [53] 的思路则相对直观：既然设备显存与算力天然存在差异，不如按设备能力为其分配差异化的批次大小与分片策略，在数据并行与张量并行的维度上将负载“摊平”。这一方案将“大卡多算、小卡少算”的工程直觉转化为可操作的实施路径。但它的划分粒度止步于子图层级，未能与 ZeRO 等显存优化手段打通。当集群规模扩大或异构维度增多时，单纯依赖比例切分往往难以兼顾通信效率与显存安全。

AMP [79] 则将并行策略的搜索推向了自动化方向。它将层间结构差异抽象为“模型异构”，将设备显存、算力与链路特征抽象为“设备异构”，在建模与量化的基础上，于 DP、TP、PP 的组合空间中建立开销模型，并借助动态规划求取近似最优配置。实验结果显示，在异构场景下该方法较基线有明显收益。但问题在于，全局划分的粒度仍然较粗，通信与显存的估计误差会直接传导至最终配置方案；若缺乏显存上界等硬性约束，实际部署中仍可能发生显存溢出（OOM），通常还需配合激活重算与分片预算等手段进行额外调参，才能稳定运行。

上述工作在缓解负载不均衡与显存压力方面提供了有益借鉴，但这些方法都有一个共同前提：模型主体可近似为结构相近的 Transformer 层堆叠。当研究对象转向模块边界清晰、各模块计算与显存曲线差异显著的多模态大语言模型，并需应对训练过程中分阶段冻结策略的动态变化时，原有的共同前提被打破，方法的有效性也随之下降。

2.3.2 异构 GPU 集群并行训练的工业实践

近年来,工业界研究更关注如下问题:在模型与数据规模确定的前提下,异构集群应如何选配 GPU 数量与型号、流水级如何映射到设备,以及如何在成本与时延约束下取得可接受的训练性能。**Espresso**[82] 针对上述需求构建框架: **Profiler** 在各类 GPU 上测量迭代时间与显存占用, **Allocator** 完成设备分配, **Stage Placer** 确定流水级映射, **Performance Estimator** 估计给定配置下的吞吐。用户输入模型、数据集、训练轮次与 SLO 等指标后,系统输出资源与 stage 方案,在成本与效率之间折中。它将测量、资源分配与性能估计串成闭环,但缺乏与 MLLM 多模块协同等方面的深度结合,实际用来训练 MLLMs 时仍然不能充分利用异构设备。

多厂商设备的混合则进一步放大了上述需求。**HetHub**[80] 针对华为昇腾、AMD、NVIDIA 等卡混在一起的集群,做了统一通信、性能预测和自动并行规划;在 768 张异构卡上训练 Llama-140B,可达到理论性能上界约 97.49%,表明多厂商混部集群同样能够支撑超大模型训练,但对系统工程要求较高。**HexiScale**[81] 则把 DP、PP、TP 的非对称划分写成约束优化,用层次图划分给各卡分计算,在 7B–30B 规模上 MFU 与同构系统只差大约 3.5%,相当于给“按能力不对称切分”补了一套可复用的算法。

Zorse[83] 更贴近大规模 LLM 训练中的常见矛盾:流水线与数据并行如何组合以降低通信与显存开销;流水级能否非对称划分、同一数据并行组内能否容纳能力不同的设备。它将上述灵活性统一纳入实现,并配备自动规划器,实验表明相对既有方法可获得一定训练加速。

ZeRO 系方法在异构环境中亦不能完全照搬同构假设下的经验。原版 ZeRO[40, 41] 中参数与梯度的 All-Gather 等集合通信,在跨节点且链路带宽参差不齐时易成为瓶颈;MiCS[85] 在超大模型与复杂网络拓扑下对高带宽链路的利用仍不充分;ZeRO++[47] 借助量化与通信重算降低流量,需在收益与精度、通用性之间权衡;AMSP[86] 等以削减跨节点通信为主,对单机内显存、算力与带宽细粒度错配的综合利用讨论相对较少。

综合上述工作可见,学术界与工业界正通过自动规划、测量驱动的配置与约束优化等途径,降低对手工调参的依赖。然而,这些路线多数仍以通用 LLM 或“集群近似同构后再谈扩展”为基本前提;当训练对象变为模块边界清晰、各模块资源曲线差异显著的多模态大语言模型,且需在多阶段训练中反复调整冻结范围时,计算图随阶段明显变化,现有方法尚难与之对齐并形成系统化的专门方案。若将上述结论与本章关于并行划分及异构集群扩展的综述一并考虑,在 MLLM 的模块异构与分阶段冻结与显存、算力、通信三重错配同时出现的条件下,仍缺少将划分、调度与显存策略统筹考虑的成套联合优化。

工程落地层面, 还需应对异构环境下的性能预测精度、与 Megatron、DeepSpeed 等框架的集成方式, 以及多阶段训练中划分与调度能否随阶段演化而及时更新等问题。Frenzy[63] 等面向内存感知的无服务器调度研究, 可为资源与阶段放置提供借鉴; Espresso、Zorse 等则尝试将性能分析、资源配置与并行规划封装为较完整的工具链。即便如此, 面向 MLLM 的模块级划分以及冻结策略切换时的自适应调度, 仍未形成成熟的一体化方案, 本文提出的 MMoH 即针对上述缺口展开研究与系统设计。

2.4 重计算显存优化技术

大规模模型与长序列训练中, 前向传播产生的中间激活 (activation) 需在反向传播时用于梯度计算, 若全部保存则显存占用随层数与序列长度线性甚至平方增长, 成为训练瓶颈。重计算 (Recomputation), 又称激活检查点 (Activation Checkpointing), 通过“以计算换显存”的思想缓解上述压力, 是后续各小节讨论的基础。本节依次从策略分类与训练框架中的实现及其局限两方面展开综述。

2.4.1 重计算基本原理与分类

重计算具体指: 前向传播时仅保存部分激活或仅保存检查点处的输入, 反向传播时对未保存的区域重新执行前向传播以得到所需激活值, 如图2.7所示, 从而在显存与计算之间进行权衡, 是缓解显存压力的常用手段 [104]。根据检查点放置的粒度和策略, 可大致分为全部重计算、选择重计算与细粒度重计算三类; 以下分别简述其思路与适用场景。

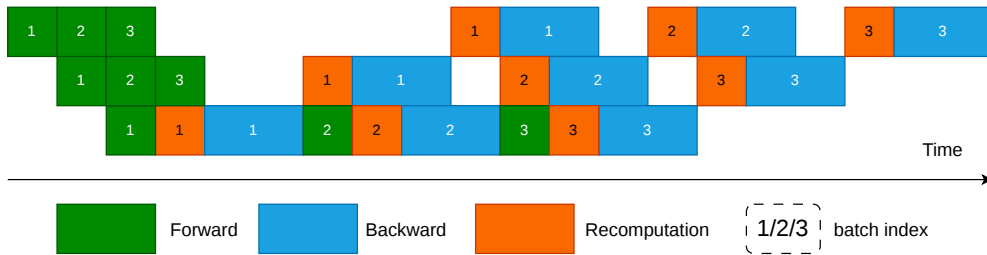


图 2.7 重计算（激活检查点）示意

全部重计算。 全部重计算 (full recomputation) 策略在前向阶段不保存任何中间激活, 仅在若干“检查点”层保存该层的输入 (或仅保存网络输入), 反向传播需要某段中间激活时, 从最近的检查点重新执行前向至该位置再计算梯度。极端情况下仅保存网络输入, 则显存占用可降至与深度无关的常数级别, 但反向传播需对整网做一次完整前向重算, 计算开销约增加前向传播的一倍。Chen 等 [104] 针

对链式计算图证明了在仅增加一次额外前向的前提下，通过动态规划可求得检查点的最优放置，使显存从 $O(n)$ 降为 $O(\sqrt{n})$ (n 为层数)，并在千层残差网络等实验中实现了显存的大幅下降（如 48 GB $\rightarrow$ 7 GB）与约 30% 的训练时间增加。全部重计算实现简单、显存收益最大，适合显存极度紧张、算力相对充裕的场景，但计算冗余较高。

选择重计算。 选择重计算（selective recomputation）不是对整个 Transformer 层统一做 checkpoint，而是只对显存占用大但重算代价相对较低的部分子模块执行重算，未选部分仍正常保存激活。Korthikanti 等 [105] 在大规模 Transformer 中提出的 selective activation recomputation 即属于此类：其核心思想是优先重算注意力中的 QK^T 、softmax、dropout 以及与 V 的乘法等中间激活，因为这些张量随序列长度和注意力头数增长较快，但额外 FLOPs 开销相对有限。相较整层重计算，该策略的显存收益略小，但能显著降低重算开销；论文报告其总时间开销约为 7%，明显低于整层重算的 39% [105]。在工程实现上，Megatron-LM 与 NeMo 将其落地为 selective 粒度，默认仅重算 core_attn，也可扩展到 mlp、moe、layernorm 等指定子模块 [106, 107]。

细粒度重计算。 细粒度重计算（fine-grained recomputation）在算子或更小粒度上决定哪些激活保存、哪些重算，从而更精细地利用不同算子的计算—显存特性。与上一段“选择重计算”主要回答“哪些模块需要重算”不同，细粒度重计算更关注“如何在模块内部重算”。例如，Transformer 中可不把整个注意力或 MLP 作为单一单位处理，而是进一步细化到 LayerNorm、dropout、MoE 激活函数、或某些投影输出等中间张量：前向时仅保留最少输入或元数据，反向时再局部恢复需要的结果。Megatron-LM/NeMo 中的 output-discarding checkpoint 就体现了这种思路：它并非简单地把整段前向再执行一遍，而是在前向后主动丢弃部分输出张量的存储，在反向阶段借助 hook 触发局部重算，从而进一步压缩激活占用 [106, 107]。这类方法比“按层”或“按子模块”的重计算更贴近实现细节，能够利用不同算子的显存占用、通信需求与 FLOPs 差异做更细致的权衡。Lynx [108] 进一步将激活重算与通信重叠调度，在 1.3B–23B GPT 模型上相较现有方法取得最高约 $1.37\times$ 的加速，表明细粒度重算若与通信、并行调度协同设计，还可以同时改善显存效率与训练吞吐。细粒度策略可结合模型结构（如 MLLMs 中 Encoder、Projector、LLM 各模块的算子组成）与异构设备的显存、算力差异，在不同设备或不同 stage 上采用不同的重计算粒度，是面向异构集群与 MLLM 的潜在优化方向。

2.4.2 重计算的主流实现及其局限性

在 Megatron-LM 与 NeMo 等框架中，重计算被实现为可配置的多级粒度。全部重计算（full）以整个 Transformer 层为粒度：前向阶段仅仅保存每层输入，反向时重算整层前向，显存下降显著但单层计算开销增加 30% 以上。框架进一步提供两种全层模式：*block* 模式通过参数指定每个流水线 stage 内参与重计算的层数，可仅对部分层做重计算以在显存与计算间折中；*uniform* 模式以连续多层的“块”为重计算单位，块内仅存块首输入，适用于层数很多或层间激活相对较小的模型，可节省激活显存但会增加重算缓冲。选择性重计算（selective）则仅在子模块级别重算：例如仅对自注意力块做重计算（保存注意力块输入，重算 softmax、dropout、QKV 等中间激活），而 MLP 等不重算。自注意力激活随序列长度平方增长、占显存大头，但其重算成本相对线性投影层较低，因此选择性重算能以较小计算代价获得较高显存收益；用户还可指定 `core_attn`、`mlp`、`moe` 等模块是否参与重算。在使用 Flash Attention 时，框架通常强制对注意力做选择性重算以配合其显存布局。上述多级粒度（全层 *block/uniform*、子模块 *selective*）为实际训练中按显存预算与算力调节重计算策略提供了细粒度控制，与 Korthikanti 等 [105] 的序列并行与选择性激活重算相结合，已广泛应用于大规模 Transformer 与 LLM 训练。

尽管如此，现有重计算策略多面向同构设备设计：重计算层数、块大小、参与重算的子模块等均需用户或策略脚本手动指定，且通常假设各设备显存与算力一致，无法根据异构集群中不同设备的显存余量与算力差异自动调节（例如在显存紧张设备上加大重算比例、在算力瓶颈设备上减少重算）。将重计算与流水线并行、stage 划分及异构分配结合，可进一步缓解显存瓶颈并提升整体吞吐：例如在流水线各 stage 内独立设置重计算策略，或在显存紧张的设备上采用更激进的重计算、在算力紧张处适当多存以减少重算，但上述调节目目前仍依赖人工或离线调参。Megatron-LM、NeMo 等支持按层或子模块（如 `core_attn`、`mlp`）配置选择性重算 [106, 107]；Colossal-Auto[49, 109] 等将自动并行策略搜索与重计算统一自动化，为大规模模型与异构环境下的联合优化提供了参考；在异构 GPU 集群上，针对 MLLM 的模块异构与分阶段冻结，设计能自动适应设备异构的细粒度重计算策略是本文的工作方向之一。

2.5 本章小结

本章承接第一章的论述，从文献角度分四方面综述了 MLLM 架构与训练数据/范式、同构假设下的并行与 ZeRO 基础设施、异构 GPU 集群上的系统化工作，以及重计算策略与框架实现。首先介绍了 MLLMs 的典型架构、训练数据规模与训练目标，以及模块异构与分阶段冻结带来的计算—显存需求差异。随后综述了数

据并行、张量并行、流水线并行、序列并行与专家并行等多维混合并行方式，以及 ZeRO 系列显存分片优化，指出其同构假设下的广泛应用与在异构、模块异构场景下的局限。接着在异构 GPU 集群一节中分两部分：先概述显存、算力与通信三重异构及 Whale 等并行与自动搜索路线，再讨论 Espresso 等系统化资源规划框架及 ZeRO 类显存分片在异构环境下的局限，并指出其对 MLLMs 模块异构与分阶段冻结的适配仍不足。最后在重计算一节中分两部分展开：先概述重计算思想及全层、选择与细粒度三类策略；再归纳现有框架中的实现。综合来看，MLLM 的模块异构与分阶段冻结、以及异构集群的显存—算力—通信差异，共同构成了现有同构导向并行与统一重计算策略难以直接应对的挑战。上述分析为本文第三章的 MMoH 混合并行框架与第四章的 HR-MMoH 异构重计算提供了问题背景与动机。

第三章 MMoH：多模态大语言模型在异构 GPU 集群上的高效混合并行

第二章指出，多模态大语言模型（MLLMs）在结构上由编码器、投影层与 LLM Backbone、解码器（可选）等异构模块组成，训练时又广泛采用分阶段冻结的训练策略，致使各层计算量与显存需求随模块类型与参数训练状态呈现显著差异；在异构 GPU 集群上若直接沿用面向同构集群或传统大语言模型（LLMs）设计的并行策略，易产生负载不均衡与资源利用率低下。针对上述问题，本章介绍 MMoH（Multimodal large language model training on Heterogeneous clusters）框架：通过需求驱动的设备拓扑与复合粒度模型划分，在异构集群上为 MLLMs 自动生成近似最优的模型划分和设备映射；并引入冻结感知的提前终止调度器，在冻结场景下识别并消除冗余的反向计算与流水级间通信，在保证训练收敛性的前提下提升训练吞吐。本节首先给出问题背景与动机，随后概述 MMoH 的整体架构与各组件职能。

3.1 引言

3.1.1 问题与动机

传统大语言模型由结构相同的 Transformer 层堆叠而成，各层在计算开销（即 FLOPs）与显存占用上随层索引变化相对均匀，因此面向 LLM 的异构并行方法（如 Whale[53]、Espresso[82]）多基于“层数”或“设备计算能力”或“设备显存大小”按比例将层划分至不同设备，即可获得较为均衡的负载分布。这类方法在同构 GPU 集群上已经被证明是有效的：当所有设备的算力与显存容量相近时，只要保证每个流水级承担的层数大致相同，即可在不做复杂优化的前提下获得接近线性加速比。

然而，多模态大语言模型在架构上与之存在本质差异。如图 3.1 所示，常用的 MLLMs 由三个模块构成：（1）编码器（Encoder），负责将图像、视频等多模态输入编码为特征表示，多采用视觉 Transformer 或轻量卷积网络；（2）投影层（Projector），将编码特征映射至与 LLM 词嵌入空间对齐的语义空间，常见形式为线性层或 Q-Former[10]；（3）LLM Backbone，承担主要的语言理解与生成计算，由参数量巨大的 Transformer 层组成 [10, 11]。编码器单层参数量与序列长度通常远小于 LLM Backbone，投影层则为小型模块，因此 MLLMs 各“层”在计算量与显存开销上的差异可达数十倍。在实际部署中，研究者往往需要在由多代 GPU 混合构成的异构集群上训练这类模型，不同节点之间不仅算力 / 显存比差异显著，互

联带宽与通信延迟也存在明显不均，这进一步放大了“模块异构 + 设备异构”叠加带来的负载失衡问题。

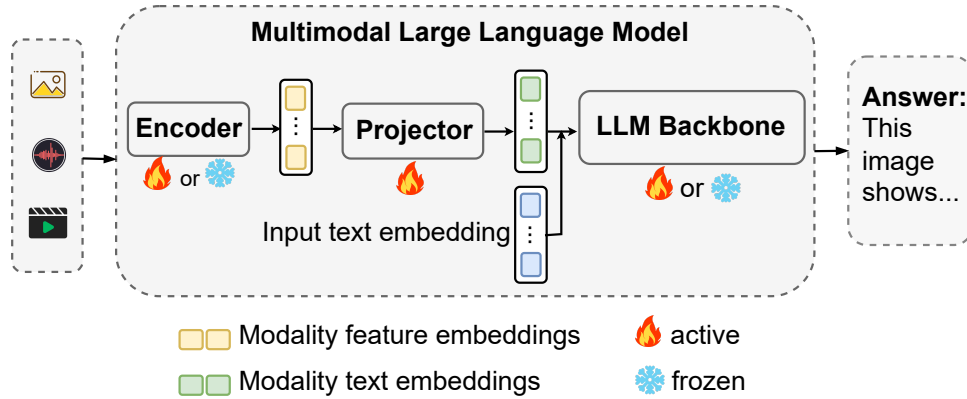


图 3.1 主流 MLLMs 的编码器—投影层—LLM 结构及冻结方式。

若仍采用与 LLM 相同的单一“层”粒度进行模型划分，将导致部分流水级过载而其他流水级空闲，无法达到负载均衡，整体迭代时间由瓶颈流水级主导，异构设备的算力与显存无法得到有效利用。图 3.2 对比了 LLM (LLaMA-3B) 与 MLLM-3B (见表 3.2) 在各层计算开销与显存占用上的变化趋势：LLM 各层较为均匀，而 MLLMs 随模块类型与冻结场景 (NF 表示不冻结，FB 表示编码器与 LLM 均冻结) 呈现显著差异。可以观察到：在编码器段，单层 FLOPs 与显存曲线整体处于较低水平，而在进入 LLM Backbone 后，相邻两层之间的计算量跳变明显，甚至在注意力块与 FFN 块之间也呈现出不同的计算—显存比；若简单地以“层数均分”作为划分准则，则很容易出现“前半段流水级几乎空载、后半段流水级长期处于饱和”的情况，导致流水线整体被少数几个高负载流水级所拖慢。

进一步地，异构环境下设备能力也呈现“阶梯状”变化：例如在由 RTX 3090、RTX 2080 Ti 与 RTX 2080 组成的集群中，不同型号 GPU 在算力与显存上的差异可达 2-3 倍以上。若不区分各模块对算力 / 显存的需求特征，而仅按照“设备总显存从大到小”或“TFLOPS 从高到低”的规则对设备进行排序并顺序映射流水级，则无法保证“高需求流水级被映射到高供给设备”，反而可能出现“轻量编码器被放置在高端 GPU、而重型 LLM 层被挤压到低端 GPU 上”的反直觉映射，进一步加剧负载不均衡。

另一方面，MLLMs 训练普遍采用冻结 (freeze) 技术 [10]：在预训练或模态对齐的训练阶段，仅更新投影层或部分 LLM 层，而编码器与 LLM Backbone 保持冻结，以利用已有预训练权重、降低过拟合并节省优化器状态与显存。冻结后，冻结参数不参与梯度计算与参数更新，对应流水级的反向计算量与梯度同步开销显著降低，其计算—显存需求与“全参数训练”时截然不同。现有异构并行策略大多假

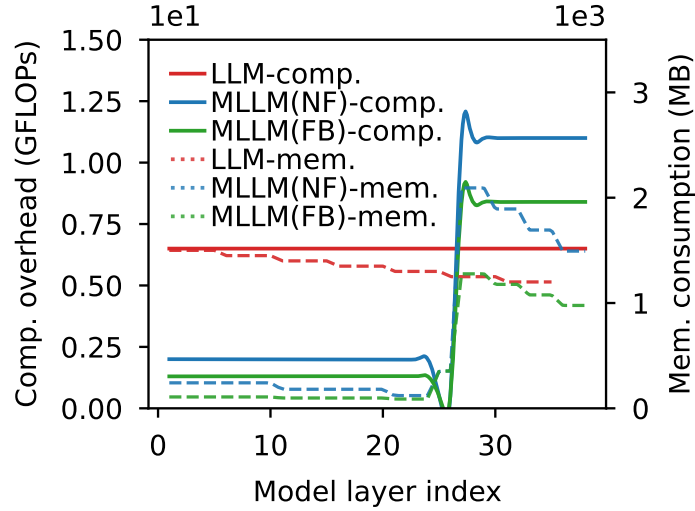


图 3.2 LLM 与 MLLM-3B 各层计算开销与显存占用的变化趋势 (NF 为不冻结, FB 为 freeze-both)。

设所有参数在整个训练过程中均处于激活状态, 无法随参数训练状态 (冻结与激活) 的动态变化而调整划分与调度; 在“仅冻结编码器” (freeze-encoder)、 “仅冻结 LLM” (freeze-llm) 或 “同时冻结编码器与 LLM” (freeze-both) 等场景下, 若继续采用为“全参数激活”场景设计的策略, 相较针对当前冻结状态专门优化后的策略, 迭代时间可落后约 13%–21%, 如图 3.3 中不同策略在两种冻结场景下的迭代时间对比所示。这意味着, 在多阶段训练流程中, 同一组设备拓扑与模型划分方案很难在所有阶段都保持近似最优: 在早期“仅训练投影层”的阶段, 编码器与大部分 LLM 层几乎不产生梯度同步与参数更新, 若仍然为其预留与 no-freeze 阶段相同的显存与计算预算, 就会造成大量资源浪费; 而在后期逐步解冻 LLM 层时, 若不重新规划流水级划分与设备映射, 又会重新暴露出负载瓶颈。

在工程实践中, 这种不匹配通常表现为两个方面的痛点: 一是人工调参成本极高。开发者需要针对不同冻结策略、不同 batchsize 与序列长度, 不断尝试手工划分与映射方案, 通过反复试错来避免 OOM 并兼顾吞吐, 这在规模较大的异构集群上几乎难以维护; 二是训练过程中的策略缺乏自适应性。一旦训练开始运行, 出于实现复杂度与稳定性考虑, 现有系统很少在中途重新划分流水级, 从而错失了在“冻结—解冻”阶段切换时重新平衡负载的机会。

此外, 根据实验结果可知, 最优并行策略并非静态不变, 而是会随训练过程中冻结状态的演变而动态调整; 现有工作尚未系统考虑这一需求, 因此难以在冻结场景下取得最优性能。

从形式化角度, 可将上述挑战概括为两类约束。多模态大模型在异构设备上的负载不均衡: 设各流水级的迭代时间为 $t_1, \dots, t_s$, 在同步流水线下一轮迭代时间由 $\max_j t_j$ 决定, 若划分不当则瓶颈流水级拉高整体时间, 其余设备空闲, 从而

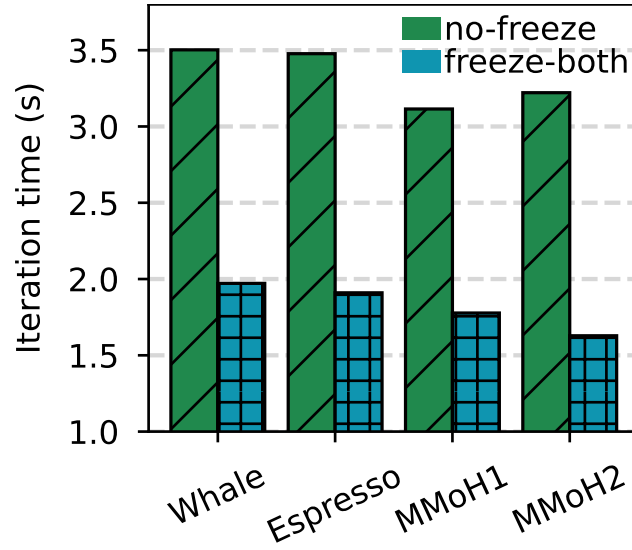


图 3.3 不同并行策略在两种冻结场景下训练 MLLM-3B 的迭代时间，MMoH1/MMoH2 为 MMoH 规划器针对各场景得到的策略。

无法充分利用异构设备上的算力与显存；多阶段冻结训练导致的最优并行策略动态变化：记冻结流水级集合为 $\mathcal{F}$ ，则 $\mathcal{F}$ 内流水级的反向传播的时间会减少，梯度同步开销近似为零，而未冻结流水级仍需完整反向与 P2P 通信，若划分与调度未显式考虑 $\mathcal{F}$ ，则无法利用“冻结前缀”带来的冗余剪枝机会，也难以在划分模型时平衡各 stage 的有效负载。

综上所述，要在异构 GPU 集群上高效训练 MLLMs，需同时兼顾两点：（1）MLLMs 的混合模块结构所导致的层间计算与显存需求非均匀性；（2）参数训练状态随训练流水级变化对最优划分与调度的影响。MMoH 针对上述两点，设计了 MMoH 并行策略规划器与提前终止调度器，在给定异构集群规格与 MLLMs 配置下自动生成近似最优的模型划分与设备映射，并在冻结场景下消除不必要的反向计算与流水级间通信，从而提升训练效率且不影响模型收敛性。

3.1.2 MMoH 框架概览

MMoH 由两大核心组件构成：MMoH 并行策略规划器（MMoH parallel strategy planner）与提前终止调度器（early-termination scheduler）。二者协同工作，分别负责离线（或按流水级）的策略生成与在线训练过程中的调度优化。图 3.4 给出了 MMoH 的整体架构：规划器以 MLLMs 配置与异构集群规格为输入，经需求驱动拓扑识别、复合粒度抽象与冻结感知划分得到并行策略；调度器在训练中利用冻结特性减少冗余计算与通信。规划器的输入包括：异构集群规格（各 GPU 的算力、显存及节点内 / 间互联带宽）、MLLMs 的模型结构与参数量、以及当前训练阶段对应的冻结状态（如编码器与 LLM 是否冻结）；输出为设备拓扑（各流水级

与 GPU 的对应关系)、模型的划分策略(每个流水级包含哪些模型层)、以及建议的 DP/TP 配置。调度器根据模型当前训练阶段的冻结状态,优化流水线调度策略,从而在冻结场景下消除不必要的反向计算与流水级间通信。与 Whale[53]、Espresso[82] 等仅按显存或算力比例做层划分、且不区分模块类型与冻结状态的方法相比,MMoH 显式建模 MLLMs 的模块异构与冻结状态,从而在多种冻结场景下获得更优的划分与更高的训练吞吐。

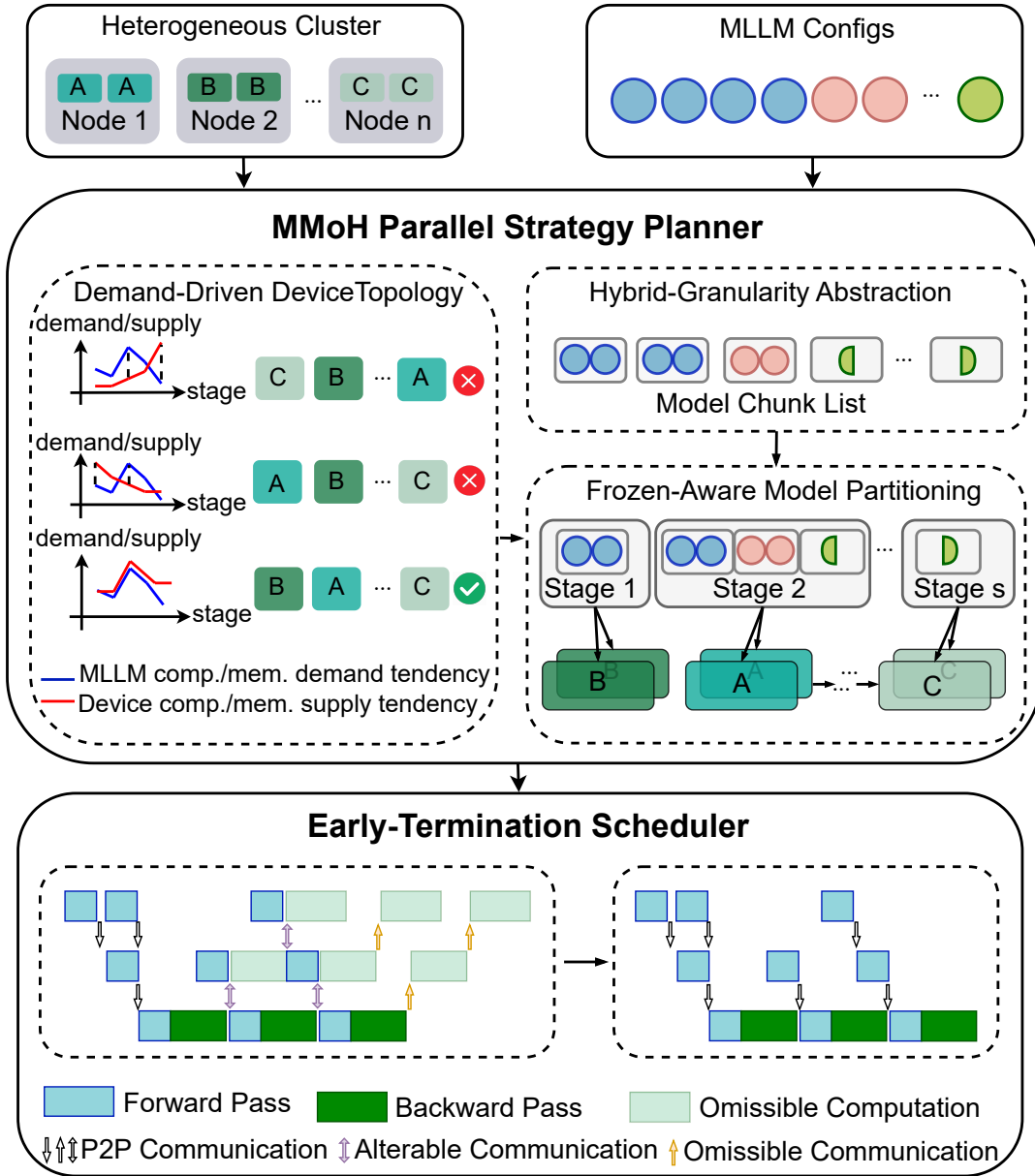


图 3.4 MMoH 框架总览。

并行策略规划器。规划器以异构集群规格（各 GPU 的算力、显存及互联拓扑）与 MLLMs 配置（模型结构、参数量、当前冻结状态）为输入，依次执行三个步骤。**第一步**为需求驱动的设备拓扑识别：根据 MLLMs 各模块的计算与显存需求趋势，与异构设备组内各设备的算力—显存供给趋势进行匹配，筛选出若干候选设备拓扑（即设备组内各流水级与 GPU 的对应关系），使需求与供给在趋势上尽可能一致。**第二步**为复合粒度抽象：针对编码器、投影层与 LLM Backbone 的结构差异，采用多级粒度将模型重组为模型块（model chunk）列表——编码器采用子模块级粒度（将连续多层合并为一块），LLM Backbone 采用子层粒度（将单层拆分为注意力块与 FFN 块），非 Transformer 层（嵌入、输出、投影）各自作为独立块，从而在划分复杂度与负载均衡之间取得折中。**第三步**为冻结感知的模型划分：在候选拓扑与模型块列表基础上，将“把块分配到各流水级”形式化为整数规划（Integer Programming, IP）问题，在满足显存约束与主流流水级约束的前提下，最小化单次迭代时间（其中通信时间显式依赖各流水级的冻结状态），求解得到各流水级所包含的块及与设备的映射。训练时，每个设备组内采用流水线并行（PP），组间采用数据并行（DP）与张量并行（TP）以扩展规模与批量大小。

提前终止调度器。调度器针对冻结技术带来的冗余进行计算与通信剪枝。在流水线并行中，若某流水级及其之前所有流水级均为冻结流水级，则该流水级无需再执行“对输出的梯度”的反向计算，也无需接收后续流水级的梯度或向后续流水级发送激活；相应的 P2P 通信与中间张量存储可安全省略，且不改变梯度传播的数学等价性。调度器在 1F1B（One-Forward-One-Backward）等流水线调度基础上，识别从流水线首端开始的连续冻结流水级并对其提前终止反向与通信，从而在 freeze-encoder、freeze-both 等场景下进一步缩短迭代时间。下文将分别对规划器的三个步骤与调度器的设计进行形式化描述与讨论。

3.2 异构设备映射与复合粒度模型划分

本节围绕 MMoH 并行策略规划器展开，形式化描述其三个核心步骤：需求驱动的设备拓扑识别、复合粒度抽象与冻结感知的模型划分，并给出相应的数学模型与约束条件。整体流程可以概括为三步：首先根据模型的计算—显存需求轮廓，从所有可能的设备排列中筛选出一组“形状匹配”的候选拓扑；随后，将原始网络重写为由若干模型块组成的线性序列，使之既能准确反映模块异构，又不会让后续搜索空间过大；最后，在给定拓扑与块列表的前提下，将“块到流水级的分配”形式化为一个以迭代时间为目标的整数规划问题并求解其近似最优解。

3.2.1 需求驱动的设备拓扑识别

在异构设备组内，不同 GPU 的算力与显存容量存在差异；若将算力与显存均较强的设备分配给计算与显存需求均较高的流水级，则能更好地匹配需求与供给，减少瓶颈流水级并降低 OOM 风险。直观地看，一旦把“最重的一段模型”错放在“最弱的一张卡”上，即便其它设备仍有大量空闲，该流水级也会成为主瓶颈，导致整体吞吐被严重限制。为避免这种“强弱错位”，MMoH 规划器首先对 MLLMs 进行剖析（profiling），在代表设备上测得各层的计算量（以 FLOPs 计）与显存占用；由于 FLOPs 与显存占用在同一模型结构下与具体设备型号关系不大，每类层仅需剖析一次。随后，规划器在此基础上枚举设备组内所有可能的设备排列（即设备拓扑），并在每一种排列下，将预先计算好的“需求侧”信息与当前排列对应的“供给侧”信息进行匹配，从而度量该拓扑是否适合当前模型。具体来说，规划器对每种拓扑计算“需求趋势”与“供给趋势”的匹配度，并据此筛选候选拓扑。设备组内共有 s 张 GPU，分属 k 种异构型号，各型号数量记为 $n_1, \dots, n_k$ （满足 $n_1 + \dots + n_k = s$ ）；则不同设备拓扑的数目为 $s!/(n_1! \dots n_k!)$ （同型号 GPU 视为不可区分，即多重组合数）。枚举后对每种设备排列进行评估，并仅保留前一半作为候选，以在后续整数规划阶段控制求解次数，在匹配质量与计算开销之间取得折中。

符号与假设。 记 MLLMs 总层数为 l （即编码器、投影层与 LLM Backbone 中所有可单独计层的单元之和），第 i 层的计算量与显存占用分别为 L_i 、 M_i （ $i = 0, 1, \dots, l-1$ ）。设备组内流水线并行度（即流水级数）记为 s ；在某一给定拓扑下，第 j 个流水级所对应设备的算力与显存容量记为 P_j 、 Q_j （ $j = 0, 1, \dots, s-1$ ）。首先，按“计算均衡”原则将 l 层划分为 s 个连续流水级，使各流水级所分配层的总 FLOPs 与各设备算力成比例，得到各流水级的总 FLOPs 与显存占用 R_j 、 T_j 。

需求趋势与供给趋势。 定义需求趋势为各流水级“单位显存下的计算量”之比构成的序列，用于刻画各流水级对算力—显存比的需求相对关系：

$$\text{Demand} = \left\{ \frac{R_1/T_1}{R_0/T_0}, \frac{R_2/T_2}{R_1/T_1}, \dots, \frac{R_{s-1}/T_{s-1}}{R_{s-2}/T_{s-2}} \right\}. \quad (3.1)$$

类似地，定义供给趋势为各设备算力—显存比之比构成的序列：

$$\text{Supply} = \left\{ \frac{P_1/Q_1}{P_0/Q_0}, \frac{P_2/Q_2}{P_1/Q_1}, \dots, \frac{P_{s-1}/Q_{s-1}}{P_{s-2}/Q_{s-2}} \right\}. \quad (3.2)$$

匹配度度量。为量化某一设备拓扑下需求与供给的匹配程度，定义匹配度（Matching Degree, MD）为两趋势序列的负指数平方误差：

$$\text{MD}(\text{Demand}, \text{Supply}) = \exp \left(-\frac{1}{s-1} \sum_{j=1}^{s-1} (x_j - y_j)^2 \right), \quad (3.3)$$

其中 x_j 、 y_j 分别为 Demand 与 Supply 的第 j 个分量。MD 取值于 $(0, 1]$ ，越大表示该拓扑下各流水级的计算—显存需求与各设备能力在趋势上越一致。规划器对所有候选拓扑按 MD 降序排序，保留 MD 较高的前半部分作为后续冻结感知的模型划分的候选拓扑，以在保证匹配质量的前提下控制求解规模。

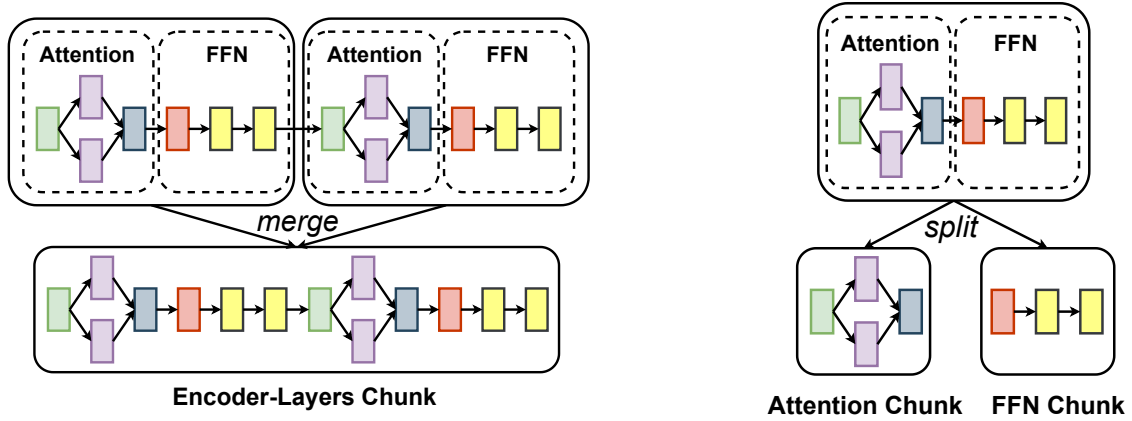
直观上，需求趋势刻画的是“从流水级 0 到 $s-1$ ，单位显存下所需算力的相对变化”：若某段流水级对应 LLM 的注意力与 FFN 块，则 R_j/T_j 往往较大；若对应编码器合并块，则相对较小。供给趋势则刻画“从设备 0 到 $s-1$ ，算力—显存比的相对变化”：将高 P_j/Q_j 的设备分配给高 R_j/T_j 的流水级，可使各 stage 在各自设备上均接近满负载，减少瓶颈并降低 OOM 风险 [112]。与直接对 R_j 、 T_j 和 P_j 、 Q_j 本身做匹配相比，利用“相邻比值”构成的趋势序列，可以在不依赖绝对尺度的前提下捕捉“先轻后重”“先重后轻”等全局模式。

在实现层面，规划器对所有设备拓扑计算 MD 并按降序排序，仅保留 MD 较高的前半部分作为后续冻结感知的模型划分的候选拓扑。这既确保了这些拓扑在趋势层面与模型需求相容，又把整数规划的求解规模控制在可接受范围内，避免在显然不匹配的拓扑上浪费时间。整体而言，该步骤使设备拓扑的选取由 MLLMs 的实际计算—显存需求驱动，而非仅按显存或算力单一维度排序，从而更好地适应“编码器轻、LLM 重”的非均匀结构以及冻结带来的需求变化。

3.2.2 复合粒度抽象

若对整个模型采用统一的“Transformer 层”粒度进行划分，将面临两难的权衡问题：编码器单层计算与显存较小，按层划分会导致块数过多、划分搜索空间过大且对负载均衡帮助有限；LLM 单层计算与显存很大，按层划分在异构设备上易造成单流水级过载或 OOM。简单地“全部细化”或“全部粗化”都会带来问题：前者使块数呈线性甚至超线性增长，整数规划的变量与约束数随之急剧膨胀；后者则使得单块负载过大，以至于再精巧的拓扑也难以获得良好的负载均衡。因此 MMLMoH 提出复合粒度抽象，将模型重组为若干模型块（model chunk）的列表，不同模块采用不同划分粒度，在划分精度与搜索效率之间取得平衡，如图 3.5 所示。

模态编码器。编码器内 Transformer 层参数量与序列长度相对较小，单层计算需求与显存开销较低。采用子模块级粒度：将连续多层合并为一个“编码器层



(a) 编码器子模块粒度（合并因子为2）

(b) LLM Backbone 子层粒度

图 3.5 复合粒度抽象示意。(a) 编码器：将连续多层合并为编码器层块；(b) LLM Backbone：将单层拆分为注意力块与 FFN 块，不增加流水级间通信量。

块”（encoder-layers chunk）。合并因子由剖析结果确定，使得每个编码器层块的计算量与显存占用均不超过 LLM 中单个注意力块或 FFN 块，从而在划分时可多个块置于同一流水级而不会造成该流水级过载。该粒度既减少了块的数量、降低了后续 IP 的变量规模，又避免了编码器部分划分过细带来的冗余。

LLM Backbone。 LLM 单层参数量大、序列长，若整层作为一块易导致某个流水级显存或算力瓶颈。采用子层粒度：将每一 Transformer 层拆分为“注意力块”（Attention chunk）与“FFN 块”（FFN chunk），划分时以块为单位分配到流水级。该分解不改变层间数据流，因此不增加流水级间通信量（块边界与层边界一致）；同时使划分粒度更细，便于在异构设备间更精细地平衡负载，并有效缓解低显存设备上的 OOM 风险。

非 Transformer 层。 嵌入层、输出层与投影层等结构各异、计算与显存差异较大，采用层级粒度，各自作为独立块（embedding chunk、output chunk、projector chunk），以准确反映其资源需求。

经复合粒度抽象后，MLLM 被表示为一个长度为 N 的模型块列表； N 及各块的大小随具体模型与冻结状态变化，为后续冻结感知的模型划分提供既不过粗也不过细的决策单元。从优化视角看，模型块相当于整数规划中的“原子变量单位”：块越小，划分的可调维度越多，但变量数量也随之上升；块越大，变量数量减少，但可调空间被压缩。复合粒度的设计正是为了在这两者之间取得折中。以本文实验中的 MLLM-3B 为例：编码器约 24 层，按合并因子 2 得到约 12 个编码器层块；投影层为 1 个独立块；LLM Backbone 12 层，每层拆为注意力块与 FFN 块，共 24 块；加上嵌入与输出等，总块数 N 约在 40 量级。该粒度下，单块计算

与显存与典型异构单卡能力相匹配，既避免块数过多导致 IP 变量空间过大，又避免块过粗导致划分无法细粒度平衡负载。

3.2.3 冻结感知的模型划分

在得到候选设备拓扑与模型块列表后，规划器将“把块分配到各流水级”形式化为整数规划问题，目标是最小化单次迭代时间，并显式考虑冻结对反向计算与梯度同步的影响。这一阶段的输入包括：（1）上一小节筛选出的若干设备拓扑；（2）由复合粒度抽象得到的模型块序列以及每个块在不同类型设备上的执行时间、显存占用；（3）当前训练阶段的冻结状态（如 no-freeze、freeze-encoder、freeze-both 等）。在此基础上，规划器对每个候选拓扑分别构建并求解一个整数规划实例，最终选取迭代时间估计值 T^* 最小的拓扑与划分方案，作为该阶段的并行策略。

决策变量与目标。 记块列表长度为 N ，设备组内流水级数为 S 。划分方案表示为序列 $P = (p_1, p_2, \dots, p_S)$ ，其中 p_i 为第 i 个流水级所分配的块数，满足 $\sum_{i=1}^S p_i = N$ 。对给定拓扑与 P ，可估计各流水级前向时间列表 $FT = (F_1, \dots, F_S)$ 与反向时间列表 $BT = (B_1, \dots, B_S)$ （由各块在对应设备上的执行时间累加得到）。记主流水级（master stage）索引为 K ，micro-batch 数为 M 。在 1F1B 调度下，单次迭代时间 T^* 可写为前向、反向与通信时间之和；其中通信时间与各流水级是否需要梯度同步有关：若某流水级内所有块均为冻结参数，则该流水级不产生梯度同步开销。规划器将通信时间建模为各流水级“非可重叠”的同步开销的最大值，且冻结流水级对应的同步开销为 0，从而在成本模型中显式体现冻结带来的通信剪枝收益。形式化地，记主流水级索引为 K 、micro-batch 数为 M ，则

$$T^* = \sum_{i=1}^{K-1} (F_i + B_i) + 2 \sum_{i=K+1}^S (F_i + B_i) + (M - (S - K))(F_K + B_K) + \text{COMM}, \quad (3.4)$$

其中第一项为 warm-up 阶段前 $K - 1$ 个流水级的前向与反向时间之和，第二项为 cool-down 阶段后 $S - K$ 个流水级各执行两次前向与反向的贡献，第三项为主流水级在稳态阶段对 $M - (S - K)$ 个 micro-batch 的计费，COMM 为通信时间。通信时间取各流水级不可与前向 / 反向重叠的梯度同步开销的最大值：

$$\text{COMM} = \max_{i=1, \dots, S} \left(AR_i - \sum_{j=1}^{i-1} B_j \right), \quad (3.5)$$

其中 AR_i 为第 i 个流水级的梯度同步（如 All-Reduce）开销，可由该流水级内激活参数数量估计；若第 i 个流水级内所有块均为冻结，则 $AR_i = 0$ 。式中“减去前

级 $\sum_{j=1}^{i-1} B_j$ ”刻画了梯度同步与反向计算的可重叠部分：若某流水级的反向计算时间足够长，则其梯度同步可被完全隐藏在反向中，对总迭代时间不再产生额外贡献。由此得到 T^* 关于 P 的显式表达式（含冻结感知的通信项），规划器以最小化 T^* 为目标在约束下求解 P 。

约束条件。 划分问题需满足以下约束：（1）**完整性约束**： $\sum_{i=1}^S p_i = N$ ，即所有块恰好分配完毕；（2）**主流水级一致性约束**：非主流水级的前向与反向时间之和不超过主流水级，即 $F_i + B_i \leq F_K + B_K$ ($\forall i \neq K$)，以保证除主流水级外不存在额外瓶颈，并使瓶颈估计与 1F1B 调度模型一致；（3）**显存约束**：各流水级模型状态与激活显存之和不超过所分配设备的显存容量，防止训练过程中发生 OOM。规划器使用 CPLEX 等整数规划求解器在候选拓扑上分别求解最小化 T^* 的 P ，并取所有候选拓扑中使 T^* 最小的划分与拓扑作为最终并行策略。

主流水级 K 的选取方式为：在给定 P 与拓扑下，计算各流水级的 $F_i + B_i$ （冻结流水级的 B_i 按提前终止规则置零），取使 $F_i + B_i$ 最大的流水级索引作为 K ，以保证 1F1B 下瓶颈估计正确。显存估计中，各流水级占用包含模型参数与优化器状态（仅对可训练参数）、激活（与 micro-batch 大小及序列长度相关）、以及梯度与临时缓冲区；冻结流水级不存储该部分参数的优化器状态与梯度，通信缓冲区也可相应省略，故通信项中其同步开销置零。在实验环境中，单次 IP 求解在候选拓扑上的耗时通常在秒级，整体规划时间可接受；若某拓扑下无可行解（如显存约束无法满足），规划器可放宽主流水级约束或尝试更大 TP 后重新求解。

通过将冻结状态纳入成本模型与通信项，同一 MLLM 在不同训练阶段（如 no-freeze、freeze-encoder、freeze-both）会得到不同的最优划分与设备拓扑，从而随训练进程动态适配，提升异构集群上的训练效率。相关研究已表明，将流水线骨架映射到异构平台的最优划分问题在一般意义下为 NP- 难，本节的整数规划形式化在给定拓扑与块列表下求取近似最优划分，在实践中在求解时间与策略质量之间取得了较好平衡。

3.3 提前终止流水线调度策略

上一节从划分与映射角度刻画了冻结状态对最优并行策略的影响，而在给定某一划分与设备拓扑后，实际训练过程仍受具体流水线调度方式的制约。流水线并行在工业界最常用的是 1F1B（One-Forward-One-Backward）及其变体：各流水级在稳态阶段交替执行前向与反向，对多个 micro-batch 形成“错位”流水。此类调度的一个关键特点是：反向阶段不仅需要完成本流水级对参数的梯度计算，还必须依赖来自后续流水级的输出梯度，以继续向前序流水级回传梯度。

当 MLLMs 中部分模块被冻结时，冻结参数不参与梯度更新，但仍需为前序层计算“对输入的梯度”，从而保证未冻结参数接收到正确的反向信号。直观地看，这会削减冻结流水级的反向计算量，却不会改变“反向必须逐级串行传播”的依赖结构。因此，若仅在成本模型中简单减小冻结层的反向 FLOPs，而不改变调度逻辑，仍然无法充分挖掘冻结带来的潜在加速空间。

MMoH 在此基础上提出提前终止流水线调度器：在不改变损失函数对可训练参数的梯度的前提下，识别并剪枝反向阶段中与冻结前缀相关的冗余计算与 P2P 通信，从而进一步缩短单次迭代时间。本节将详细说明冗余产生的原因、调度器的判定规则与执行流程，以及其对成本模型与划分策略的反馈作用。

3.3.1 冗余识别与剪枝规则

在标准反向传播中，每个流水级需完成两类计算：**对参数的梯度**（用于更新该流水级参数）与**对输入的梯度**（用于向上一流水级传递梯度）。对于冻结流水级而言，对参数的梯度不再需要计算，但在一般设置下仍须计算“对输入的梯度”，以便驱动前序流水级执行反向；这一点与传统“仅冻结优化器状态”的实现方式一致。

然而，当存在“前缀冻结”时，上述惯用实现会引入可以被安全删除的冗余计算。设流水级从 0 到 $S-1$ 依次排列，若流水级 i 及其之前所有流水级 $(0, \dots, i-1)$ 均为冻结流水级，则可以注意到：

- 流水级 i 的输出仅被后续流水级的前向与反向使用，而不会再被任何可训练参数直接消费；
- 损失函数 $\mathcal{L}$ 对冻结前缀中任一参数 θ_f 的梯度 $\partial\mathcal{L}/\partial\theta_f$ 始终被约束为零；
- 对输入的梯度 $\partial\mathcal{L}/\partial x_f$ 仅用于驱动更早的冻结流水级执行反向，而这些反向本身也不会产生非零的参数梯度。

由此可以得到一个关键结论：若某流水级及其前缀均为冻结流水级，则该流水级在反向阶段所参与的“输出梯度计算与回传”对任何可训练参数的最终梯度均无影响，仅产生数值上为零或被丢弃的中间结果。换言之，冻结前缀中的反向链路不会再通向可训练参数的更新路径，其梯度信息既不参与参数更新，也不影响后续非冻结流水级的梯度正确性。因此，在数学上可以安全省略这部分计算与通信，而不改变未冻结参数的梯度。

据此，MMoH 将冗余识别与剪枝规则概括为：

若存在整数 D ，使得流水级 $0, \dots, D-1$ 均为冻结流水级，且流水级 D 为第一个包含可训练参数的流水级，则对任意 **micro-batch**，流水级 $0, \dots, D-1$ 在完成前向计算后，无需再执行针对该 **micro-batch** 的反向计算及与后续流水级相关的 **P2P** 通信。

上述规则直接导致两类收益：一是计算剪枝，即对冻结前缀不再调度反向算子，减少了 **kernel** 启动与算子执行时间；二是通信剪枝，即省略对应的激活缓存与梯度 **P2P**，降低显存占用和链路带宽消耗。由于冻结前缀常覆盖编码器与部分早期 LLM 层，在典型的 MLLM 训练阶段（如 **freeze-encoder**、**freeze-both**）中，这两类收益对端到端迭代时间具有可观影响。

3.3.2 调度流程与实现细节

在具体实现上，提前终止调度器并不修改 1F1B 的整体时间步组织方式，而是对每个时间步中“需要执行反向与通信的流水级集合”做动态裁剪。其执行流程可概括为三个步骤：

1. **冻结前缀检测**。在每个训练阶段开始时（例如切换冻结策略后），调度器读取由用户或上层框架提供的“流水级—冻结标记”数组，从流水级 0 起顺序扫描，直至遇到第一个包含可训练参数的流水级，将其索引记为 D 。若首流水级即为可训练流水级，则 $D = 0$ ，退化为标准 1F1B 调度。
2. **时间步级调度裁剪**。对于每个时间步 t 与 **micro-batch** m ，调度器首先按照标准 1F1B 规则计算“应当执行前向 / 反向的流水级集合”。在此基础上，对所有索引 $i < D$ 且被标记为“反向”的任务直接跳过，仅保留其前向任务。对应的 **P2P** 接收与发送操作也一并取消，避免在运行时创建多余通信。
3. **资源复用与同步**。由于冻结前缀在反向阶段不再参与某些 **micro-batch** 的计算，其 GPU 资源可以更早地被下一轮 **micro-batch** 的前向阶段复用。调度器在实现时通过调整 CUDA stream 与 NCCL group 的启动顺序，确保这种“前向抢占”不会破坏 1F1B 对有依赖张量的先后约束。

图 3.6 给出了一个简化示例：在 4 流水级、4 个 **micro-batch** 的场景下，当前两流水级为冻结流水级时，子图 (a)、(b)、(c) 分别为标准 1F1B、冻结流水级反向耗时缩短但调度骨架仍为 1F1B 的情形，以及采用提前终止调度后跳过冻结前缀上对输入的梯度计算及相应 **P2P** 通信与张量驻留的情形；相较 (a)、(b)，(c) 在时间轴上压缩了与冻结前缀相关的反向与通信区间，从而缩短迭代长度。

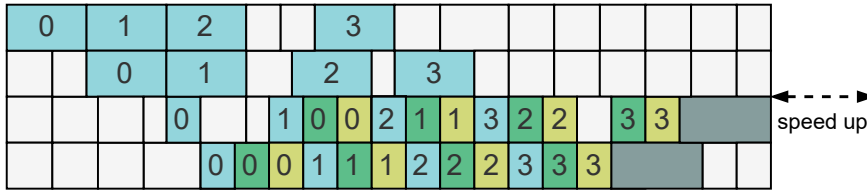
从接口层面看，MMoH 在 Megatron-LM 等框架之上增添了一层轻量级调度控制：原有训练脚本只需在每个流水级维护一个布尔型“是否包含可训练参数”的标记，调度器即可在运行时据此自动完成冻结前缀检测与裁剪，无需用户显式编写新的反向算子。



(a) 1F1B pipeline scheduling when no stage is frozen



(b) 1F1B pipeline scheduling when the first two stages are frozen



(c) early-termination scheduling when the first two stages are frozen



图 3.6 提前终止调度器效果示意。

3.3.3 对成本模型与划分策略的影响

在存在连续冻结前缀（前 D 个流水级）的情况下，迭代时间 T^* 的表达式需与前述调度行为保持一致。采用提前终止后，前 D 个流水级在单轮迭代中仅贡献前向时间，其反向与梯度同步开销视为零，因此 T^* 可改写为

$$T^* = \sum_{i=1}^D F_i + \sum_{i=D+1}^{K-1} (F_i + B_i) + 2 \sum_{i=K+1}^S (F_i + B_i) + (M - (S - K))(F_K + B_K) + \text{COMM}. \quad (3.6)$$

其中 D 为流水线首端连续冻结流水级的个数, K 为非冻结流水级中计算时间 $F_i + B_i$ 最大的流水级索引; COMM 仍取各流水级不可重叠的梯度同步开销的最大值, 且对前 D 个流水级有 $AR_i = 0$, 以便在 1F1B 下正确估计瓶颈与可重叠程度。

与未引入提前终止的情形相比, 上式在两个方面影响了整数规划阶段的最优解形态: 一方面, 冻结前缀上的反向与通信被完全剪枝, 使得将更多计算量“推向”这部分流水级不会增加迭代时间, 从而在求解过程中自然倾向于将一部分重计算块划分到冻结前缀, 以减轻后续非冻结流水级的负载; 另一方面, 通信项中对冻结前缀 AR_i 的归零, 使得主瓶颈更有可能出现在包含大量可训练参数的后段流水级上, 这与实际训练中“解冻阶段成为性能瓶颈”的经验观察是一致的。

从全局角度看, 提前终止调度器与冻结感知的模型划分形成了一个闭环: 划分阶段根据冻结状态与调度模型预估各流水级的前向 / 反向与通信时间, 调度阶段在运行时据此执行提前终止以兑现模型中的剪枝收益; 反过来, 调度规则的改变又会影响成本模型的具体形式, 使得后续阶段的划分能够更准确地利用冻结前缀带来的冗余空间。

需要强调的是, 提前终止思想并不局限于 1F1B 调度本身, 而是与调度框架正交。对于 Chimera、Hanayo 等近年来提出的双向或波式流水线调度机制, 只要其反向阶段仍遵循“从后向前逐级传播梯度”的基本模式, 就可以在对应的时间步上应用同样的冻结前缀检测与任务裁剪逻辑。图 3.7 给出了在 Chimera (双向流水线) 与 Hanayo (波式流水线) 下的示意图: 在前段连续冻结流水级上, 同样可以提前终止反向与通信, 以进一步缩短迭代时间。

3.3.4 实现要点

MMoH 的规划器与调度器可与 Megatron-LM[37] 及 PyTorch 分布式后端集成: 规划器在给定集群与模型配置下离线输出设备拓扑与块一流水级映射, 训练脚本据此构建 PP 通信组与各 rank 上的子模块; 调度器在运行时根据各流水级的冻结标记决定是否调度反向与 P2P。Profiling 在代表设备上以“层”或“块”为粒度测量前向 / 反向 FLOPs 与显存占用, 结果缓存在配置文件中供规划器重复使用。整数规划采用 CPLEX 求解; 若某拓扑下无可行解, 可放宽主流水级约束或增大 TP 后重新求解, 作为回退策略。

3.4 实验验证

本节从实验设置、实验结果与实验分析三方面对 MMoH 进行验证, 并与 Megatron-LM、Whale、Espresso 等基线方法进行对比。

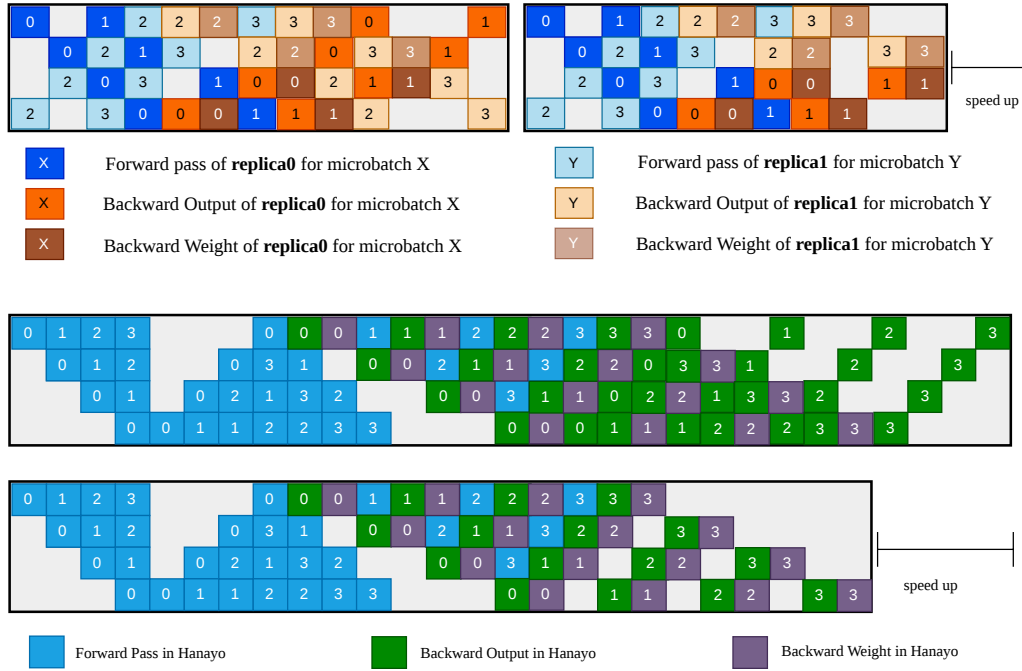


图 3.7 提前终止在其它流水线调度机制上的适用性。上：Chimera（双向流水线）；下：Hanayo（波式流水线）。

3.4.1 实验设置

实验平台与异构集群配置。 实验在两种异构 GPU 集群上进行。**Cluster-S** 包含 4 个节点、共 16 张 GPU：1 个节点配备 4 张 NVIDIA RTX 3090，2 个节点各配备 4 张 RTX 2080 Ti，1 个节点配备 4 张 RTX 2080，单机总显存约 216 GB。**Cluster-L** 包含 8 个节点、共 32 张 GPU：4 个节点为 RTX 3090、2 个节点为 RTX 2080 Ti、2 个节点为 RTX 2080，单机总显存约 536 GB。节点间通过 100 Gbps InfiniBand 互联；RTX 3090 节点内 GPU 通过 PCIe 4.0 互联，RTX 2080/2080 Ti 节点内通过 PCIe 3.0 互联；各节点均为双路 Intel Xeon 4310@2.10 GHz、128 GB DDR4 内存。软件环境为 Ubuntu 18.04、CUDA 11.8、cuDNN 8.7.0、NCCL 2.19.3 与 PyTorch 2.2。表 3.1 汇总两种异构集群的节点与显存配置。

模型与数据集。 为验证 MMoH 在不同规模与结构下的表现，构建两种 MLLMs。**MLLM-3B** 由 ViT 编码器（约 0.2B 参数、24 层）、MLP-S 投影层（约 73M 参数）与 Mistral 2.8B Backbone（12 层）组成；**MLLM-12B** 由 InternViT 编码器（约 6B 参数、45 层）、MLP-L 投影层（约 149M 参数）与 Yi 6B Backbone（20 层）组成。MLLM-3B 在 Cluster-S 上训练，MLLM-12B 在 Cluster-L 上训练，以与集群总显存规模相匹配。训练数据采用 LLaVA Visual Instruct 数据集（约 60 万条图文指令数

表 3.1 实验用异构 GPU 集群配置

集群类型 (GPU 数)	节点编号	GPU 型号 (每节点张数)	总显存 (GB)
Cluster-S (16)	1	RTX 3090 (4)	216
	2, 3	RTX 2080 Ti (4)	
	4	RTX 2080 (4)	
Cluster-L (32)	1-4	RTX 3090 (4)	536
	5, 6	RTX 2080 Ti (4)	
	7, 8	RTX 2080 (4)	

据), 用于模态对齐与指令微调; 训练时采用与 LLaVA 类似的采样与数据混合策略 [11]。关键超参数方面: MLLM-3B 的 global batchsize 为 64、micro-batchsize 为 2、最大序列长度 1024; MLLM-12B 的 global batchsize 为 32、micro-batchsize 为 1、最大序列长度 1024, 以在异构集群显存约束下尽可能提高有效批量并避免 OOM。表 3.2 汇总两种 MLLM 各模块的参数量与层数配置。

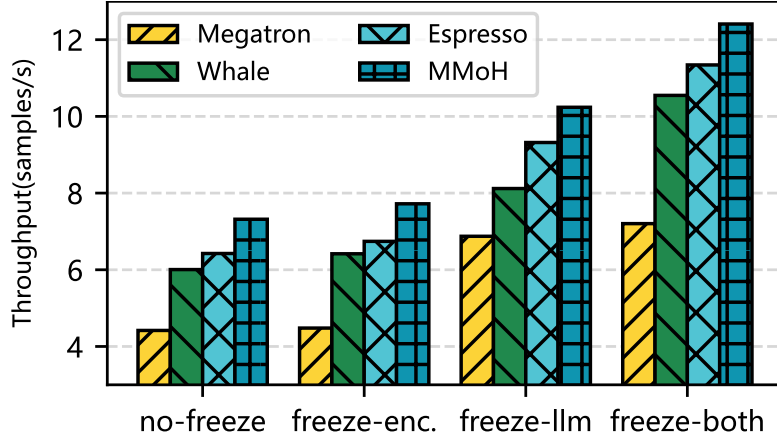
表 3.2 实验用 MLLM 模块配置 (模型参数)

MLLM	模块	类型	参数量	层数
MLLM-3B	ViT	编码器	0.2B	24
	MLP-S	投影层	73M	2
	Mistral	LLM Backbone	2.8B	12
MLLM-12B	InternViT	编码器	6B	45
	MLP-L	投影层	149M	2
	Yi	LLM Backbone	6B	20

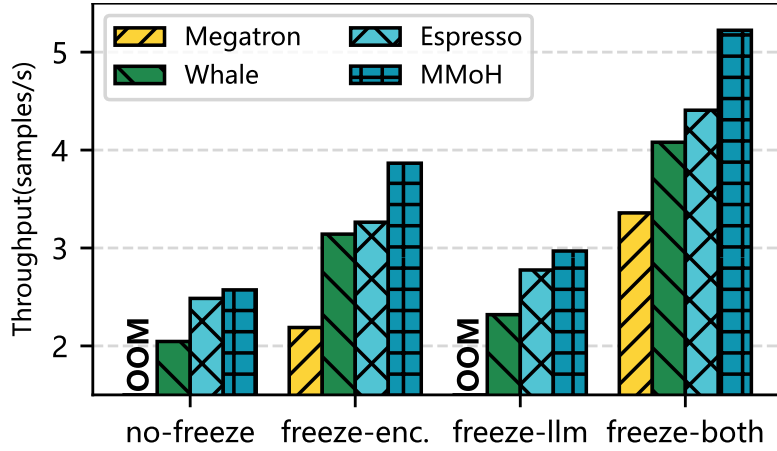
对比方法与评价指标。对比方法包括: (1) **Megatron-LM**[37], 面向同构集群的流水线划分框架, 在异构环境下沿用 **Whale** 的设备拓扑以缓解 OOM; (2) **Whale**[53], 按显存与算力组织设备并做层级划分; (3) **Espresso**[82], 基于计算一显存比 (CMR) 的流水级放置与 **Whale** 式划分相结合。评价指标为训练吞吐 (samples/s)。每种配置进行 10 轮预热与 50 轮测量, 取稳定流水级的平均吞吐作为报告值; 若发生 OOM, 则尝试提高 TP、降低 DP, 若 TP=4 仍 OOM 则记为不可行。

3.4.2 实验结果

在 no-freeze、freeze-encoder、freeze-llm、freeze-both 四种冻结场景下，MMoH 在两种集群与两种模型上均取得最高吞吐，且在不同场景下均能自动调整设备拓扑与划分策略。图 3.8 给出了两种集群与两种 MLLMs 下的端到端吞吐对比，柱长表示吞吐（samples/s），OOM 表示该配置下发生显存溢出无法训练。



(a) MLLM-3B 在 Cluster-S 上



(b) MLLM-12B 在 Cluster-L 上

图 3.8 四种冻结场景下 MMoH 与基线在两种异构集群上的训练吞吐对比。柱越长表示吞吐越高，OOM 表示显存溢出。

MLLM-3B 在 Cluster-S 上。 相对 Whale 与 Espresso，MMoH 在 no-freeze 下分别达到约 $1.22\times$ 与 $1.07\times$ 的加速；在 freeze-both 下优势更明显，可达约 $1.18\times$ – $1.72\times$ 。Megatron 在所有场景下吞吐最低，因其同构假设导致负载不均；在部分场景下需将 TP 升至 4 方能训练，频繁的 TP 通信进一步限制性能。

MLLM-12B 在 Cluster-L 上。 在 no-freeze 与 freeze-llm 下，Megatron 因粗粒度划分导致的负载不均而出现 OOM，无法完成训练；Whale 与 Espresso 通过异构感知的拓扑与划分避免了 OOM，但 MMoH 仍取得最高吞吐。在 no-freeze 下，MMoH 相对 Espresso 与 Whale 分别约 $1.04\times$ 与 $1.26\times$ ；在 freeze-encoder、freeze-both 下，提前终止调度器带来显著增益，MMoH 相对 Megatron、Whale、Espresso 可达约 $1.19\times$ – $1.77\times$ 的加速。上述结果说明，MMoH 的复合粒度划分与冻结感知策略在不同规模与冻结场景下均能有效提升异构集群上的训练效率。

3.4.3 实验分析

划分策略对比。 Whale 与 Espresso 采用单一“层”粒度划分并扩展到 MLLMs；MMoH 采用复合粒度（编码器合并为块、LLM 拆分为注意力/FFN 块）。表 3.3 对比了 Whale、Espresso 与 MMoH 在 MLLM-3B、MLLM-12B 及四种冻结场景下的设备拓扑、各流水级块数及相对 Whale 的加速比；表中 A、B、C 分别表示 NVIDIA RTX 3090、RTX 2080 Ti 与 RTX 2080 GPU。在相同冻结场景下，MMoH 得到的设备拓扑与各流水级块数分布随冻结状态变化：例如 freeze-encoder 时前段计算需求下降，MMoH 会将较弱 GPU 置于首流水级；freeze-llm 时后段需求下降，设备顺序相应调整，与需求—供给匹配度一致。该动态调整能力是现有方法所不具备的，也是 MMoH 在多种场景下均能取得较优性能的重要原因。

表 3.3 不同冻结场景下的模型划分对比。

冻结模式	Whale（层粒度）		Espresso（层粒度）		MMoH（复合粒度）	
	MLLM-3B	MLLM-12B	MLLM-3B	MLLM-12B	MLLM-3B	MLLM-12B
no-freeze	[A,B,B,C]	[A,A,A,A,B,B,C,C]	[B,B,C,A]	[B,B,C,C,A,A,A,A]	[B,A,B,C]	[B,B,C,C,A,A,A,A]
	[30,3,3,2]	[15,15,17,9,3,3,2]	[26,3,2,7]	[7,7,6,6,15,11,8,7]	[11,14,6,4]	[7,7,6,6,18,16,13,15]
	1.00×	1.00×	1.07×	1.22×	1.22×	1.26×
freeze-encoder	[A,B,B,C]	[A,A,A,A,B,B,C,C]	[B,B,C,A]	[B,B,C,C,A,A,A,A]	[C,A,B,B]	[C,C,B,B,A,A,A,A]
	[31,3,3,1]	[38,11,5,5,2,3,2,1]	[28,3,2,5]	[8,8,6,6,22,13,13,12]	[12,13,6,4]	[12,11,11,11,11,11,10]
	1.00×	1.00×	1.05×	1.04×	1.20×	1.23×
freeze-llm	[A,B,B,C]	[A,A,A,A,B,B,C,C]	[B,B,C,A]	[B,B,C,C,A,A,A,A]	[B,B,A,C]	[C,C,B,B,A,A,A,A]
	[27,4,4,3]	[13,13,13,13,4,4,3]	[14,14,3,7]	[3,3,3,3,13,13,17,12]	[7,8,16,4]	[3,3,5,5,13,14,23,22]
	1.00×	1.00×	1.15×	1.20×	1.26×	1.33×
freeze-both	[A,B,B,C]	[A,A,A,A,B,B,C,C]	[B,B,C,A]	[B,B,C,C,A,A,A,A]	[B,A,B,C]	[A,B,B,C,C,A,A,A]
	[30,3,3,2]	[38,9,5,5,3,3,2,2]	[28,2,2,6]	[18,5,4,4,16,7,7,6]	[14,14,4,3]	[36,4,5,3,4,14,12,10]
	1.00×	1.00×	1.08×	1.20×	1.18×	1.28×

负载均衡。 在 MLLM-12B 上对比各流水级前向计算时间分布可知，MMoH 的划分使各流水级计算时间分布更集中、峰值更低，迭代时间由瓶颈流水级主导的程度减轻，说明复合粒度与冻结感知划分有效平衡了异构设备上的负载（图 3.9）。

在 no-freeze 与 freeze-llm 等场景下，MMoH 在流水级 5–7 等位置采用更细粒度划分，平滑了负载尖峰，相较 Espresso 进一步缩短了迭代时间。

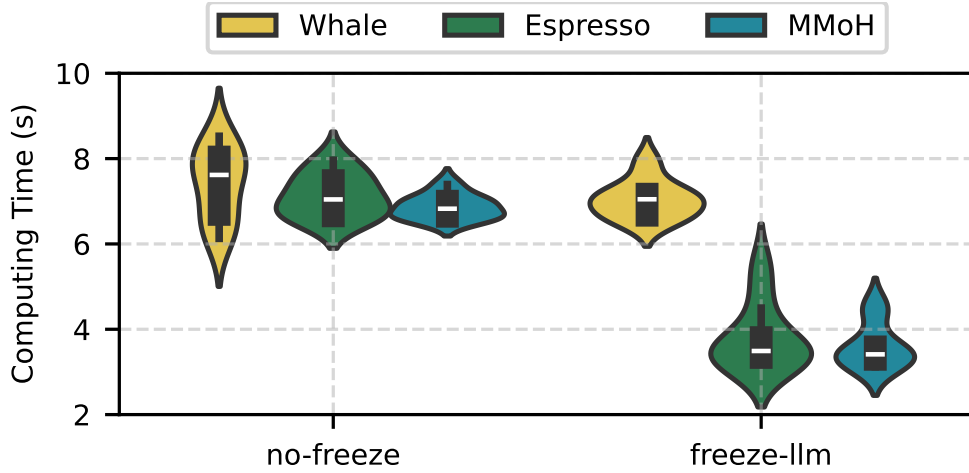


图 3.9 MMoH 与 Whale、Espresso 在 MLLM-12B 上、两种冻结设置下的负载均衡对比。分布越集中表示划分越均衡，柱高越低表示迭代时间越短。

提前终止调度器消融。 为隔离提前终止调度器的贡献，将 MMoH 的调度器接入 Espresso（记为 Espresso+）或从 MMoH 中移除（记为 MMoH-），在 freeze-encoder、freeze-both 下对比。结果表明：Espresso+ 相对 Espresso 有明显提升，MMoH 相对 MMoH- 也有约 8%–11% 的提升；完整 MMoH（规划器 + 调度器）相对仅使用规划器的 MMoH- 可再提升约 8%–19% 吞吐。这表明该调度器在冻结场景下能有效消除冗余计算与通信，且与规划器协同可进一步放大收益；同时，将调度器接入 Espresso 也能带来增益，说明提前终止策略具有较好的可移植性。图 3.10 给出了 freeze-encoder 与 freeze-both 下 Espresso、Espresso+、MMoH- 与完整 MMoH 的吞吐对比及计算开销分布。

局限与扩展。 MMoH 当前在 1F1B 等同步流水线调度下验证；与 Zero-Bubble 等异步或重叠通信的流水线调度 [98]、以及 DualPipe 等双向流水线 [99] 的结合，有望在通信密集型场景下进一步缩短迭代时间，留作未来工作。此外，规划器依赖离线或按阶段 profiling，若训练过程中动态改变 batchsize 或序列长度，需重新求解划分；第四章将介绍的激活重计算优化与 MMoH 的划分与调度正交，可在同一流水线策略下叠加使用，实现显存与计算—通信的联合优化。

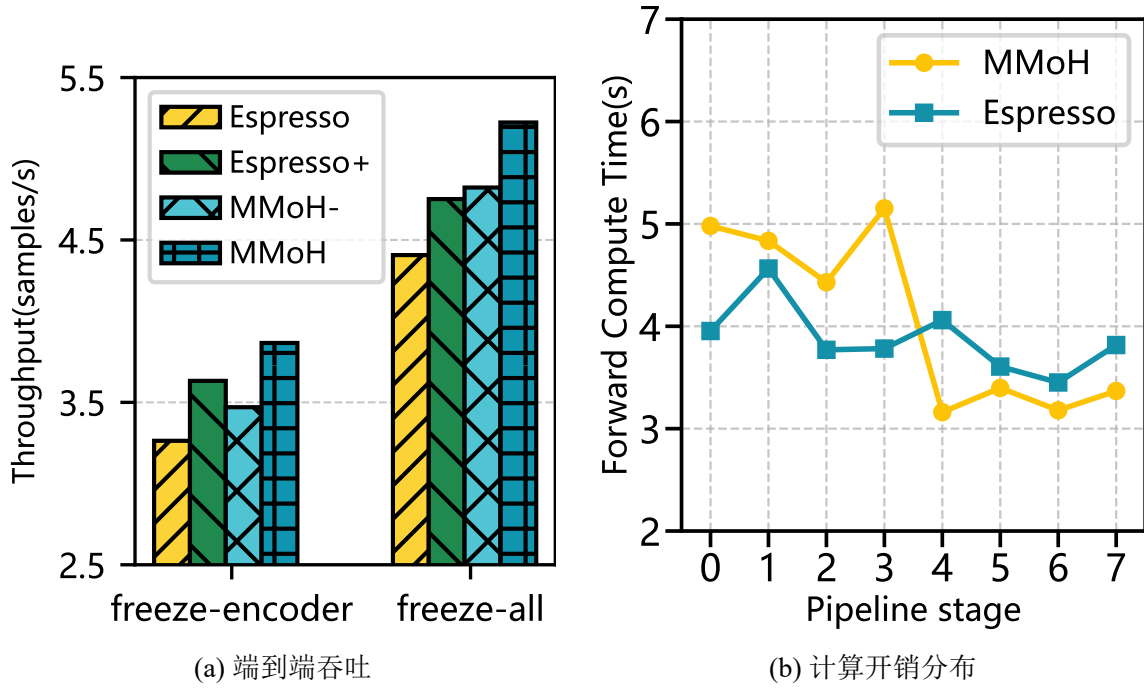


图 3.10 提前终止调度器消融。Espresso+ 为接入该调度器的 Espresso，MMoH- 为去掉该调度器的 MMoH。

3.5 本章小结

本章介绍了面向异构 GPU 集群的多模态大语言模型高效混合并行框架 MMoH。针对 MLLMs 的混合模块结构与分流水级冻结带来的计算—显存需求非均匀性与动态变化，MMoH 通过需求驱动的设备拓扑识别、复合粒度模型抽象与冻结感知的整数规划划分，在给定集群与模型配置下自动生成与当前训练流水级相匹配的流水线并行策略；通过提前终止调度器在连续冻结前缀流水级省略不必要的反向计算与 P2P 通信，在保持收敛性的前提下进一步提升吞吐。在两种异构集群（Cluster-S、Cluster-L）与两种规模 MLLMs（MLLM-3B、MLLM-12B）上的实验表明，MMoH 相对 Megatron-LM、Whale、Espresso 等现有方法可获得最高约 $1.77\times$ 的加速，且在不同冻结场景下均能自适应地调整设备拓扑与划分策略。本章将问题形式化为迭代时间由瓶颈流水级主导、以及冻结集合导致的反向与通信不对称，并通过规划器（需求驱动拓扑、复合粒度、冻结感知 IP）与提前终止调度器的协同，在异构集群上实现了与训练阶段相匹配的划分与调度；后续工作可将提前终止与 Zero-Bubble、DualPipe 等调度结合，并与第四章的激活重计算优化叠加，为多模态大模型在资源受限与异构环境下的高效训练提供完整方案。

第四章 HR-MMoH: 多模态大语言模型在异构 GPU 集群上的重计算策略

第二章讨论了多模态大语言模型（MLLMs）在模块异构与分阶段冻结下算力与显存需求的差别；第三章借助 MMoH 在异构集群上完成了设备拓扑识别、复合粒度划分与冻结感知调度，从划分与流水调度侧提升了训练效率。然而各设备显存容量与可承受批量并不一致，仅靠划分与调度仍难以在各流水级上同时消化长序列与大批量带来的激活驻留，显存往往仍是制约端到端可训练配置的关键因素。本章提出 HR-MMoH（Heterogeneous Recomputation for training multimodal large language models on heterogeneous GPU clusters），在 MMoH 输出结果之上引入流水级差异化的重计算配置，与第三章形成“划分—调度—重计算”相继衔接的优化链条。本节首先归纳异构重计算的问题背景与动机，随后给出 HR-MMoH 框架的总体架构与各模块职能。

4.1 引言

4.1.1 问题与动机

第三章的 MMoH 框架借助需求驱动的设备拓扑、复合粒度划分与提前终止流水线调度，已在异构 GPU 集群上降低了 MLLM 训练迭代时间。但集群内显存与算力并不均匀：在显存偏紧的节点上，单靠流水线划分有时仍撑不起更大的批量或更长的序列，显存仍是主要制约。重计算（Recomputation）亦称梯度检查点（Gradient Checkpointing）[104]：前向阶段有意少存或不存部分中间激活，反向时再补做一次前向以恢复所需激活，用额外计算换取常驻显存下降。在数学上与常规训练等价的前提下，它便于在更大批量、更长序列或更大模型上继续训练，因而在研究与工程实践中被普遍采用。

Megatron-LM 等主流实现里的重计算选项，通常按同构场景来写：一次训练里，各流水级、各设备共用同一套重计算粒度（整层 full、子模块 selective 等）、同一套层级切分方式（如 uniform、block），以及同一组重算子模块（如仅 core_attn，或 core_attn+mlp+layernorm）。

同构假设搬到异构集群上会失配：显存吃紧的流水级若仍用偏粗粒度或偏少的重算，容易 OOM；显存宽裕的流水级若被迫与前者对齐，又会在本可存激活的地方多做重算，拖长迭代、空耗算力。换言之，全局一致的检查点策略很难在设备能力参差不齐时同时管好显存与效率。

针对上述矛盾，本章在 MMoH 已给出的流水级划分与调度方案之上，为各

流水级分别设定重计算粒度、分块方式与子模块集合：显存压力大的阶段收紧检查点以降低驻留激活，显存宽裕的阶段则减少不必要的重算以节省时间，从而在OOM 风险与迭代开销之间取得更细粒度的折中。

4.1.2 HR-MMoH 框架概览

HR-MMoH 以第三章 MMoH 的并行策略规划器与提前终止调度器的输出为前置条件：在确定的设备拓扑、流水级映射与冻结感知调度规则下，进一步为每个流水级求解一组重计算超参（粒度、uniform/block 切分、层块大小、recompute_modules 列表等），使各级在本地显存预算内尽可能少做冗余重算。图 4.1 给出了 HR-MMoH 与 MMoH 的层次关系及数据流：上游为 MLLM 配置与异构集群规格，经 MMoH 得到划分与调度；HR-MMoH 在训练脚本与引擎侧注入按流水级区分的重计算策略，并与冻结状态及批量、序列长度等运行配置联动。

与 Megatron-LM 默认的全局统一重计算策略相比，HR-MMoH 显式利用各流水级显存余量与算力差异，避免部分计算设备显存不足部分多余重计算的两难问题，与仅在单卡或同构假设下手工调参相比，该框架将流水级维度的配置与 MMoH 的规划结果对齐，便于在异构集群上复现与扩展。

第 4.2 节给出重计算策略的形式化描述、定量分析与实现要点；第 4.3 节报告实验设置、结果与讨论；第 4.4 节对本章工作进行小结。

4.2 异构重计算

4.2.1 现有的重计算策略与定量分析

Megatron-LM[36, 37, 78] 是大模型分布式训练的常用基座之一，其并行训练栈里对重计算有一组固定配置项，训练脚本通过 `-recompute-granularity`、`-recompute-method`、`-recompute-num-layers`、`-recompute-modules` 等参数下发。

表 4.1 归纳了与重计算直接相关的主要字段及含义。其中 `recompute_granularity` 决定整层(full)或子模块级(selective)重计算;full 时须同时指定 `recompute_method` 与 `recompute_num_layers`,selective 时二者须为 None,仅由 `recompute_modules` 指定要重算的子模块。`distribute_saved_activations` 为 True 时，可将 checkpoint 保存的激活在张量并行组内分片存储以进一步省显存，但仅对 full 粒度且未启用序列并行的情形有效。

整层重计算 (full)：以层为 checkpoint 单元，前向在每个块边界只留输入，反向时把该段前向重跑一遍再接反传。full 下由 `recompute_method` 与 `recompute_num_layers` 共同规定“隔多少层留一个激活”。`recompute_method`

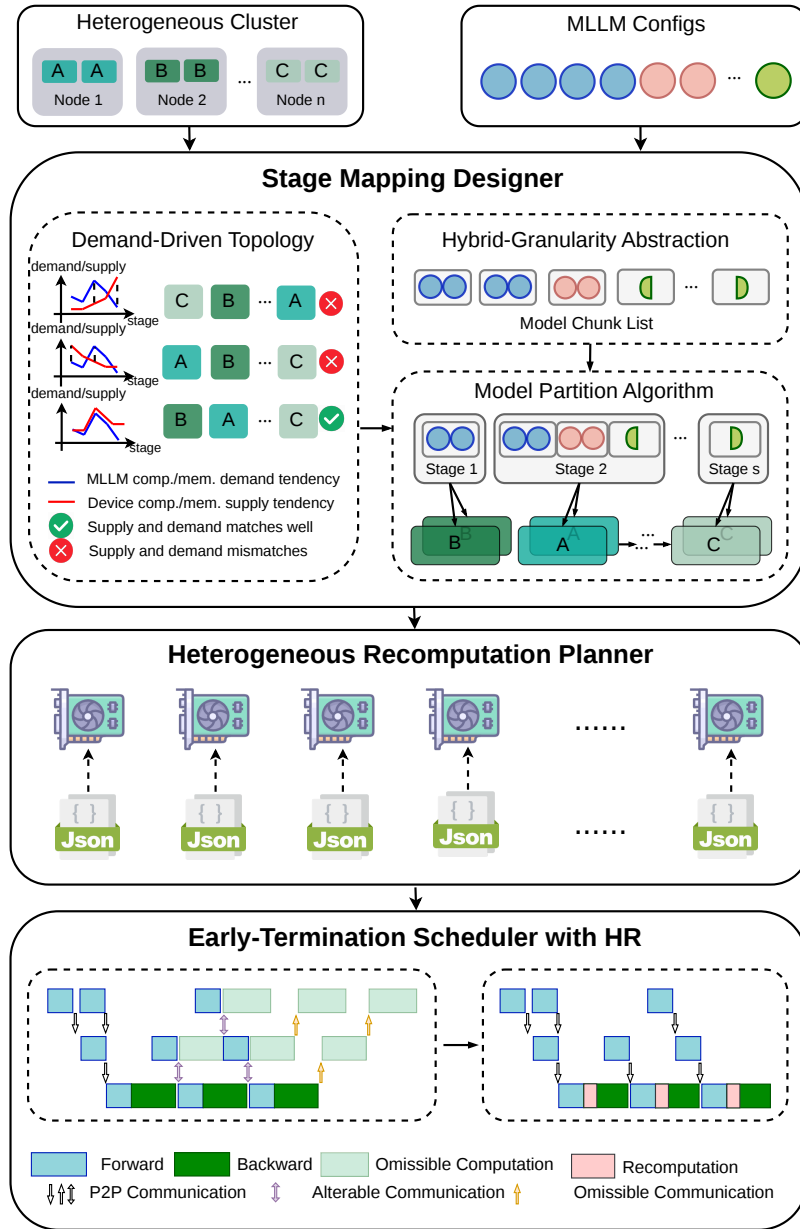


图 4.1 HR-MMoH 框架总览及其与 MMoH 的衔接关系。MMoH 负责划分与冻结感知调度；HR-MMoH 在各流水级上配置差异化的重计算策略，以进一步适配异构显存与算力。

有两种典型取法：

- **uniform**（均匀分块）：将当前流水级内的所有层均匀分成多个 chunk，每个 chunk 包含 `recompute_num_layers` 层；每个 chunk 的输入被 checkpoint，chunk 内前向一次执行，反向时整个 chunk 先重计算再反传。chunk 数少则显存更省，但单次重算层数较多。
- **block**（仅前 N 层）：在当前流水级内，仅对前 `recompute_num_layers`

表 4.1 Megatron-LM 重计算相关配置参数

参数	类型 / 取值	说明
recompute_granularity	None full selective	重计算粒度。None 表示不重计算；full 对整层做重计算；selective 仅对 recompute_modules 中列出的子模块重算。
recompute_method	None uniform block	仅用于 full 粒度：如何划分“哪些层”做 checkpoint。uniform 为均匀分块，block 为仅对每流水级前若干层做 checkpoint。 selective 时必须为 None 。
recompute_num_layers	None 正整数	仅用于 full 粒度。uniform 时表示每个 chunk 的层数；block 时表示每个 stage 内做重计算的层数。 selective 时必须为 None 。
distribute_saved_activations	布尔	为 True 时，将 checkpoint 保存的激活在张量并行组内分片存储；仅 full 粒度且非序列并行时有效。
recompute_modules	字符串列表	仅用于 selective 。要重算的子模块名列表（见下文）。默认 ["core_attn"]。

层逐层重算；其后各层不重算，中间激活照常保留。显存宽裕时可用它少做重算，把激活留在显存里。

选择重计算 (selective) 只在层内对 recompute_modules 列出的子模块做重算，其余算子仍按常规模式存激活。这时 recompute_method 与 recompute_num_layers 必须置为 None；selective 会作用到所有流水级上的每一层。各子模块按 recompute_modules 决定前向里哪些中间结果可以不常驻、反向里再补算。

子模块可选项由 recompute_modules 给出。实现上分两类：一类为标准 checkpoint（留输入、反向重前向）；另一类为 output-discarding（前向结束即丢输出占用，反向借梯度钩子等机制再算回），后者更省显存。未显式配置时默认 ["core_attn"]；稠密模型里常见的“压显存”组合是 [core_attn, mlp, layernorm]。表 4.2 列出常用模块名、语义及所用 checkpoint 类型。

上述接口在实现上把同一作业内的各流水级、各 GPU 绑在同一套粒度、方法、层数与模块列表上，相当于默认同构。异构场景下，这套“一刀切”会带来两类毛病：显存紧的级仍可能 OOM，显存松的级又被迫多算；同时各级的重算增

表 4.2 Megatron selective 粒度下可配置的子模块

模块名	含义	Checkpoint 类型
core_attn	核心注意力 (QKV→Attention→ 输出)	标准
mlp	稠密 MLP (非 MoE)	标准
layernorm	输入 LayerNorm 与 MLP 前 LayerNorm	Output-discarding
moe	整块 MoE 层 (router + experts + combine)	标准
shared_experts	MoE 的共享专家	标准
moe_act	MoE 专家内部激活 (如 SiLU) 与专家权重	Output-discarding
mha_up_proj	MLA 的 QKV 上投影与 RoPE	Output-discarding

量相近，弱卡更容易在迭代时间上掉队。后文据此讨论如何按流水级拆开配置。

重计算技术的定量分析。 为比较各策略下常驻激活规模，本文在固定输入形状与 BF16 前提下，对单层 Transformer 反向所需中间激活的占用作逐项估算。设层输入张量 X 形状为 $[B, S, H]$ (B 为 batchsize, S 为序列长度, H 为隐藏维)，激活按 BF16 计，每元素 2 字节。结构取 Pre-LayerNorm 的 decoder-only 单层： X 经 LayerNorm 进注意力，残差后再 LayerNorm 进 MLP (线性 → GELU → 线性 + Dropout)，再残差得到层输出。注意力头数记为 A ；MLP 中间维取常见 $4H$ 。

为避免与第三章中流水线并行度记号 S (流水级个数) 相混淆，本小节以下用 S 表示层输入的序列长度 (token 维长度)。表 4.3 汇总本小节使用的主要记号。

(1) **不重计算 (None)**。以下只统计反向要用到的中间激活，dtype 为 BF16，故每项按 2 字节 / 元素计入。

注意力子层：Q/K/V 共享输入为 $2BSH$ ，计算得到的 Q/K/V 大小均为 $2BSH$ ，共计 $6BSH$ ，计算 QK^T 后进行 softmax 的中间结果为 $2BAS^2$ ，Dropout 的 mask 为 BAS^2 ，Dropout 的输出为 $2BAS^2$ ，最后与 V 相乘的中间结果为 $2BSH$ ，最后的 Dropout 的 mask 矩阵为 BSH ，合计

$$N_{\text{attn}} = 11BSH + 5BAS^2. \quad (4.1)$$

其中 $5BAS^2$ 来自 QK^T 、softmax 与 dropout 等，随 S 平方放大，长序列下往往占主导。图 4.2 示意 None 时注意力子层需要存储的中间激活值以及需要的显存大小。

MLP 子层：中间维度 $4H$ ，第一个线性层的输入为 $2BSH$ ，需要保存的输出为 $8BSH$ ，经过中间激活后第二个线性层的输入为 $8BSH$ ，最后的 Dropout 层的

表 4.3 定量分析小节主要符号说明

符号	含义
B	batchsize
S	输入序列长度
H	模型隐藏维度
A	注意力头数。
X	输入张量，形状为 $[B, S, H]$ 。
h_1	注意力子层与残差连接后的中间张量，形状为 $[B, S, H]$ 。
N_{attn}	单个注意力子层为反向传播保留的中间激活总量。
N_{mlp}	单个 MLP 子层为反向传播保留的中间激活总量。
N_{ln}	单个 LayerNorm 子层为反向传播保留的中间激活总量。
N_{None}	不采用重计算时，单层 Transformer 需保留的中间激活总量。
$N_{\text{selective}}^{\text{core_attn}}$	仅对 core_attn 使用 selective 重计算时，单层需保留的中间激活总量。
$N_{\text{selective}}^{\text{core_attn+mlp}}$	对 core_attn 与 mlp 使用 selective 重计算时，单层需保留的中间激活总量。
$N_{\text{selective}}^{\text{core_attn+mlp+ln}}$	对 core_attn、mlp 与 layernorm 使用 selective 重计算时，单层需保留的中间激活总量。
N_{full}	采用 full 重计算时，单层需保留的中间激活总量。

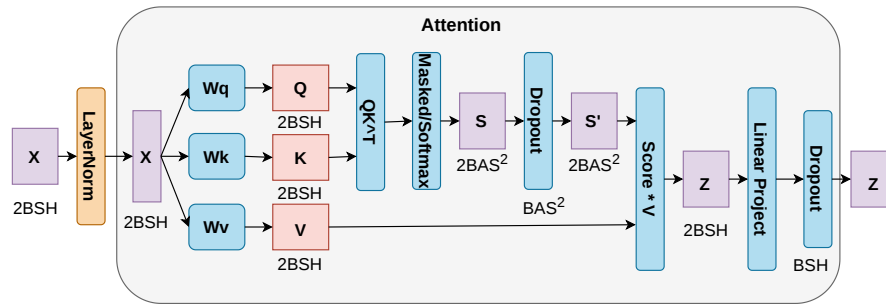


图 4.2 注意力子层中间激活示意

mask 矩阵为 BSH ，合计

$$N_{\text{mlp}} = 19BSH. \quad (4.2)$$

图 4.3 对应给出 MLP 子层在 None 下的激活分解。

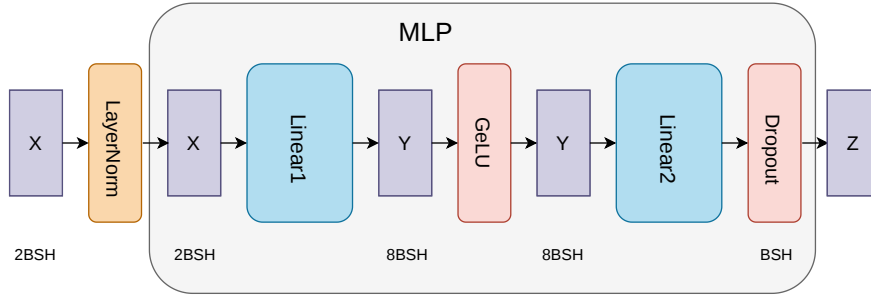


图 4.3 MLP 子层中间激活示意

两个 *LayerNorm*（注意力前、MLP 前）各保存输入 $2BSH$ ，合计

$$N_{\text{ln}} = 4BSH. \quad (4.3)$$

于是，不采用重计算时单层需保存的中间激活总量为

$$N_{\text{None}} = N_{\text{attn}} + N_{\text{mlp}} + N_{\text{ln}} = 34BSH + 5BAS^2. \quad (4.4)$$

其中 $5BAS^2$ 在长序列下仍可能是显存大头。

(2) 选择重计算 (selective)。指定子模块只留输入，反向时在模块内重前向再接反传；未列入 `recompute_modules` 的部分仍按 `None` 全量存激活。若仅对 `core_attn` 开 `selective`，则 $QK^\top$ 、`softmax` 及对应 `mask` 等不再常驻，块边界上的输入（如层输入与 LN 输出）须保留以供重算；MLP 与两处 *LayerNorm* 仍分别付出 N_{mlp} 、 N_{ln} ，整层常驻规模由 $34BSH + 5BAS^2$ 降至约 $23BSH$ 。

再对 `mlp` 开 `selective`，则 MLP 内 $19BSH$ 的激活也推迟到反向再算，仅边界输入常驻，总量约 $6BSH$ 。若把 `layernorm` 一并纳入，LN 输出可在前向末尾丢掉，总量可压到约 $4BSH$ 。与 `None` 对读，记

$$N_{\text{selective}}^{\text{core_attn}} = 23BSH, \quad (4.5)$$

$$N_{\text{selective}}^{\text{core_attn+mlp}} = 6BSH, \quad (4.6)$$

$$N_{\text{selective}}^{\text{core_attn+mlp+ln}} = 4BSH. \quad (4.7)$$

表示不同模块组合下常驻激活规模。相对 `None`，`selective` 把注意力与 MLP 主体里的大块中间结果移出常驻，只留子模块边界输入，显存随规模的量级由 $O(BSH + BAS^2)$ 变为 $O(BSH)$ 。

(3) 整层重计算 (full)。以整层为单元：前向只留层输入 X ，反向整层重前向后再反传。于是

$$N_{\text{full}} = 2BSH. \quad (4.8)$$

常驻显存最省，反向多一整段前向，计算开销最大。

表 4.4 汇总了不采用重计算、选择重计算与整层重计算时单层需保存的中间激活总量。

表 4.4 单层 Transformer 中间激活量

策略	需保存的激活（元素个数）	说明
None	$34BSH + 5BAS^2$	整层所有中间激活均常驻
selective [core_attn]	$23BSH$	仅注意力内部重算，MLP 与 LN 仍常驻
selective [core_attn, mlp]	$6BSH$	注意力与 MLP 内部重算，仅边界与 LN 常驻
selective [core_attn, mlp, layernorm]	$4BSH$	三者均重算，仅两个子模块边界的输入常驻
full	$2BSH$	仅保存层输入，反向时整层重算

4.2.2 异构重计算策略设计

HR-MMoH 将 Megatron-LM 的重计算选项抽象为可按流水级独立取值的离散决策：在保持单卡前向一反向数学语义不变的前提下，通过调节重计算粒度与子模块集合，在算力与显存之间进行折中。本节依次给出显存分解与优化形式、基于剖析的可行配置生成方法、结合冻结状态与流水级位置的启发式、多路分支搜索伪代码，以及与 MMoH 的组合方式。

记号说明。 为与第三章及表 4.3 的记号稍作区分，本节与算法 4.1 中： S 表示流水线并行度（流水级个数）， s 表示流水级编号（与第三章整数规划、提前终止部分相同）；各流水级可用显存上界记为 Q_s ；候选重计算策略数记为 R （本文实现取 $R=6$ ），以区别于第三章主流流水级索引 K ；算法中的非冻结流水级集合记为 $\mathcal{U}$ ；对 $\mathcal{U}$ 排序后得到的遍历序列记为 W_{front} 、 W_{back} 。

显存分解与决策变量。 在 MMoH 规划器已完成模型划分与设备映射的前提下, 流水级 s 映射至设备 d_s , 并由若干模型块 (**chunk**) 组成。单次训练迭代处于稳定调度阶段时, 设备 d_s 上的峰值显存可写为四项之和: (1) 模型参数; (2) 梯度; (3) 优化器状态 (如 Adam 的一阶与二阶矩); (4) 中间激活 (含流水线并行在 1F1B 等调度下需暂存的激活, 其峰值与 **micro-batch** 配置、流水级在流水线中的位置及重计算策略均相关)。在给定划分、优化器类型、数据类型及是否启用张量并行分片等训练设定后, (1) – (3) 通常不随重计算配置 θ_s 改变; 冻结流水级在提前终止语义下还可省略梯度与优化器状态的部分或全部, 具体以第三章调度与实现为准。决策变量 θ_s 在 Megatron-LM 配置空间中取值: `recompute_granularity` $\in \{\text{None}, \text{full}, \text{selective}\}$; 若为 `full`, 还需给定 `recompute_method` 与 `recompute_num_layers`; 若为 `selective`, 则由 `recompute_modules` 指定子模块列表。记流水级 s 在 θ_s 下的四项显存分别为 $\text{Mem}_s^{\text{param}}$ 、 $\text{Mem}_s^{\text{grad}}$ 、 $\text{Mem}_s^{\text{opt}}$ 与 $\text{Mem}_s^{\text{act}}(\theta_s)$; Q_s 为 d_s 上可用于训练的显存上界 (**quota**), 并已在实践中扣除驱动、缓存与碎片等余量。则可行配置需满足: 对任意流水级 s , 四项之和不超过 Q_s 。迭代时间方面, 记 $T_s(\theta_s)$ 为流水级 s 在一次迭代中前向、重计算与反向的本地耗时及该级所承担的流水线通信时间之和 (通信已并入 T_s , 以与流水线瓶颈目标一致), 则整体目标可写为最小化最慢流水级时间 $\max_s T_s(\theta_s)$, 即

$$\begin{aligned} \min_{\{\theta_s\}_{s=1}^S} \quad & \max_{1 \leq s \leq S} T_s(\theta_s) \\ \text{s.t.} \quad & \text{Mem}_s^{\text{param}} + \text{Mem}_s^{\text{grad}} + \text{Mem}_s^{\text{opt}} + \text{Mem}_s^{\text{act}}(\theta_s) \leq Q_s, \quad \forall s \in \{1, \dots, S\}. \end{aligned} \quad (4.9)$$

式 (4.9) 是离散组合问题: θ_s 来自有限集, 穷举随级数指数膨胀。 T_s 与显存峰值还受 **micro-batch**、算子融合等影响, 下文以实测剖析为主。

基于剖析的可行配置生成。 对每个流水级 s , 在型号相同或接近的 GPU 上, 使并行度、精度与 **micro-batch** 等与正式训练保持一致, 对每个候选 θ 采集经过 **warmup** 后的稳态训练片段, 记录峰值显存, 并基于本级平均迭代时间和算子耗时估计 $T_s(\theta)$ 。把测得的 $\text{Mem}_s^{\text{act}}(\theta)$ 与 $\text{Mem}_s^{\text{param}}$ 、 $\text{Mem}_s^{\text{grad}}$ 、 $\text{Mem}_s^{\text{opt}}$ 相加得 $\text{TotalMem}(s, \theta)$, 凡满足式 (4.9) 的 θ 归入可行集 $\mathcal{F}_s$ 。 $\mathcal{F}_s$ 非空时, 若仅按本级最小化 $T_s(\theta)$ 选取 θ , 未必使全局目标 $\max_s T_s$ 最优, 还需避免个别流水级因重计算配置过于激进而显著增大计算耗时, 进而导致出现较多的流水线气泡。本文策略为: 对显存压力较大的流水级优先采用更节省显存的重计算配置以降低 $\text{Mem}_s^{\text{act}}$, 对显存余量较大的流水级则尽量保留较轻配置以抑制重计算, 具体由下一小节的多路分支搜索实现。据此可将典型情形粗分为两类: (1) 显存相对不足: 宜采用 `full` 或子模块覆盖较全

的 selective (如 `[core_attn, mlp, layernorm]`), 以降低 $\text{Mem}_s^{\text{act}}$; (2) 显存相对充足: 宜优先 None、仅对 `[core_attn]` 启用 selective 以控制 T_s 无意义增大。

冻结状态与流水级位置的启发式。 结合第三章冻结训练与提前终止调度, 可在求解式 (4.9) 时引入启发式先验, 以压缩搜索空间并增强配置结果的可解释性。

(1) 冻结流水级。冻结流水级通常不执行完整反向传播与优化器更新 (具体以 MMoH 调度为准), 梯度与优化器状态所占显存比例下降, 峰值显存主要由模型参数与流水线并行下暂存的中间激活决定。此时若采用过强的重计算策略, 额外开销主要体现在重复前向计算: 在最坏情形下近似于在该流水级增加与重计算粒度相当的前向传播; 相对于冻结流水级单次前向为主的计算特征, 该比例可能显著偏高。因此默认优先选取 None 或仅覆盖少量子模块的 selective, 除非剖析表明 $\text{Mem}_s^{\text{act}}$ 仍接近 Q_s 。

(2) 非冻结流水级。非冻结流水级需执行完整反向传播并维护优化器状态, $\text{Mem}_s^{\text{grad}}$ 与 $\text{Mem}_s^{\text{opt}}$ 上升, $\text{Mem}_s^{\text{act}}$ 。在反向传播计算开销约为前向传播开销的 2 倍的粗略假设下, 整层重计算可近似视为额外引入与前向相当的计算, 其相对于原前向与反向总工作量的占比约为三分之一。非冻结流水级通常较冻结流水级更能承受此类计算增量, 故在可行集非空时更易选取 full 或 `recompute_modules` 覆盖更全的 selective。

(3) 流水级在流水线中的位置。在 1F1B 等调度下, 靠近输入端的流水级在稳定阶段往往需要同时保存更多 micro-batch 的中间激活值, 其峰值显存通常高于靠近输出端的流水级 (也取决于流水线深度、micro-batch 数目及是否采用交错调度等)。基于此观察, 搜索时可令前半段流水级从较保守的 θ 出发, 在显存约束下沿候选重计算方案逐步提高重计算强度; 后半段流水级在显存允许时从较激进的 θ 出发, 再沿候选重计算方案回退以降低不必要的重计算。该位置启发与 (1) (2) 相互独立, 并与冻结配置一并体现在算法 4.1 中。

多路分支搜索与全序候选集。 据此构造长度有限的候选全序 $\Theta = (\theta^1, \dots, \theta^6)$: 在 Megatron-LM 语义下自保守端 (激活常驻规模较大、重计算较少) 至激进端 (激活常驻规模较小、重计算较多) 排列。下文 (1) – (6) 的编号仅用于说明各候选配置的含义; 工程实现中应依据剖析得到的 $\text{Mem}_s^{\text{act}}(\theta)$ 对 Θ 重新排序, 使在代表性流水级上满足 $\text{Mem}_s^{\text{act}}(\theta^i) \geq \text{Mem}_s^{\text{act}}(\theta^{i+1})$ 。各候选项含义如下: (1) None: 不启用重计算; (2) Selective: 采用 selective 粒度, 仅对注意力子模块 (`core_attn`) 启用重计算; (3) Selective-LayerNorm: 仅对 LayerNorm 子模块启用重计算; (4) Selective-Attn+MLP: 对注意力与 MLP 子模块启用重计算; (5)

Selective-LayerNorm+Attn+MLP：对注意力、LayerNorm 与 MLP 子模块启用重计算；（6）Full：采用 full 粒度，对整层进行重计算。

对非冻结流水级，以 $s = \lfloor S/2 \rfloor$ 为界划分为靠近输入的前半段与靠近输出的后半段：前半段按 s 升序遍历，自 Θ 的较小下标（较保守配置）起搜索可行解；后半段按 s 降序遍历，自较大下标（较激进配置）起搜索，并在显存约束允许时沿下标递减方向回调，以降低冗余重计算。冻结流水级固定为 θ^1 (None)。运行过程中若仍发生 OOM，则将该流水级在 Θ 中的下标向增大方向推进，直至满足式 (4.9) 或候选耗尽。算法 4.1 给出离线搜索主流程；其中 TotalMem 与式 (4.9) 约束左端一致，冻结流水级的梯度与优化器状态按第三章语义置零或依据剖析表取值。运行时 OOM 处理策略与上文一致：对发生显存溢出的流水级沿 Θ 增大下标直至获得可行配置或判定失败。

实现与配置使用。 MMoH 给出模型划分和设备映射后，HR-MMoH 的重计算规划器为每个流水级生成一个重计算配置，所有流水级的重计算配置组织为一个 JSON 文件，由流水线并行组中各 rank 在进程初始化时读入。同一流水级内多卡（如张量并行）须共享同一 θ_s ，以保证检查点边界与张量并行通信组一致；不同流水级之间则可取不同 θ_s ，从而实现流水级维度的异构重计算。除重计算范围按 θ_s 区分外，单卡计算图与梯度语义与标准 Megatron-LM 一致，从而在更大批量或更长序列下仍满足式 (4.9)。

与 MMoH 的协同。 HR-MMoH 与 MMoH 在模型划分、设备映射及提前终止调度上的关系可概括为：先由 MMoH 确定划分与映射并在既定调度下运行；再在此基础上配置各流水级重计算参数 θ_s ；当显存仍不可行时将约束违反信息反馈至 MMoH 以调整划分或映射，并迭代上述过程。其中，划分与映射决定参数与计算负载分布，调度策略决定各流水级前向一反向执行路径及相关通信；HR-MMoH 在不改变单卡数学语义的前提下调节 θ_s ，以提高式 (4.9) 的可行性。若已采用最激进候选 θ^R 仍发生 OOM，表明在当前划分与映射下仅依靠重计算仍无法满足训练条件，应将显存溢出信息反馈至 MMoH 规划器（例如调整该流水级在优化目标中的显存相关项、减少该流水级所含模型块数量等），并在新映射上重新剖析后执行算法 4.1。映射更新后，亦可相应将部分 θ_s 回调至较保守配置，以避免长期承担过量的重计算开销。提前终止调度在冻结流水级上省略部分反向计算与阶段间通信，与按流水级独立选取的 θ_s 相容：冻结流水级通常维持较轻的 θ_s ，非冻结流水级仍依据式 (4.9) 与 $\max_s T_s$ 相关启发式独立配置。

算法 4.1 HR-MMoH 多路分支重计算配置搜索

已知: 流水线并行度 S ; 冻结标记 $\text{frozen}[s]$; 可用显存上界 $\{Q_s\}$

已知: 全序候选 $\Theta = (\theta^1, \dots, \theta^R)$ ($R=6$, $\text{Mem}_s^{\text{act}}(\theta^i)$ 随 i 增大非增) ; 剖析表 $\text{Mem}_s^{\text{param}}, \text{Mem}_s^{\text{grad}}, \text{Mem}_s^{\text{opt}}, \text{Mem}_s^{\text{act}}(\theta^i)$

求: 各流水级 θ_s

```

1:  $\text{TotalMem}(s, \theta^i) \leftarrow \text{Mem}_s^{\text{param}} + \text{Mem}_s^{\text{grad}} + \text{Mem}_s^{\text{opt}} + \text{Mem}_s^{\text{act}}(\theta^i)$ 
2:  $\text{mid} \leftarrow \lfloor S/2 \rfloor$ ;  $\forall s$ , 若  $\text{frozen}[s]$  则  $\theta_s \leftarrow \theta^1$ 
3:  $\mathcal{U} \leftarrow \{s \mid \text{frozen}[s] = \text{false}\}$ ;  $\mathcal{U}_{\text{front}} \leftarrow \{s \in \mathcal{U} \mid s \leq \text{mid}\}$ ;  $\mathcal{U}_{\text{back}} \leftarrow \mathcal{U} \setminus \mathcal{U}_{\text{front}}$ 
4:  $W_{\text{front}} \leftarrow \text{sort}_{\text{asc}}(\mathcal{U}_{\text{front}})$ ;  $W_{\text{back}} \leftarrow \text{sort}_{\text{desc}}(\mathcal{U}_{\text{back}})$ 
5: for all  $s \in W_{\text{front}}$  do
6:    $j \leftarrow 1$ 
7:   while  $j \leq R$  and  $\text{TotalMem}(s, \theta^j) > Q_s$  do
8:      $j \leftarrow j + 1$ 
9:   end while
10:  if  $j > R$  then
11:    trigger MMoH Planner
12:  else
13:     $\theta_s \leftarrow \theta^j$ 
14:  end if
15: end for
16: for all  $s \in W_{\text{back}}$  do
17:    $j \leftarrow R$ 
18:   if  $\text{TotalMem}(s, \theta^R) > Q_s$  then
19:     trigger MMoH Planner
20:   else
21:     while  $j > 1$  and  $\text{TotalMem}(s, \theta^{j-1}) \leq Q_s$  do
22:        $j \leftarrow j - 1$ 
23:     end while
24:      $\theta_s \leftarrow \theta^j$ 
25:   end if
26: end for

```

4.3 实验验证

4.3.1 实验设置

平台与配置思路验证。HR-MMoH 的实验环境与第三章 MMoH 实验一致, 采用更具有代表性的异构集群 (Cluster-L) 及两个模型中更大的 MLLM (MLLM-12B)。为了验证 HR-MMoH 对于大 batch 加长序列场景下的能力, 本文使用更大的 batchsize, 编码器和 LLM 的序列长度也进行扩大, 都变成了第三章中实验配置的 2 倍。具体来说, 本文把 batchsize 从 1 增加为 2, 编码器的序列长度从 576 变为 1172, LLM

的序列长度从 1024 变为 2048。为了验证“按流水级差异化配置”的合理性，对 MLLM-12B 的不同模型层按照 4.2.1 章节进行中间激活的显存定量剖析，在每流水级对应的设备上，分别计算 `recompute_granularity=None`、`selective` 且 `recompute_modules=[core_attn]`、`[core_attn, mlp]`、`[core_attn, mlp, layernorm]` 以及 `full + uniform` 且不同 `recompute_num_layers` 下的激活峰值与单流水级前向 + 反向时间（含重计算）。据此可得到各流水级在满足显存约束下的可选配置集合及对应的 T_s ，用于验证显存紧张流水级与充裕流水级在“最省显存”与“最少重算”配置上的差异是否显著，以及按流水级赋配置后各 T_s 是否更均衡。

对比与指标。 异构重计算的对比基线为全局统一配置：所有流水级不采用重计算，记为 `no-recomp`；所有流水级采用相同的 `full recomputation`，即在前向传播中丢弃所有的中间激活值，记为 `full`；所有流水级采用相同的默认 `selective`，即只重计算 `attention` 模块。模型训练的冻结情况与第三章一致，使用全部的 4 种冻结情况：`no-freeze`，`freeze-encoder`，`freeze-llm`，`freeze-both`。分别使用 Megatron, Whale, Espresso, MMoH 作为模型划分策略，对于所有策略使用之前提到的全局统一配置。对于 MMoH，我们还使用异构的重计算配置 HR-MMoH。对比指标包括：完整训练的迭代时间 `seconds/iteration`。

4.3.2 实验结果

迭代时间对比。 本节首先对比 Megatron、Whale、Espresso、MMoH 在不同冻结策略和不同常规重计算配置下的速度，如图 4.4 所示。

图中按冻结策略拆分为四个子图，自上而下、自左至右依次为 (a) `no-freeze`、(b) `freeze-encoder`、(c) `freeze-llm` 与 (d) `freeze-both`：每个子图在同一冻结条件下对比各框架在全局统一的 `no-recomp`、`full` 与默认 `selective` 等重计算配置下的 `iteration time (seconds/iteration)`。由于数据的 `batchsize` 更大，`encoder` 和 `llm` 的 `sequence length` 更长，模型训练时需要的显存开销剧增。Megatron 在所有冻结策略下都出现了 OOM，即使使用了全部重计算并且使用最大的 TP size，仍然无法训练，因为其同构假设导致负载不均；Whale 和 Espresso 通过异构感知的拓扑与划分避免了部分情况下的 OOM，所以两者表现略好与 Megatron，可以适应更多的冻结情况，但是 Espresso 的负载均衡总体上还是优于 Whale，在 `no-freeze` 的情况下，Espresso 在开启全部重计算后就可以正常训练，但是 Whale 在 `no-freeze` 的情况下无法正常训练，即使使用最大的 TP size 和最激进的全部重计算。总体上 Espresso 在更多场景下表现更好。MMoH 的负载均衡显著优于其它方法，在不开

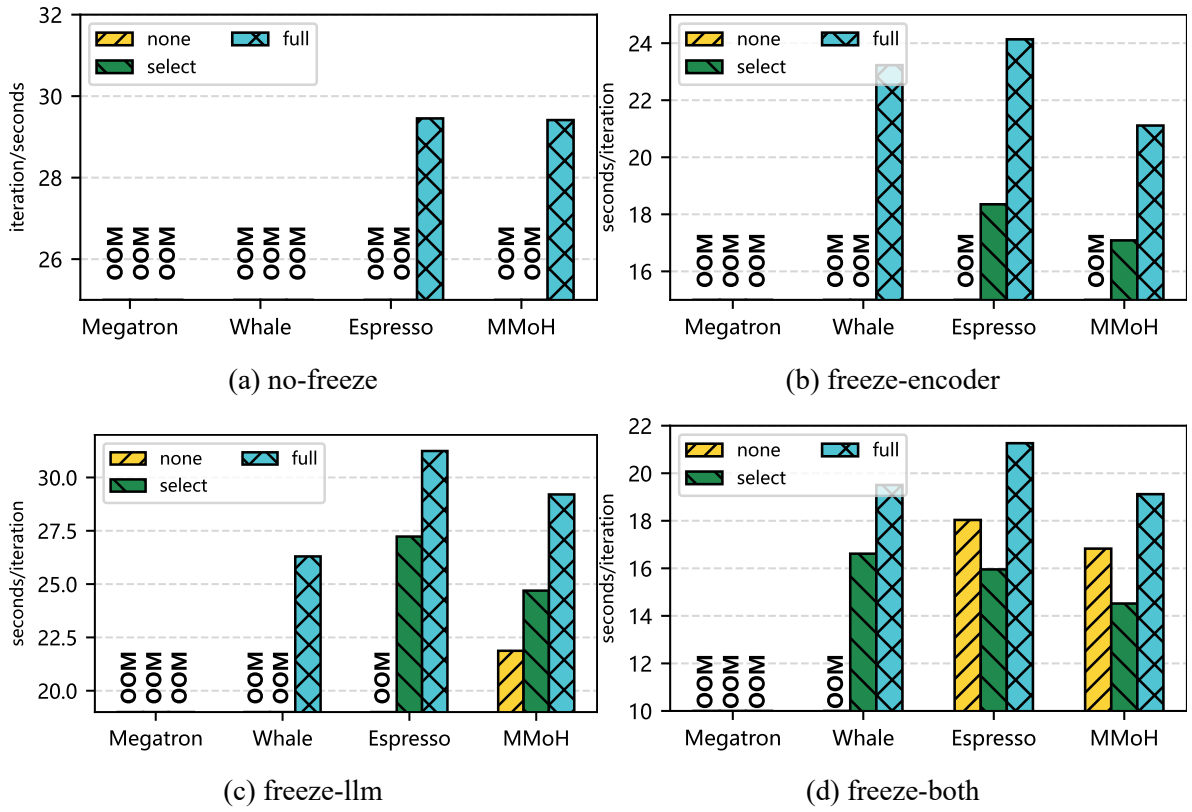


图 4.4 不同冻结策略下各框架 iteration time (seconds/iteration) 对比（自左至右、自上至下依次为 no-freeze、freeze-encoder、freeze-llm、freeze-both）。

启任何重计算且冻结 llm 的情况下，MMoH 可以正常训练，而其余三者要么彻底无法训练，要么需要使用重计算策略。在所有的冻结策略以及默认的重计算方式下，MMoH 的迭代时间都低于其余三者，这印证了本文在第三章中提出的方法的有效性。

Whale 与 Espresso 在可运行配置内随重计算档位变化呈现明显的迭代时间波动——no-recomp 往往受显存约束更敏感，full 与 selective 则以额外重算换取可行性与迭代时间下降。随着子图由 (a) 过渡到 (d)，冻结范围扩大使得反传与跨阶段通信规模下降，各框架的整体 iteration time 水平与档位间差距随之重塑，但异构划分质量不同仍会造成 Whale 与 Espresso 在部分子图、部分档位上互有领先。MMoH 在四个子图的各重计算档位上均保持领先或接近最优，说明在放大 batch 与序列的显存紧张设定下，复合粒度划分与冻结感知调度仍能稳定提供更低的 iteration time。

在前一小节对全局统一重计算配置的基础上，本节进一步在各框架内部对 no-recomp（图中记为 none）、full、默认 selective 以及 HR-MMoH 的按流水级异构重计算配置分别搜索可行解，并在每种冻结策略下取各框架所能达到的最短迭代时间（seconds/iteration）进行并列对比，如图 4.5 所示。这样设置

的意义在于：统一档位对比揭示“全局一刀切”在异构流水线上的局限，而“每框架各自最优”对比则更公平地衡量划分与重计算协同优化后仍能否进一步压缩迭代时间。对 HR-MMoH 而言，最优配置由 4.2 节所述剖析驱动的候选枚举与约束判定给出，在 MMoH 已提供的模型划分与设备映射上为各流水级独立选择 `recompute_granularity`、`recompute_modules` 等参数，使显存压力大的流水级必要时更激进地释放激活、显存压力小的流水级尽量少做多余重算，从而在满足各卡显存上限的前提下拉低瓶颈流水级的耗时。

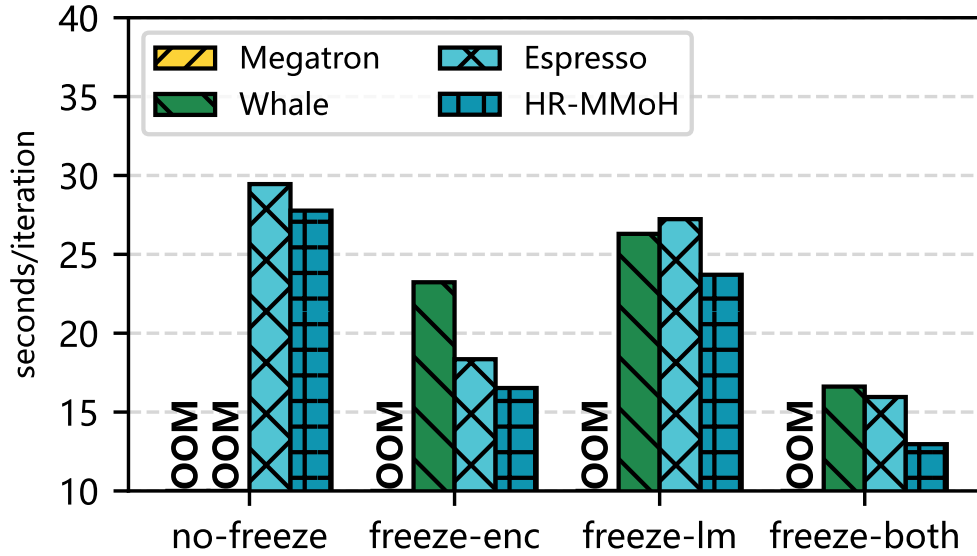


图 4.5 不同冻结策略下各框架在 none、full、selective 及 HR-MMoH 配置族上的最短 iteration time (seconds/iteration) 对比。

从图4.5可以读出：在四种冻结条件下，启用异构重计算的 HR-MMoH 均取得**最短 iteration time**，说明在放大 batch 与序列、显存余量收紧的设置下，按流水级差异化的重计算仍能系统性优于各框架在全球 none/full/selective 下所能达到的最优效果。

具体到数值，HR-MMoH 在 no-freeze、freeze-encoder、freeze-llm、freeze-both 下的迭代时间分别为 27.7714、16.5245、23.7048 与 12.9649 seconds/iteration。Espresso 同样覆盖上述四种冻结，其各自最短迭代时间为 29.4536、18.3514、27.2301 与 15.9547 seconds/iteration；与之相比，HR-MMoH 的迭代时间分别缩短 6.1%、11.0%、14.9% 与 23.1%。可见相对这一强异构划分基线，优势随冻结范围扩大而**更为显著**：freeze-both 场景下反传与跨阶段通信进一步收缩，流水级间算力与显存需求的差异仍存，统一重算更容易在非瓶颈级产生“过度重算”，而 HR-MMoH 通过按级收紧或放松检查点，更充分地利用了节余下来的计算与通信窗口。

Whale 在本设定下仅能在 freeze-encoder、freeze-llm 与 freeze-both 三种冻结策

略上完成训练（no-freeze 仍不可行），其对应最短迭代时间为 23.2292、26.2986 与 16.6158 seconds/iteration；HR-MMoH 相对 Whale 分别缩短 40.6%、11.0% 与 28.2%。其中 freeze-encoder 一列差距最大，与 Whale 在该场景下仍需依赖较重全局重算、流水级负载与显存峰值更难同时压平有关；HR-MMoH 则在剖析结果指导下把“重算压力”主要留给显存真正吃紧的阶段，从而更明显降低端到端迭代时间。

Megatron 因同构并行假设下的负载与显存分配失衡，在本实验的四种冻结策略与所试重计算、TP 配置下均触发 OOM，无法给出有效迭代时间，故不参与上图中的迭代时间对比；这也与前文大图子图中 Megatron 持续缺失的现象相互印证。

综合而言，在每种冻结策略下将 HR-MMoH 与当前可运行的异构划分方法中较优者（Espresso 和 Whale）相比，端到端加速比落在约 $1.06\times-1.41\times$ 区间：相对 Espresso 的提升相对较小（ $1.06x-1.23x$ ），相对 Whale 在可训练子集上则出现更大幅度的迭代时间下降（ $1.11x-1.41x$ ）。这表明 HR-MMoH 并非单一“更重的全局重算”，而是在 MMoH 划分之上对显存维度的精细化续优化，可与第三章的划分与调度形成互补。

消融实验以及配置策略。本节对比 MMoH 与 HR-MMoH 在不同冻结策略下的 iteration time 及重计算配置差异，以验证在同一划分与调度之上引入按流水级异构重计算能否进一步带来端到端收益；结果如图4.6所示。

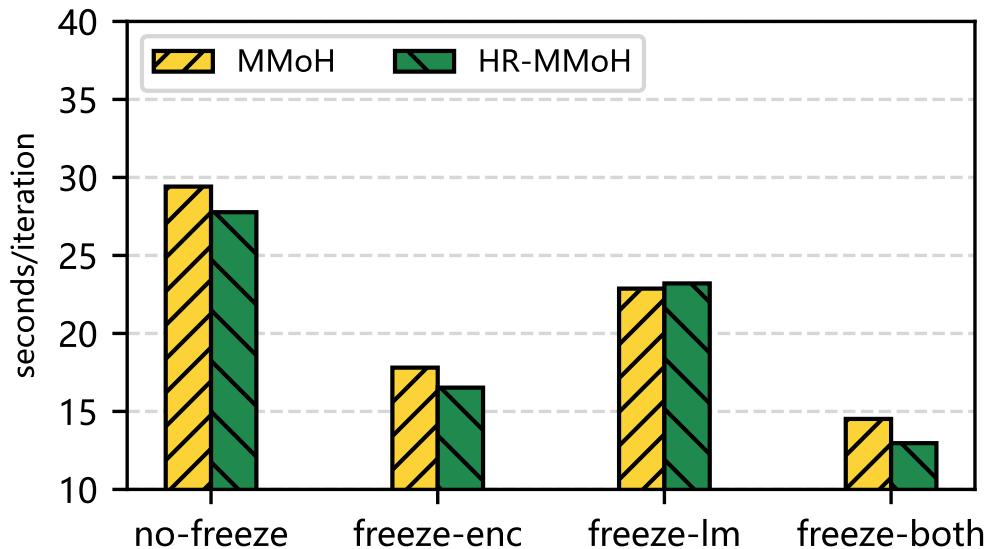


图 4.6 不同冻结策略下 MMoH 与 HR-MMoH 的 iteration time 及配置对比（消融）。

实验显示，在所有四种冻结配置下的其中三种（no-freeze, freeze-encoder, freeze-both）下，启用异构重计算的 HR-MMoH 均取得了较大的收益，说明在放大 batch 与序列、显存余量收紧的设置下，按流水级差异化重计算仍能系统性

优于 MMoH 在全局 none/full/selective 配置。具体来说，HR-MMoH 在 no-freeze、freeze-encoder、freeze-both 下的迭代时间加速比分别为 1.06x, 1.04x, 1.12x。值得一提的是，在 freeze-llm 的场景下，HR-MMoH 的最优配置和 MMoH 的配置相同，即都不使用任何重计算策略，这导致两者之间并无明显的速度差距，这是因为在 freeze-llm 的场景下，显存开销较小，不需要使用重计算策略也可以在 TP 设置为 4 时正常训练。但是在 freeze-both 的这种显存开销更小的场景下，HR-MMoH 的迭代时间加速比达到了 1.12x，这表明在某些显存开销较小的场景下，按设备差异化重计算仍能发挥有效性。具体来看，MMoH 在使用默认的全局 selective 重计算策略时，迭代时间为最短，相比全局 none 重计算策略，TP 可以从 4 减少为 2，大幅减少了通信开销，而 HR-MMoH 在使用了按设备差异化重计算策略后，识别出了在冻结情况下不需要重计算的流水级，有了进一步的加速。

4.3.3 实验分析

异构重计算的优势。 异构集群上各流水级显存与算力分布不均，统一配置要么为防 OOM 而整体偏保守导致重算过多，要么为减重算而整体偏激进导致瓶颈流水级 OOM。按流水级配置后，可在满足各流水级显存约束的前提下，将“重算预算”更多分配给显存紧张、算力相对充裕的流水级，从而在系统层面降低迭代时间。

与 MMoH 划分的配合。 MMoH 的复合粒度与冻结感知划分已使各流水级负载趋于均衡；HR-MMoH 在此基础上对显存维度做细粒度调节，二者结合可形成“划分—调度—重计算优化”的闭环。未来工作可将 HR-MMoH 的配置搜索与 MMoH 的整数规划联合建模，在考虑模型划分和设备映射的时候就考虑重计算给迭代时间和显存开销带来的影响。

4.4 本章小结

本章在 MMoH 的异构流水线划分与提前终止调度基础上，针对显存受限流水级仍可能 OOM、而现有重计算策略多采用全局统一配置难以兼顾显存与效率的问题，介绍了 HR-MMoH 的异构重计算思路。首先系统梳理了 Megatron-LM 的重计算配置粒度与层级划分方式，并指出其同构假设下的局限；随后给出了 HR-MMoH 的设计要点：以各流水级显存上限、算力为约束，为各流水级独立配置重计算粒度与子模块集合，通过基于剖析的配置策略在满足显存不溢出的前提下最小化迭代时间并平衡各流水级负载，并与 MMoH 划分及提前终止调度器协同。实验结果表明，在 no-freeze、freeze-encoder、freeze-llm、freeze-both 四种冻结场景下，HR-MMoH 均取得最短迭代时间，相比现有方法最高达到 1.41x 的加速

比。HR-MMoH 与第三章的 MMoH 共同构成“划分—调度—重计算优化”的联合优化框架，为异构 GPU 集群上 MLLM 的高效训练提供了又一个有力手段。

第五章 总结与展望

本章对全文工作进行总结，概括主要创新点与研究的局限性，并对未来研究方向进行展望。

5.1 全文工作总结

本文围绕“面向异构 GPU 集群的多模态大语言模型并行训练策略”这一主题，针对实际环境中普遍存在的设备异构（显存、算力、通信）与 MLLM 的模块异构、分阶段冻结等特性，从流水线划分与调度与重计算优化两个层面开展了研究，提出了 MMoH 与 HR-MMoH 两套方法，并在异构集群上进行了实验验证。下文从主要创新点与研究的局限性两方面进行总结。

5.1.1 主要创新点

本文的主要创新点可归纳为以下两方面。

(1) MMoH：面向异构 GPU 集群的 MLLM 高效混合并行框架。

针对异构集群上 MLLM 训练时存在的负载不均衡、资源利用率低以及冻结场景下冗余计算与通信等问题，本文提出了 MMoH 框架。其创新点包括：需求驱动的设备拓扑识别，根据 MLLM 各模块的计算与显存需求趋势与异构设备的算力—显存供给进行匹配，筛选候选设备拓扑，使需求与供给在趋势上一致；复合粒度模型抽象，针对编码器、投影层与 LLM 基座的结构差异，采用多级粒度将模型重组为模型块列表（编码器按子模块合并、LLM 按注意力/FFN 子层拆分），在划分复杂度与负载均衡之间取得折中；冻结感知的模型划分，将块到阶段的分配形式化为整数规划问题，在满足显存与主阶段约束下最小化迭代时间，且目标函数显式依赖各阶段的冻结状态，从而随训练阶段动态调整划分策略；提前终止调度器，在 1F1B 等流水线调度基础上，识别从首端开始的连续冻结阶段并对其省略反向计算与阶段间通信，在保持梯度传播数学等价性的前提下提升吞吐。在两种异构集群（Cluster-S、Cluster-L）与两种规模 MLLM（MLLM-3B、MLLM-12B）上的实验表明，MMoH 在 no-freeze、freeze-encoder、freeze-llm、freeze-both 四种场景下相对 Megatron-LM、Whale、Espresso 均可取得最高吞吐，最高可达约 $1.77\times$ 加速，且提前终止调度器在冻结场景下可带来约 8%–19% 的额外增益，并具有良好的可移植性。

(2) HR-MMoH：面向异构设备与异构模型的重计算策略优化。

针对现有重计算策略在同构假设下采用全局统一配置、难以在异构集群上同时兼顾显存安全与计算效率的问题，本文在 MMoH 的划分与调度基础上，提出并实

现了 HR-MMoH 的异构重计算方法：使各流水级根据自身显存与算力条件采用差异化的重计算粒度（如 full/selective）与子模块选择（如 `recompute_modules`），在降低 OOM 风险的同时减少多余重算、平衡阶段负载，与 MMoH 形成“划分—调度—重计算优化”的联合优化。第四章在放大 batch 与序列长度的显存紧张设定下表明，HR-MMoH 在 no-freeze、freeze-encoder、freeze-llm、freeze-both 四种冻结场景中均取得最高吞吐；相对 Espresso 的迭代时间缩短约 6.1%–23.1%，在 Whale 可训练的三种冻结场景中缩短约 11.0%–40.6%。这说明按流水级差异化配置能够系统性优于全局统一的 none/full/selective 方案，并验证了与 MMoH 协同优化的有效性。

综上，本文在异构 GPU 集群上 MLLM 并行训练这一方向上，从划分与调度与显存优化两条线给出了系统性的方法设计与实验支撑，为实际部署中的混合集群与分阶段训练场景提供了可参考的技术路径。

5.1.2 研究的局限性

本研究仍存在以下局限性，值得在后续工作中改进。

(1) MMoH 方面。规划器中的整数规划求解在模型层数与设备数较大时可能面临组合爆炸，当前采用启发式或商用求解器在有限时间内求近似解，最优性保证与可扩展性仍有提升空间；实验仅在两种规模的 MLLM（3B、12B）与两种异构集群配置上进行，对更大规模模型（如 70B+）、更多代际与型号混合的集群以及更长序列、更多模态的 MLLM 的泛化能力有待进一步验证。

(2) HR-MMoH 方面。尽管第四章已完成端到端实验并验证了按流水级异构重计算的收益，当前策略搜索仍以启发式与剖析驱动为主，候选配置空间、搜索顺序与回退机制依赖经验设计，尚未给出更强的最优性保证与复杂度分析；实验主要在 Cluster-L 与 MLLM-12B 上开展，对更大规模模型、更复杂模型结构（如 MoE 模型）及更长上下文设定下的鲁棒性仍需补充；此外，HR-MMoH 与 MMoH 的联合优化目前仍采用“先划分、后配置重计算”的分层流程，尚未形成统一目标下的联合求解框架。

(3) 通用性方面。当前工作主要针对视觉—语言类 MLLM 与 NVIDIA GPU 生态，对音频、视频等多模态输入、以及 AMD、国产 GPU 等异构硬件与软件栈的适配与评估尚未涉及；与专家并行、上下文并行等已有技术的协同优化仍有扩展空间。

5.2 未来工作展望

结合上述总结与局限性，从理论深化与应用拓展两方面对后续研究进行展望。

5.2.1 理论深化

在理论层面，可从以下方向深化。

(1) **划分与调度的形式化分析与优化。**对 MLLM 在异构流水线上的“需求—供给”匹配建立更形式化的度量（如基于负载方差、瓶颈阶段占比等），并研究其与迭代时间、吞吐之间的定量关系；对冻结场景下提前终止的通信与计算节省给出上界分析；探索在保证近似比或 regret 意义下的在线 / 自适应划分与调度算法，以应对训练过程中阶段切换与资源波动。

(2) **重计算的自动配置与联合优化。**在 HR-MMoH 已验证有效的基础上，进一步发展更系统的自动配置方法：以各阶段显存上限、算力与通信负载为输入，在满足显存约束下自动选择重计算粒度、子模块集合与分块参数，并显式最小化重算引入的额外时间与流水线气泡；进一步将重计算与 MMoH 的划分、提前终止调度联合建模为统一优化问题，探索分层联合求解或迭代反馈式联合优化，以进一步提升端到端吞吐与稳定性。

(3) **与 ZeRO 等的联合优化。**研究在异构集群上 ZeRO 分片与 MMoH 流水线划分的协同：例如不同阶段采用不同 ZeRO 阶段或不同分片策略，以匹配各设备显存与通信能力；将重计算与序列并行、专家并行等纳入同一决策空间，并给出可扩展的求解或搜索方法。

5.2.2 应用拓展

在应用层面，可从以下方向拓展。

(1) **更大规模模型与更多模态。**在 70B 及以上参数规模的 MLLM、更长序列（如 32K+ token）以及视频、音频等多模态输入上验证 MMoH 与 HR-MMoH 的可扩展性；针对 MoE 结构、多阶段生成（如扩散模型与 LLM 联合训练）等复杂计算图，扩展复合粒度抽象与冻结感知划分的建模方式，并设计相应的调度与显存优化策略。

(2) **多硬件与多框架支持。**将设备能力建模与拓扑识别扩展到 AMD GPU、国产加速卡以及 CPU—GPU 混合节点，抽象统一的“算力—显存—带宽”供给描述，并在多后端（如 PyTorch、Megatron、DeepSpeed 等）上实现或对接 MMoH 的划分与调度逻辑，形成可复用的异构 MLLM 训练工具链。

(3) **从训练到推理与部署。**将异构感知的划分与调度思想延伸至大模型与 MLLM 的推理与部署阶段：例如在异构边缘—云集群上对模型进行分层放置与动态调度，在满足延迟与吞吐要求的前提下降低资源占用与成本；探索训练阶段得到的划分与设备映射对后续推理拓扑设计的借鉴意义，形成训练—推理一致的高效异构部署方案。

综上所述，本文在面向异构 GPU 集群的 MLLM 并行训练策略方面取得了阶段性成果，但仍有许多理论问题与应用场景有待深入。期望后续工作能够在形式化分析、重计算的完整实现与联合优化、以及更大规模与多硬件生态的验证上继续推进，为多模态大模型在异构环境下的高效训练与部署提供更完善的技术支撑。

致 谢

攻读硕士学位期间，本论文的完成离不开各位老师、同学与家人的关心与支持，在此谨致谢忱。

参考文献

- [1] Miao X, Wang Y, Jiang Y, et al. Galvatron: Efficient Transformer Training over Multiple GPUs Using Automatic Parallelism [J]. Proceedings of the VLDB Endowment. 2022, 16 (3): 470–479.
- [2] Vaswani A, Shazeer N, Parmar N, et al. Attention is All You Need [C]. Advances in Neural Information Processing Systems. 2017.
- [3] Devlin J, Chang M-W, Lee K, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [C]. Proceedings of NAACL-HLT. 2019: 4171–4186.
- [4] Radford A, Wu J, Child R, et al. Language Models are Unsupervised Multitask Learners [J]. OpenAI blog. 2019, 1 (8): 9.
- [5] Raffel C, Shazeer N, Roberts A, et al. Exploring the Limits of Transfer Learning with a Unified Text-to-text Transformer [J]. Journal of Machine Learning Research. 2020, 21 (1): 5485–5551.
- [6] Brown T, Mann B, Ryder N, et al. Language Models are Few-shot Learners [J]. Advances in Neural Information Processing Systems. 2020, 33: 1877–1901.
- [7] OpenAI. GPT-4 Technical Report [J]. arXiv.org. 2023. 2023-03-15.
- [8] Dosovitskiy A, Beyer L, Kolesnikov A, et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale [C]. International Conference on Learning Representations. 2020.
- [9] Zhai X, Kolesnikov A, Houlsby N, et al. Scaling Vision Transformers [J]. 2022: 12104–12113.
- [10] Li J, Li D, Savarese S, et al. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models [J]. International Conference on Machine Learning. 2023: 19730–19742.
- [11] Liu H, Li C, Wu Q, et al. Visual Instruction Tuning [J]. Advances in Neural Information Processing Systems. 2023, 36.
- [12] Alayrac J-B, Donahue J, Luc P, et al. Flamingo: a Visual Language Model for Few-shot Learning [J]. Advances in Neural Information Processing Systems. 2022, 35: 23716–23736.
- [13] Radford A, Kim J W, Hallacy C, et al. Learning Transferable Visual Models from Natural Language Supervision [C]. International Conference on Machine Learning.

2021: 8748–8763.

[14] Ramesh A, Pavlov M, Goh G, et al. Zero-shot Text-to-image Generation [C]. International Conference on Machine Learning. 2021: 8821–8831.

[15] Dai W, Li J, Li D, et al. InstructBLIP: Towards General-purpose Vision-Language Models with Instruction Tuning [C]. Advances in Neural Information Processing Systems (NeurIPS). 2023: 49250–49267.

[16] Liu H, Li C, Wu Q, et al. Improved Baselines with Visual Instruction Tuning [J]. arXiv preprint arXiv:2310.03744. 2023. LLaVA-1.5.

[17] Bai J, Bai S, Chu Y, et al. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond [J]. arXiv preprint arXiv:2308.12966. 2023.

[18] Chen Z, Wu J, Wang W, et al. InternVL: Scaling up Vision Foundation Models and Aligning for Generic Visual-Language Tasks [C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2024: 13142–13153.

[19] Zhu D, Chen J, Shen X, et al. MiniGPT-4: Enhancing Vision-language Understanding with Advanced Large Language Models [J]. arXiv preprint arXiv:2304.10592. 2023.

[20] Liang Y, et al. The Revolution of Multimodal Large Language Models: A Survey [C]. Findings of the Association for Computational Linguistics: ACL 2024. 2024.

[21] Wang Z, et al. Embodied Multimodal Agents: A Survey [J]. arXiv preprint. 2024.

[22] Wu C, et al. Medical Vision-Language Models: A Survey [J]. arXiv preprint arXiv:2312.06203. 2024.

[23] Goyal Y, Khanna G, Gupta A, et al. Making the V in VQA Matter: Elevating the Role of Image Understanding in Visual Question Answering [C]. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2017: 6904–6913.

[24] Hudson D A, Manning C D. GQA: A New Dataset for Real-World Visual Reasoning and Compositional Question Answering [C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2019: 6700–6709.

[25] Yue X, Ni Y, Zhang K, et al. MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI [J]. arXiv preprint arXiv:2311.14402. 2024.

[26] Kaplan J, McCandlish S, Henighan T, et al. Scaling Laws for Neural Language Models [J]. arXiv preprint arXiv:2001.08361. 2020.

- [27] Hoffmann J, Borgeaud S, Mensch A, et al. Training Compute-Optimal Large Language Models [C]. Advances in Neural Information Processing Systems (NeurIPS). 2022: 30016–30030.
- [28] Sardana N, Frankle J. Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws [C]. Proceedings of the International Conference on Machine Learning (ICML). 2024: 29891–29922.
- [29] Rae J W, et al. Scaling Language Models: Methods, Analysis and Insights from Training Gopher [J]. arXiv preprint arXiv:2112.11446. 2022.
- [30] DeepSeek-AI. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism [J]. arXiv preprint arXiv:2401.02954. 2024.
- [31] Yang A, Yang B, Hui B, et al. Qwen2 Technical Report [J]. arXiv preprint arXiv:2407.10671. 2024.
- [32] Touvron H, Martin L, Stone K, et al. Llama 2: Open Foundation and Fine-tuned Chat Models [J]. arXiv preprint arXiv:2307.09288. 2023.
- [33] Dubey A, Jauhri A, Pandey A, et al. The LLaMA 3 Herd of Models [J]. arXiv preprint arXiv:2407.21783. 2024.
- [34] Team G, et al. Gemma: Open Models Based on Gemini Research and Technology [J]. arXiv preprint arXiv:2403.08295. 2024.
- [35] Jiang A Q, Sablayrolles A, Mensch A, et al. Mistral 7B [J]. arXiv preprint arXiv:2310.06825. 2023.
- [36] Narayanan D, Shoeybi M, Casper J, et al. Efficient Large-scale Language Model Training on GPU Clusters Using Megatron-LM [C]. Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2021: 1–15.
- [37] Shoeybi M, Patwary M, Puri R, et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism [J]. arXiv preprint. 2020.
- [38] Xie S, et al. Data Selection for Language Models via Importance Resampling [J]. Advances in Neural Information Processing Systems (NeurIPS). 2023, 36.
- [39] Bai Y, et al. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding [J]. arXiv preprint arXiv:2308.14508. 2024.
- [40] Rajbhandari S, Rasley J, Ruwase O, et al. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models [C]. SC20: International Conference for High Performance Computing, Networking, Storage and Analysis. 2020: 1–16.

- [41] Rajbhandari S, Ruwase O, Rasley J, et al. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning [J]. arXiv preprint. 2021.
- [42] Li S, Zhao Y, Varma R, et al. PyTorch Distributed: Experiences on Accelerating Data Parallel Training [J]. Proceedings of the VLDB Endowment. 2020, 13 (12): 3005–3018.
- [43] Sergeev A, Del Balso M. Horovod: Fast and Easy Distributed Deep Learning in TensorFlow [J]. arXiv preprint. 2018.
- [44] Huang Y, Cheng Y, Bapna A, et al. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism [J]. arXiv preprint. 2019.
- [45] Narayanan D, Harlap A, Phanishayee A, et al. PipeDream: Generalized Pipeline Parallelism for DNN Training [C]. Proceedings of the 27th ACM Symposium on Operating Systems Principles. 2019: 1–15.
- [46] Chen Z, Lepikhin D D, Lee H, et al. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding [J]. 2020.
- [47] Wang G, Qin H, Jacobs S A, et al. ZeRO++: Extremely Efficient Collective Communication for Giant Model Training [J]. arXiv preprint. 2023.
- [48] Micikevicius P, Narang S, Alben J, et al. Mixed Precision Training [C]. International Conference on Learning Representations. 2018.
- [49] Liu Y, Li S, Fang J, et al. Colossal-Auto: Unified Automation of Parallelization and Activation Checkpoint for Large-scale Models [J]. arXiv preprint. 2023.
- [50] Dao T, Fu D Y, Ermon S, et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness [C]. Advances in Neural Information Processing Systems (NeurIPS). 2022: 16344–16359.
- [51] Dao T, Fu D Y, Ermon S, et al. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision [C]. Advances in Neural Information Processing Systems (NeurIPS). 2024.
- [52] Zhao Y, Gu A, Varma R, et al. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel [C]. Proceedings of the International Conference on Very Large Data Bases (VLDB). 2023: 3846–3856. PyTorch Native FSDP, 2022.
- [53] Jia X, Jiang L, Wang A, et al. Whale: Efficient Giant Model Training over Heterogeneous GPUs [J]. 2022.
- [54] Park J H, Yun G, Chang M Y, et al. HetPipe: Enabling Large DNN Training on (Whimpy) Heterogeneous GPU Clusters through Integration of Pipelined Model Parallelism and Data Parallelism [C]. 2020 USENIX Annual Technical Conference (USENIX

ATC 20). 2020: 307–321.

[55] NVIDIA Corporation. NVIDIA A100 Tensor Core GPU Architecture [R]. 2020. Ampere Architecture.

[56] NVIDIA Corporation. NVIDIA H100 Tensor Core GPU Architecture [R]. 2022. Hopper Architecture.

[57] NVIDIA Corporation. NVIDIA Blackwell Architecture and GPU Roadmap [R]. 2024. GTC 2024.

[58] Rombach R, Blattmann A, Lorenz D, et al. High-resolution Image Synthesis with Latent Diffusion Models [C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022: 10684–10695.

[59] Ronneberger O, Fischer P, Brox T. U-Net: Convolutional Networks for Biomedical Image Segmentation [C]. Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015. 2015: 234–241.

[60] Wu S, Fei H, Qu L, et al. Next-GPT: Any-to-any Multimodal LLM [J]. arXiv preprint arXiv:2309.05519. 2023.

[61] Dong R, Han C, Peng Y, et al. DreamLLM: Synergistic Multimodal Comprehension and Creation [J]. arXiv preprint arXiv:2309.11499. 2023.

[62] Kingma D P, Ba J. Adam: A Method for Stochastic Optimization [C]. International Conference on Learning Representations (ICLR). 2015. arXiv:1412.6980, 2014.

[63] Yu Z, et al. Frenzy: Memory-aware Serverless Scheduling for Large Model Training [J]. arXiv preprint. 2024.

[64] Li S, et al. Towards Scalable and Reliable LLM Training on Public Cloud [J]. arXiv preprint. 2024.

[65] NVIDIA Corporation. NVIDIA AI Training: Best Practices for Large Language Models [J]. 2024. GTC / Developer Blog.

[66] Hu E J, Shen Y, Wallis P, et al. LoRA: Low-Rank Adaptation of Large Language Models [C]. International Conference on Learning Representations (ICLR). 2022. arXiv:2106.09685, 2021.

[67] Dettmers T, Pagnoni A, Holtzman A, et al. QLoRA: Efficient Finetuning of Quantized LLMs [J]. Advances in Neural Information Processing Systems (NeurIPS). 2023, 36.

[68] Lialin V, Deshpande V, et al. Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning [J]. arXiv preprint arXiv:2303.15647. 2024.

[69] Schuhmann C, Beaumont R, Vencu R, et al. LAION-5B: An Open Large-

Scale Dataset for Training Next Generation Image-Text Models [C]. Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track. 2022. arXiv:2210.08402.

[70] Zhang Y, et al. A Comprehensive Survey and Guide to Multimodal Large Language Models in Vision-Language Tasks [C]. arXiv preprint arXiv:2411.06284. 2024.

[71] Zhou B, Khashabi D, et al. A Survey of Multimodal Large Language Model from A Data-centric Perspective [J]. arXiv preprint arXiv:2405.10739. 2024.

[72] Kuznetsova A, Rom H, Alldrin N, et al. The Open Images Dataset V4: Unified Image Classification, Object Detection, and Visual Relationship Detection at Scale [J]. International Journal of Computer Vision. 2020, 128 (7): 1956–1981.

[73] Lin T-Y, Maire M, Belongie S, et al. Microsoft COCO: Common Objects in Context [C]. European Conference on Computer Vision (ECCV). 2014: 740–755.

[74] Gao L, Biderman S, Black S, et al. The Pile: An 800GB Dataset of Diverse Text for Language Modeling [J]. arXiv preprint arXiv:2101.00027. 2020.

[75] Abu-El-Haija S, Kothari N, Lee J, et al. YouTube-8M: A Large-Scale Video Classification Benchmark [J]. 2016.

[76] Bain M, Nagrani A, Varol G, et al. Frozen in Time: A Joint Video and Image Encoder for End-to-End Retrieval [J]. arXiv preprint arXiv:2104.00650. 2021.

[77] Singh A, Natarajan V, Shah M, et al. Towards VQA Models That Can Read [C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2019: 8317–8326.

[78] Korthikanti V A, Casper J, Lym S, et al. Reducing Activation Recomputation in Large Transformer Models [C]. Proceedings of Machine Learning and Systems (MLSys). 2023: 1–20.

[79] Li D, Wang H, Xing E, et al. AMP: Automatically Finding Model Parallel Strategies with Heterogeneity Awareness [C]. Advances in Neural Information Processing Systems. 2022: 6630–6639.

[80] Xu S, Huang Z, Zeng Y, et al. HETHUB: A Distributed Training System with Heterogeneous Cluster for Large-Scale Models [J]. arXiv preprint arXiv:2405.16256. 2024.

[81] Yan R, Jiang Y, Nie X, et al. HexiScale: Accommodating Large Language Model Training over Heterogeneous Environment [J]. arXiv preprint arXiv:2409.01143. 2024.

[82] iCloud Lab, ECNU. Espresso: Cost-efficient Resource Provisioning for

Large Model Training on Heterogeneous GPU Clusters. <https://github.com/icloud-ecnu/Espresso>. 2025. Heterogeneous GPU allocation and stage placement for distributed training.

[83] Guo R B, Anand U, Daudjee K, et al. Zorse: Optimizing LLM Training Efficiency on Heterogeneous GPU Clusters [J]. arXiv preprint arXiv:2507.10392. 2025.

[84] Guo R B, Anand U, Chen A, et al. Cephalo: Harnessing Heterogeneous GPU Clusters for Training Transformer Models [C]. Proceedings of the International Conference on Supercomputing (ICS). 2025. arXiv:2411.01075, 2024.

[85] Zhang Z, Zheng S, Wang Y, et al. MiCS: Near-linear Scaling for Training Gigantic Model on Public Cloud [J]. Proceedings of the VLDB Endowment. 2022, 16 (1): 37–50.

[86] Chen Q, Hu Q, Ye Z, et al. AMSP: Super-Scaling LLM Training via Advanced Model States Partitioning [J]. arXiv preprint. 2023.

[87] Zhu L, et al. Vision-Language Pre-training: Basics, Recent Progress, and Future Outlook [J]. Pattern Recognition. 2024.

[88] Liu H, Li C, Wu Q, et al. LLaVA-NeXT: Improved Baselines with Visual Instruction Tuning [J]. arXiv preprint arXiv:2402.12975. 2024.

[89] Team G, et al. Gemini: A Family of Highly Capable Multimodal Models [J]. arXiv preprint arXiv:2312.11805. 2023.

[90] Chowdhery A, Narang S, Devlin J, et al. PaLM: Scaling Language Modeling with Pathways [J]. Journal of Machine Learning Research. 2023, 24. arXiv:2204.02311, 2022.

[91] Li M. Scaling Distributed Machine Learning with the Parameter Server [C]. Proceedings of the 2014 International Conference on Big Data Science and Computing. Beijing, China, 2014: 1–1.

[92] Kim S, Yu G I, Park H, et al. Parallax: Sparsity-aware Data Parallel Training of Deep Neural Networks [C]. Proceedings of the Fourteenth EuroSys Conference. Dresden, Germany, 2019: 1–15.

[93] Jacobs S A, Khabbazan M, Baydin A G, et al. DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models [J]. arXiv preprint arXiv:2309.14509. 2023.

[94] Li Z, Zhuang S, Guo S, et al. TeraPipe: Token-Level Pipeline Parallelism for Training Large-Scale Language Models [J]. arXiv preprint. 2021.

[95] Li Z, Zhuang S, Guo S, et al. TeraPipe: Token-Level Pipeline Parallelism for

Training Large-Scale Language Models [C]. Proceedings of the International Conference on Machine Learning (ICML). 2022.

[96] Li S, Hoefler T. Chimera: Efficiently Training Large-Scale Neural Networks with Bidirectional Pipelines [J]. Proceedings of the International Conference for High Performance Computing (SC). 2023.

[97] Li Z, et al. Hanayo: Wave-like Pipeline Parallelism for Efficient Large Model Training [J]. arXiv preprint. 2024.

[98] Qi P, Wan X, Huang G, et al. Zero Bubble Pipeline Parallelism [C]. International Conference on Learning Representations. 2024.

[99] DeepSeek-AI. DualPipe: Bidirectional Pipeline Parallelism for Large-Scale Training. <https://github.com/deepseek-ai/DualPipe>. 2025. Accessed: 2025.

[100] Liu Z, Lin Z, et al. Ring Attention with Blockwise Transformers for Near-Infinite Context [J]. arXiv preprint arXiv:2310.01889. 2024.

[101] DeepSeek-AI. DeepSeek-MoE: Scaling Mixture-of-Experts Language Models with Limited Resources [J]. arXiv preprint arXiv:2401.06066. 2024.

[102] Rasley J, et al. DeepSpeed: Extreme-Scale Model Training for Everyone [J]. Microsoft Research Blog. 2022.

[103] Rasley J, Rajbhandari S, Awni Y, et al. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters [J]. Proceedings of the ACM SIGKDD. 2020.

[104] Chen T, Xu B, Zhang C, et al. Training Deep Nets with Sublinear Memory Cost [J]. arXiv preprint arXiv:1604.06174. 2016.

[105] Korthikanti V, Casper J, Lym S, et al. Reducing Activation Recomputation in Large Transformer Models [C]. Proceedings of Machine Learning and Systems (MLSys). 2023.

[106] NVIDIA. Megatron-LM: Activation Recomputation [J]. NVIDIA Deep Learning Documentation. 2024.

[107] NVIDIA. NeMo Framework: Activation Recomputation [J]. NVIDIA NeMo Documentation. 2024.

[108] Li Z, Lin Z, Wang C, et al. Lynx: Efficient Memory Management for Large Model Training with Overlapped Activation Recomputation [J]. arXiv preprint arXiv:2406.08756. 2024.

[109] Liu Y, et al. Colossal-Auto: Unified Automation of Parallelization and Activation Checkpoint for Large-scale Models [J]. Journal of Machine Learning Research.

2023.

[110] Zheng L, et al. Efficient Large Language Model Training: A Survey [J]. arXiv preprint arXiv:2312.03863. 2024.

[111] Li S, et al. A Survey on Distributed Deep Learning [J]. IEEE Access. 2022.

[112] Jia X, et al. Whale: Efficient Giant Model Training over Heterogeneous GPUs [J]. USENIX ATC. 2022.

作者在学习期间取得的学术成果

发表的学术论文

- [1] J. Fan et al., "MMoH: Efficient Training of Multimodal Large Language Models on Heterogeneous Clusters," 2025 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA), Shenyang, China, 2025, pp. 990-997, doi: 10.1109/ISPA67752.2025.00135.(CCF-C 类会议)

附录 A 本文中英文对照表

表 A.1 给出正文常用术语的中英文对照；第二列在必要时附缩写或原文记号。

表 A.1 本文主要术语中英文对照表

中文	英文
自注意力机制	Self-Attention
深度神经网络	Deep Neural Network (DNN)
卷积神经网络	Convolutional Neural Network (CNN)
循环神经网络	Recurrent Neural Network (RNN)
计算机视觉	Computer Vision (CV)
自然语言处理	Natural Language Processing (NLP)
语音识别	Speech Recognition (SR)
大语言模型	Large Language Model (LLM)
多模态大语言模型	Multimodal Large Language Model (MLLM)
通用人工智能	Artificial General Intelligence (AGI)
迁移学习	Transfer Learning
少样本学习	Few-Shot Learning
零样本学习	Zero-Shot Learning
扩展定律	Scaling Laws
图文对比学习 CLIP	Contrastive Language-Image Pre-Training
最优先进水平	state-of-the-art (SOTA)
模态编码器	Modality Encoder
输入投影层	Input Projector
LLM 基座	LLM Backbone
输出投影层	Output Projector
模态生成器	Modality Generator
扩散模型	diffusion model
模块异构	module heterogeneity
模态对齐	modality alignment
指令微调	instruction tuning
数据并行	data parallelism (DP)

续下页

续表 A.1

中文	英文
分布式数据并行	Distributed Data Parallel (DDP)
张量并行	tensor parallelism (TP)
流水线并行	pipeline parallelism (PP)
序列并行	sequence parallelism (SP)
上下文并行	context parallelism (CP)
专家并行	expert parallelism (EP)
混合专家	Mixture of Experts (MoE)
混合并行	hybrid parallelism
微批次	micro-batch
小批量	mini-batch
一前向一反向调度	One-Forward-One-Backward (1F1B)
全归约	AllReduce
全收集	All-Gather
归约分散	Reduce-Scatter
全对全通信	All-to-All
点对点通信	Peer-to-Peer (P2P)
参数服务器	parameter server
流水级	pipeline stage
流水线气泡	pipeline bubble
混合精度训练	mixed-precision training
全分片数据并行	Fully Sharded Data Parallel (FSDP)
零冗余优化器	Zero Redundancy Optimizer (ZeRO)
环形注意力	Ring Attention
激活 / 激活值	activation
显存溢出	Out Of Memory (OOM)
负载均衡	load balancing
剖析 / 性能剖析	profiling
设备拓扑	device topology
整数规划	integer programming
复合粒度（模型抽象）	multi-granularity model abstraction

续下页

续表 A.1

中文	英文
成本模型	cost model
服务等级目标	Service Level Objective (SLO)
模型算力利用率	Model FLOPs Utilization (MFU)
虚拟工作节点	Virtual Worker (VW)
剖析器	Profiler
GPU 分配器	GPU Allocator
阶段放置器	Stage Placer
全部重计算	full recomputation
选择重计算	selective recomputation
细粒度重计算	fine-grained recomputation
Top- k 专家路由	Top- k (expert routing)